

مبانی و الگوریتم‌های کلان‌داده

جزوه جامع درسی و مرجع دانشگاهی تحلیل کلان‌داده

بر اساس سرفصل‌ها و اسلایدهای کتاب (MMDS) Datasets Massive of Mining
تألیف: ژوره لسکوک، آناند راجارامان، جفری اولمن

گردآورنده: احسان شهبازی

مجموعه یادداشتهای آموزشی کارشناسی ارشد

۷ مهر ۱۴۰۵

پیش‌گفتار: کلان‌داده و مبانی محاسبات مقیاس‌پذیر

مقدمه و انگیزه

در دهه‌های اخیر، با رشد تصاعدی وب، شبکه‌های اجتماعی، حسگرهای اینترنت اشیا^۱، معاملات مالی و سامانه‌های پایش علمی، جوامع بشری وارد عصری شده‌اند که حجم و سرعت تولید داده‌ها از توان تحلیلی و ذخیره‌سازی ابزارهای سنتی پایگاه داده فراتر رفته است. این پدیده تحت عنوان **کلان‌داده**^۲ شناخته می‌شود. مواجهه با کلان‌داده صرفاً یک مسئله ارتقای سخت‌افزاری نیست؛ بلکه نیازمند بازنگری بنیادین در نظریه الگوریتم‌ها، مدل‌های ریاضی و معماری سیستم‌های محاسباتی است. الگوریتم‌هایی که دارای مرتبه زمانی درجه دوم ($O(n^2)$) یا حتی درجه اول خطی همراه با دسترسی تصادفی به حافظه هستند، بر روی کلان‌داده‌هایی با مقیاس ترابایت و پتابایت عملاً ناکارآمد بوده و متوقف می‌شوند. از این رو، چارچوب نظری جدیدی لازم است که الگوریتم‌های زیرخطی^۳، الگوریتم‌های برخط جریانی^۴، روش‌های تصادفی‌سازی و تقریب^۵، و پردازش موازی و توزیع‌شده^۶ را در کانون توجه قرار دهد.

مرجع اصلی و رویکرد آموزشی این اثر

این کتاب و مجموعه یادداشت‌های جامع درسی بر پایه سرفصل‌های رسمی دوره کارشناسی ارشد مهندسی کامپیوتر و علم داده و با تکیه بر مرجع پیشگام و معتبر دانشگاه استنفورد، یعنی کتاب:

[۱۰] (MMDS) Datasets Massive of Mining

Ullman D. Jeffrey Rajaraman, Anand Leskovec, Jure

تدوین شده است. در تدوین این مجموعه، اصول آموزشی زیر به عنوان راهنمای عمل مد نظر قرار گرفته‌اند:

۱. **محوریت اسلایدهای درسی در ساختار و جهت‌گیری:** ساختار هر فصل، تقدم و تاخر مباحث و میزان تاکید بر مفاهیم دقیقاً منطبق بر اسلایدهای رسمی درس تنظیم شده است تا دانشجویان پیوستگی کاملی میان جلسات درس و متن مکتوب احساس کنند.

۲. **جامعیت و خودپسندگی با بهره‌گیری از متن مرجع:** اسلایدهای درسی به دلیل ماهیت ارائه شفاهی، غالباً موجز و حاوی نکات کلیدی هستند. در این اثر، با رجوع دقیق به متن کتاب مرجع، کلیه تعاریف ریاضی، اثبات‌های راهگشا، تحلیل پیچیدگی الگوریتم‌ها و مثال‌های عددی گام‌به‌گام تشریح شده‌اند به گونه‌ای که مطالعه این کتاب بی‌نیاز از بازخوانی مدام اسلایدها باشد.

۳. **نگارش دانشگاهی و اصطلاح‌شناسی دقیق:** متن با نگارش علمی، روان و استاندارد فارسی نگاشته شده و برای تمامی اصطلاحات تخصصی، در اولین کاربرد، معادل انگلیسی آن‌ها در پاورقی ذکر شده است تا هم غنای علمی زبان فارسی حفظ شود و هم ارتباط دانشجویان با مقالات و ادبیات بین‌المللی گسسته نگردد.

۴. **تمرکز بر بینش شهودی و محاسبات واقعی:** ریاضیات و فرمول‌ها هرگز بدون توضیح شهودی رها نشده‌اند؛ برای هر رابطه ریاضی، معنای هر متغیر، دلیل بهره‌گیری از فرمول و چگونگی تفسیر نتایج تشریح گردیده است.

^۱Internet of Things (IoT)

^۲Big Data

^۳Sublinear Algorithms

^۴Streaming Algorithms

^۵Randomized and Approximation Algorithms

^۶Distributed and Parallel Computing

نقشه راه فصول کتاب

محتوای این اثر در سیزده فصل اصلی و بر اساس یک جریان منطقی منسجم به شرح زیر سازمان‌دهی شده است:

- **فصل ۱: داده‌کاوی و فضاهای با بعد بالا** – بررسی ماهیت داده‌کاوی، مقایسه رویکرد یادگیری ماشین در برابر مدل‌های آماری، اصل بونفرونی^۷ و اجتناب از کشف الگوهای کاذب، و چالش‌های محاسباتی در فضاهای ابعادی بزرگ.
- **فصل ۲: نگاشت-کاهش و پشته نرم‌افزاری** – معماری سیستم‌های فایل توزیع‌شده (مانند HDFS)، مدل برنامه‌نویسی نگاشت-کاهش^۸، شیوه‌های مدیریت خطا، و پیاده‌سازی عملیات ماتریسی و جبر رابطه‌ای در مقیاس وسیع.
- **فصل ۳: یافتن آیتم‌های مشابه** – مسئله مقایسه اسناد در وب، روش شینگلینگ^۹، معیار تشابه جاکارد^{۱۰}، تکنیک Min-Hashing، و درهم‌سازی حساس به مکان^{۱۱} جهت کاهش ابعاد جستجو.
- **فصل ۴: کاوش جریان‌های داده** – مدل محاسباتی جریان داده، نمونه‌گیری مخزنی^{۱۲}، فیلترهای بلوم^{۱۳}، تخمین تعداد اعضای متمایز با الگوریتم فلزوله-مارتین^{۱۴}، و شمارش بیت‌های یک در پنجره لغزان با الگوریتم DGIM.
- **فصل ۵: تحلیل پیوندها و رتبه‌بندی وب** – ساختار گراف وب، الگوریتم مشهور رتبه‌بندی صفحه^{۱۵}، تله‌های عنکبوتی و بن‌بست‌ها، راهکار بازآغازی تصادفی، رتبه‌بندی وابسته به موضوع، و الگوریتم HITS.
- **فصل ۶: مجموعه‌آیتم‌های پرتکرار** – مدل سبد بازار^{۱۶}، قوانین انجمنی^{۱۷}، الگوریتم A-Priori و بهینه‌سازی‌های حافظه‌محور نظیر PCY و Multistage، و الگوریتم‌های توزیع‌شده SON و Toivonen.
- **فصل ۷: خوشه‌بندی در ابعاد بالا** – محدودیت‌های روش‌های خوشه‌بندی سنتی در ابعاد بالا، الگوریتم‌های سلسله‌مراتبی، K-Means و بهینه‌سازی‌های BFR و CURE برای خوشه‌بندی داده‌های عظیم با شکل‌های پیچیده.
- **فصل ۸: تبلیغات برخط در وب** – مدل تطابق دوبخشی برخط^{۱۸}، نسبت رقابتی^{۱۹}، الگوریتم BALANCE و سازوکار حراج آگهی گوگل (AdWords).
- **فصل ۹: سامانه‌های توصیه‌گر** – ماتریس سودمندی^{۲۰}، فیلترینگ محتوامحور، فیلترینگ مشارکتی^{۲۱}، و تجزیه ماتریسی با کاهش مقدار منفرد (SVD) و فاکتورسازی ماتریس UV.
- **فصل ۱۰: کاوش گراف‌های شبکه‌های اجتماعی** – ساختار جوامع در شبکه‌های اجتماعی، مرکزیت بینابینی و الگوریتم گیروان-نیومن^{۲۲}، معیار پیمانه‌ای^{۲۳}، و خوشه‌بندی طیفی^{۲۴} با ماتریس لاپلاسی.
- **فصل ۱۱: کاهش بعد** – تجزیه مقادیر منفرد^{۲۵}، تحلیل مؤلفه‌های اصلی^{۲۶}، لم جانسون-لیندن‌اشترواس^{۲۷}، و تجزیه ماتریسی CUR.

-
- Bonferroni's Principle^۷
 - MapReduce^۸
 - Shingling^۹
 - Jaccard Similarity^{۱۰}
 - Locality-Sensitive Hashing (LSH)^{۱۱}
 - Reservoir Sampling^{۱۲}
 - Bloom Filters^{۱۳}
 - Flajolet-Martin Algorithm^{۱۴}
 - PageRank^{۱۵}
 - Market-Basket Model^{۱۶}
 - Association Rules^{۱۷}
 - Online Bipartite Matching^{۱۸}
 - Competitive Ratio^{۱۹}
 - Utility Matrix^{۲۰}
 - Collaborative Filtering^{۲۱}
 - Girvan-Newman Algorithm^{۲۲}
 - Modularity^{۲۳}
 - Spectral Clustering^{۲۴}
 - Singular Value Decomposition (SVD)^{۲۵}
 - Principal Component Analysis (PCA)^{۲۶}
 - Johnson-Lindenstrauss Lemma^{۲۷}

- **فصل ۱۲: یادگیری ماشین در مقیاس بزرگ** – ماشین‌های بردار پشتیبان^{۲۸}، تابع هزینه لولا^{۲۹}، گرادیان کاهشی تصادفی^{۳۰} با یک گذر، و موازی‌سازی الگوریتم‌های یادگیری.
- **فصل ۱۳: شبکه‌های عصبی و یادگیری عمیق** – پرسپترون چندلایه، پس‌انتشار خطا^{۳۱}، شبکه‌های پیچشی (CNN) و ساختارهای توجه و ترنسفورمرها در کلان‌داده و پردازش‌های توزیع‌شده GPU.

راهنمای مطالعه و آمادگی برای آزمون

برای بهره‌گیری حداکثری از این کتاب توصیه می‌شود:

۱. در ابتدای هر فصل، بخش «مقدمه و اهداف فصل» را مرور فرمایید تا نقشه مفهومی مبحث در ذهن شکل گیرد.
۲. به کادرهای مشخص‌شده با رنگ‌های مختلف توجه فرمایید:
 - کادرهای **آبی** بیانگر تعاریف پایه و اصطلاحات رسمی هستند.
 - کادرهای **سبز** قضایا، لم‌ها و گزاره‌های ریاضی اثبات‌شده هستند.
 - کادرهای **خاکستری** مشخصات گام‌به‌گام و شبه‌کد الگوریتم‌ها هستند.
 - کادرهای **نارنجی** مثال‌های عددی حل‌شده را در بر دارند.
 - کادرهای **قرمز** حاوی نکات طلایی امتحانی و دام‌های متداول مفهومی می‌باشند.
۳. مثال‌های عددی را به صورت دستی روی کاغذ دنبال کنید تا مهارت لازم در آزمون‌ها و تمرین‌های عملی حاصل گردد.

شناسنامه علمی و عوامل تدوین اثر

- **عنوان کتاب:** مبانی و الگوریتم‌های کلان‌داده (Mining of Massive Datasets)
- **گردآورنده و پژوهشگر:** احسان شهبازی (Ehsan Shahbazi)
- **پست الکترونیک:** ehsan.shahbazipc@gmail.com
- **مخزن و صفحه رسمی گیت‌هاب:** github.com/EhsanShahbazi
- **مرجع درسی:** کتاب Mining of Massive Datasets، دانشگاه استنفورد
- **مقطع و جامعه مخاطب:** دانشجویان تحصیلات تکمیلی مهندسی کامپیوتر، هوش مصنوعی و علم داده

^{۲۸} Support Vector Machines (SVM)
^{۲۹} Hinge Loss
^{۳۰} Stochastic Gradient Descent (SGD)
^{۳۱} Backpropagation

فهرست مطالب

آ	پیش‌گفتار: کلان‌داده و مبانی محاسبات مقیاس‌پذیر
۱	اول مبانی کلان‌داده و رایانش توزیع‌شده
۲	۱ داده‌کاوی و فضاهای با بعد بالا
۲	۱.۱ مقدمه و تعاریف پایه
۳	۲.۱ داده‌کاوی در تقاطع پایگاه‌های داده، یادگیری ماشین و نظریه الگوریتم‌ها
۳	۳.۱ محدودیت‌های آماری در داده‌کاوی و اصل بونفرونی
۵	۴.۱ مفاهیم کمکی و ابزارهای ضروری برای کلان‌داده
۵	۱.۴.۱ معیار اهمیت کلمات در اسناد: TF-IDF
۵	۲.۴.۱ توابع درهم‌سازی و رفتار آن‌ها
۵	۳.۴.۱ قوانین توانی و مفهوم دنباله طولانی
۷	۲ نگاشت-کاهش و پشته نرم‌افزاری پردازش توزیع‌شده
۷	۱.۲ مقدمه: چالش‌های معماری در مقیاس وب
۷	۲.۲ سیستم‌های فایل توزیع‌شده (DFS / GFS / HDFS)
۸	۳.۲ مدل برنامه‌نویسی نگاشت-کاهش (MapReduce)
۹	۴.۲ بهینه‌سازی‌های اجرایی در نگاشت-کاهش
۹	۱.۴.۲ تابع ترکیب‌کننده (Combiner)
۹	۲.۴.۲ تابع افرازبندی (Partitioner)
۹	۳.۴.۲ مدیریت گره‌های کند (Stragglers) با اجرای گمانه‌زنانه
۱۰	۵.۲ پیاده‌سازی اعمال جبر رابطه‌ای در MapReduce
۱۰	۱.۵.۲ پیوند طبیعی (Natural Join)
۱۰	۲.۵.۲ گروه‌بندی و تجمیع (Grouping and Aggregation)
۱۰	۳.۵.۲ ضرب ماتریس در بردار (Matrix-Vector Multiplication)
۱۱	۴.۵.۲ ضرب دو ماتریس بزرگ در نگاشت-کاهش (Matrix Multiplication)
۱۲	۳ یافتن آیت‌های مشابه و درهم‌سازی حساس به مکان (LSH)
۱۲	۱.۳ مقدمه و تعریف مسئله
۱۳	۲.۳ گام نخست: شینگلینگ اسناد
۱۳	۱.۲.۳ انتخاب اندازه مناسب برای k
۱۳	۳.۳ معیار تشابه جاکارد و ماتریس مشخصه
۱۴	۱.۳.۳ ماتریس مشخصه (Characteristic Matrix)
۱۵	۴.۳ گام دوم: کمینه‌سازی درهم‌سازی (Min-Hashing)
۱۵	۱.۴.۳ جایگشت سطرها و تابع Min-Hash
۱۵	۲.۴.۳ الگوریتم شبیه‌سازی Min-Hash بدون اعمال جایگشت فیزیکی
۱۶	۵.۳ گام سوم: درهم‌سازی حساس به مکان (LSH) برای امضاها
۱۶	۱.۵.۳ روش دسته‌بندی نواری (Banding Technique)
۱۶	۲.۵.۳ تحلیل ریاضی منحنی اس-شکل (S-Curve Analysis)
۱۷	۶.۳ معیارهای فاصله و خانواده‌های درهم‌سازی حساس به مکان

۱۹

دوم کاوش جریان‌های داده و وب

۴ کاوش جریان‌های داده (Data Streams)

۲۰	۱.۴	مقدمه و مدل محاسباتی جریان داده
۲۰	۲.۴	نمونه‌گیری از جریان‌های داده
۲۱	۱.۲.۴	نمونه‌گیری با نسبت ثابت (Fixed Proportion Sampling)
۲۱	۲.۲.۴	نمونه‌گیری با اندازه ثابت: الگوریتم مخزنی (Reservoir Sampling)
۲۲	۳.۴	فیلتر کردن جریان‌های داده: فیلتر بلوم (Bloom Filter)
۲۳	۱.۳.۴	تحلیل احتمال خطای مثبت کاذب و مقدار بهینه k
۲۳	۴.۴	شمارش عناصر متمایز: الگوریتم فلزوله-مارتین (Flajolet-Martin)
۲۴	۱.۴.۴	تکنیک کاهش واریانس: میانگین میانه‌ها (Median of Means)
۲۴	۵.۴	شمارش بیت‌ها در پنجره لغزان: الگوریتم DGIM
۲۵	۶.۴	تخمین گشتاورهای جریان: الگوریتم آلون-ماتیس-سگدی (AMS)
۲۶	۷.۴	پنجره‌های با کاهش نمایی (Decaying Windows)

۵ تحلیل پیوندها و رتبه‌بندی وب (Link Analysis & PageRank)

۲۸	۱.۵	مقدمه: گراف وب و چالش بازیابی اطلاعات
۲۸	۲.۵	ساختار کلان گراف وب: مدل پایبونی (Bow-Tie Model)
۲۹	۳.۵	فرمول‌بندی ریاضی الگوریتم PageRank
۲۹	۱.۳.۵	بیان ماتریسی و بردار ویژه
۲۹	۲.۳.۵	الگوریتم تکرار توانی (Power Iteration)
۳۰	۴.۵	ناهنجاری‌های گراف وب و راهکارهای اصلاحی
۳۰	۱.۴.۵	۱. تله‌های عنکبوتی (Spider Traps)
۳۰	۲.۴.۵	۲. بن‌بست‌ها (Dead Ends)
۳۱	۵.۵	الگوریتم نهایی گوگل با تله‌پورتیشن (مالیات‌گیری)
۳۱	۶.۵	الگوریتم‌های پیشرفته: رتبه‌بندی حساس به موضوع و TrustRank
۳۱	۱.۶.۵	PageRank وابسته به موضوع (Topic-Sensitive PageRank)
۳۲	۲.۶.۵	مبارزه با مزارع اسپم با TrustRank
۳۲	۷.۵	الگوریتم قطب‌ها و مراجع (HITS)

۶ مجموعه‌آیتم‌های پرتکرار و قوانین انجمنی

۳۴	۱.۶	مقدمه و مدل سبد بازار (Market-Basket Model)
۳۴	۲.۶	تعاریف پایه: پشتیبانی، اطمینان و قوانین انجمنی
۳۵	۳.۶	اصل یکنوایی و الگوریتم A-Priori
۳۶	۱.۳.۶	مراحل اجرای الگوریتم A-Priori
۳۶	۴.۶	الگوریتم‌های بهینه‌سازی مصرف حافظه
۳۶	۱.۴.۶	الگوریتم پارک-چن-یو (PCY)
۳۷	۲.۴.۶	الگوریتم‌های چندمرحله‌ای و چنددرهم‌سازی (Multistage & Multihash)
۳۷	۵.۶	الگوریتم‌های با تعداد گذر محدود (یک یا دو گذر)
۳۷	۱.۵.۶	الگوریتم ساواسره-امینسکی-ناواته (SON)
۳۷	۲.۵.۶	الگوریتم توینون (Toivonen's Algorithm) و مرز منفی

۳۹

سوم خوشه‌بندی و سامانه‌های وب

۷ خوشه‌بندی در فضاهای با بعد بالا (High-Dimensional Clustering)

۴۰	۱.۷	مقدمه و چالش‌های خوشه‌بندی کلان‌داده
۴۰	۲.۷	خوشه‌بندی سلسله‌مراتبی تراکمی (Agglomerative)
۴۰	۱.۲.۷	معیارهای فاصله میان دو خوشه
۴۱	۳.۷	الگوریتم K-Means و نسخه بهینه K-Means++
۴۱	۱.۳.۷	الگوریتم K-Means++ برای مقاردهی اولیه هوشمند
۴۲	۴.۷	خوشه‌بندی کلان‌داده در فضای اقلیدسی: الگوریتم BFR
۴۲	۱.۴.۷	فرضیات بنیادین BFR
۴۲	۲.۴.۷	خلاصه‌سازی داده‌ها با آماره‌های سه‌گانه $(N, SUM, SUMSQ)$

۴۲	سه مجموعه داده در حافظه اصلی	۳.۴.۷
۴۲	معیار فاصله ماهالانوبیس (Mahalanobis Distance)	۴.۴.۷
۴۳	خوشه‌بندی اشکال دلخواه و غیرکروی: الگوریتم CURE	۵.۷
۴۳	خوشه‌بندی در فضاهای غیر اقلیدسی: مفهوم شبه‌مرکز (Clustroid)	۶.۷

۸ تبلیغات برخط در وب (Online Advertising & AdWords)

۴۵	مقدمه: الگوریتم‌های برخط و اقتصاد موتورهای جستجو	۱.۸
۴۵	مسئله تطابق دوبخشی برخط (Online Bipartite Matching)	۲.۸
۴۶	الگوریتم حریصانه (Greedy Algorithm) و تحلیل نسبت رقابتی	۱.۲.۸
۴۶	الگوریتم BALANCE با پیشنهادهای برابر	۳.۸
۴۶	مسئله عمومی AdWords در گوگل (پیشنهادهای نابرابر)	۴.۸
۴۷	تابع امتیازدهی عمومی AdWords	۱.۴.۸
۴۷	نرخ کلیک (CTR) و رتبه‌بندی آگهی‌ها	۲.۴.۸
۴۷	سازوکار حراج قیمت دوم (Vickrey-Clarke-Groves Auction)	۳.۴.۸

۹ سامانه‌های توصیه‌گر (Recommender Systems)

۴۹	مقدمه و پدیده دنباله طولانی	۱.۹
۴۹	ماتریس سودمندی (Utility Matrix)	۲.۹
۴۹	روش‌های مبتنی بر محتوا (Content-Based Recommendation)	۳.۹
۴۹	پروفایل آیتم‌ها و پروفایل کاربر	۱.۳.۹
۵۰	فیلترینگ مشارکتی (Collaborative Filtering)	۴.۹
۵۰	کسینوس مرکزی (همبستگی پیرسون)	۱.۴.۹
۵۱	پیش‌بینی امتیاز کاربر به آیتم (فیلترینگ کاربر-کاربر)	۲.۴.۹
۵۱	فیلترینگ آیتم-آیتم در برابر کاربر-کاربر	۳.۴.۹
۵۱	مدل‌های عامل پنهان و فاکتورسازی ماتریسی (Matrix Factorization)	۵.۹
۵۱	تابع هدف بهینه‌سازی و منظم‌سازی (Regularization)	۱.۵.۹
۵۲	بهینه‌سازی با گرادینت تصادفی (SGD)	۲.۵.۹
۵۲	مدل پایه با اثرات انحراف (Baseline Predictors)	۳.۵.۹

چهارم تحلیل شبکه‌های اجتماعی و کاهش بعد

۱۰ کاوش گراف‌های شبکه‌های اجتماعی (Social-Network Graphs)

۵۴	مقدمه و ویژگی‌های توپولوژیک شبکه‌های اجتماعی	۱.۱۰
۵۴	تشخیص جوامع و مرکزیت بینابینی یال‌ها	۲.۱۰
۵۵	الگوریتم گیروان-نیومن (Girvan-Newman Algorithm)	۱.۲.۱۰
۵۵	معیار پیمانه‌ای (Modularity)	۳.۱۰
۵۶	خوشه‌بندی طیفی گراف‌ها (Spectral Clustering)	۴.۱۰
۵۶	ماتریس لاپلاسیان گراف (Graph Laplacian)	۱.۴.۱۰
۵۶	برداری فیدلر (Fiedler Vector) و افراز گراف	۲.۴.۱۰

۱۱ روش‌های کاهش بعد (Dimensionality Reduction)

۵۸	مقدمه و انگیزه کاهش بعد	۱.۱۱
۵۸	تجزیه مقادیر منفرد (Singular Value Decomposition - SVD)	۲.۱۱
۵۹	تفسیر هندسی و فیزیکی SVD	۱.۲.۱۱
۵۹	تقریب با رتبه پایین و قضیه اکارت-یانگ-میرسکی	۲.۲.۱۱
۵۹	تحلیل مؤلفه‌های اصلی (Principal Component Analysis - PCA)	۳.۱۱
۶۰	تجزیه ماتریسی CUR: حفظ تنک بودن و تفسیرپذیری	۴.۱۱
۶۰	الگوریتم انتخاب سطرها و ستون‌ها بر مبنای امتیاز اهرمی	۱.۴.۱۱
۶۱	تصویرسازی‌های تصادفی و لم جانسون-لیندن اشتراوس (JL Lemma)	۵.۱۱

۶۳

پنجم یادگیری در مقیاس بزرگ و یادگیری عمیق

۶۴

۱۲ یادگیری ماشین در مقیاس بزرگ (Large-Scale Machine Learning)

۶۴	۱.۱۲	مقدمه و چالش‌های یادگیری ماشین در کلان‌داده
۶۴	۲.۱۲	روش نزدیک‌ترین همسایه در مقیاس بزرگ (k -NN)
۶۴	۳.۱۲	طبقه‌بندی خطی و الگوریتم پرسپترون (Perceptron)
۶۵	۴.۱۲	ماشین‌های بردار پشتیبان در مقیاس بزرگ (SVM)
۶۶	۱.۴.۱۲	فرم نرم حاشیه و تابع هزینه لولا (Hinge Loss)
۶۶	۵.۱۲	بهینه‌سازی در مقیاس بزرگ با گرادینت تصادفی (SGD)
۶۷	۶.۱۲	موازی‌سازی یادگیری ماشین در نگاشت-کاهش

۶۸

۱۳ شبکه‌های عصبی و یادگیری عمیق (Neural Networks & Deep Learning)

۶۸	۱.۱۳	مقدمه و گذار از پرسپترون به یادگیری عمیق
۶۸	۲.۱۳	معماری پرسپترون چندلایه (MLP)
۶۹	۱.۲.۱۳	انتشار رو به جلو (Forward Propagation)
۶۹	۳.۱۳	توابع فعال‌ساز (Activation Functions)
۷۰	۴.۱۳	الگوریتم پس‌انتشار خطا (Backpropagation)
۷۰	۵.۱۳	شبکه‌های عصبی پیچشی (CNN) برای داده‌های مکانی
۷۱	۶.۱۳	شبکه‌های بازگشتی (RNN) و ترنسفورمرها برای داده‌های متوالی و متنی
۷۱	۷.۱۳	زیرساخت‌های توزیع‌شده برای آموزش یادگیری عمیق در مقیاس بزرگ

بخش اول

مبانی کلان داده و رایانش توزیع شده

فصل ۱

داده‌کاوی و فضاهای با بعد بالا

۱.۱ مقدمه و تعاریف پایه

در دنیای مدرن، سازمان‌ها، موتورهای جستجو، شبکه‌های اجتماعی و سامانه‌های علمی با حجم عظیمی از داده‌های دیجیتال روبرو هستند. گردآوری و ذخیره‌سازی داده‌ها گام نخست است، اما ارزش واقعی داده‌ها زمانی پدیدار می‌شود که بتوان از آن‌ها ^۱ دانش و بینش عملیاتی استخراج نمود. رشته علمی و مهندسی که به این مقصود اختصاص دارد، **داده‌کاوی** ^۲ نامیده می‌شود.

تعریف ۱.۱: داده‌کاوی

داده‌کاوی به فرآیند کشف خودکار یا نیمه‌خودکار الگوها، مدل‌ها و همبستگی‌های پنهان در مجموعه داده‌های عظیم اطلاق می‌شود، به گونه‌ای که الگوهای به‌دست‌آمده دارای ویژگی‌های زیر باشند:

۱. **معتبر بودن**^a: بر روی داده‌های جدید نیز با درجه اطمینان مشخصی صادق باشد (پرهیز از بیش‌برازش).
۲. **سودمند بودن**^b: بتوان بر مبنای آن تصمیمی عملیاتی یا تجاری اتخاذ کرد.
۳. **غیربدیهی و غیرمنتظره بودن**^c: صرفاً تکرار بدیهیات و روابط از پیش شناخته‌شده نباشد.
۴. **قابل فهم بودن برای انسان**^d: ساختار کشف‌شده توسط تحلیل‌گران قابل تفسیر باشد.

Valid^a
Useful^b
Unexpected^c
Understandable^d

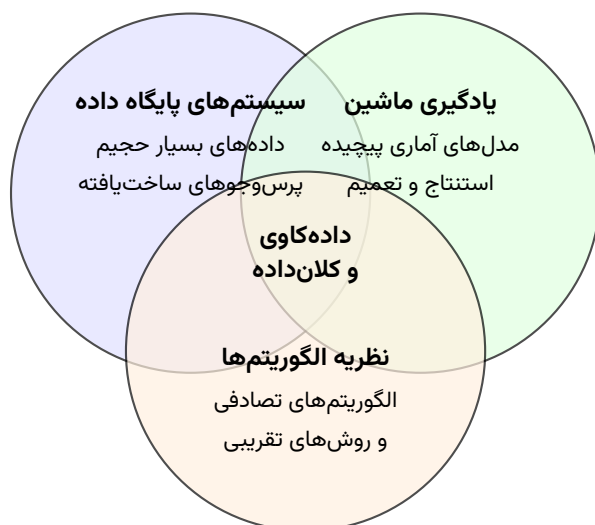
از منظر اهداف تحلیلی، روش‌های داده‌کاوی غالباً به دو دسته بزرگ تقسیم می‌شوند:

- **روش‌های توصیفی**^۳: یافتن الگوهای ساختاری که داده‌های موجود را به شیوه‌ای قابل درک برای انسان توصیف و دسته‌بندی می‌کنند. شاخص‌ترین نمونه این روش‌ها، **خوشه‌بندی**^۴ و **کاوش قوانین انجمنی**^۵ است.
- **روش‌های پیش‌بینانه**^۶: استفاده از متغیرهای معلوم برای پیش‌بینی مقادیر مجهول یا رخداد‌های آینده. نمونه‌های بارز آن شامل الگوریتم‌های **طبقه‌بندی**^۷، **رگرسیون**^۸ و **سامانه‌های توصیه‌گر**^۹ است.

Knowledge^۱
Data Mining^۲
Descriptive Methods^۳
Clustering^۴
Association Rules^۵
Predictive Methods^۶
Classification^۷
Regression^۸
Recommender Systems^۹

۲.۱ داده‌کاوی در تقاطع پایگاه‌های داده، یادگیری ماشین و نظریه الگوریتم‌ها

داده‌کاوی رشته‌ای مجزا و در عین حال پیونددهنده میان چند حوزه کلاسیک در علوم کامپیوتر است (شکل ۱.۱):



شکل ۱.۱: جایگاه داده‌کاوی در تقاطع پایگاه داده، یادگیری ماشین و نظریه الگوریتم‌ها

تفاوت فرهنگ‌های علمی در این زمینه‌ها شایان توجه است:

۱. دیدگاه پایگاه داده^{۱۰}: در این دیدگاه، داده‌کاوی به مثابه یک پردازش تحلیلی در مقیاس بسیار وسیع (OLAP) تلقی می‌شود؛ هدف، اجرای بهینه پرس‌وجوهایی است که میلیون‌ها رکورد را غربال می‌کنند و خروجی نهایی، پاسخ صریح همان پرس‌وجو است.
۲. دیدگاه یادگیری ماشین^{۱۱}: در این فرهنگ، هدف استنتاج یک مدل ریاضی و تخمین پارامترهای آن مدل است. خروجی نهایی، بردار وزن‌ها، درخت تصمیم یا ساختار شبکه عصبی است.
۳. دیدگاه کلان‌داده (MMDS)^{۱۲}: در درس کلان‌داده، ما هر دو دیدگاه را ادغام می‌کنیم، اما تأکید اصلی بر مقیاس‌پذیری^{۱۳}، معماری‌های محاسباتی^{۱۴} (نظیر پردازش موازی و جریانی) و پیچیدگی محاسباتی است.

۳.۱ محدودیت‌های آماری در داده‌کاوی و اصل بونفرونی

یکی از بزرگ‌ترین خطرات در کاوش کلان‌داده‌ها این است که تحلیل‌گر با جستجو در میان انبوهی از متغیرها، الگوهایی را کشف کند که کاملاً تصادفی و ساختگی هستند، اما از نظر ظاهری بسیار مهم جلوه می‌کنند. این چالش در علم آمار تحت عنوان اصل بونفرونی^{۱۴} شناخته می‌شود.

قضیه ۱.۱: اصل بونفرونی

اگر شما برای یافتن الگوهای جالب، فضایی بسیار بزرگ‌تر از آنچه که اندازه داده‌هایتان می‌تواند پشتیبانی کند را جستجو نمایید، با قطعیت ریاضی الگوهایی پیدا خواهید کرد که صرفاً حاصل تصادف هستند و هیچ رابطه علت و معلولی یا اعتبار تجربی در جهان واقعیت ندارند.

به بیان دقیق‌تر، اگر یک فرضیه وجود داشته باشد و احتمال رخداد تصادفی آن در یک آزمایش p باشد، چنانچه N آزمایش مستقل انجام دهیم، امید ریاضی تعداد دفعاتی که این فرضیه به طور تصادفی ارضا می‌شود برابر است با:

$$E = N \times p \quad (1.1)$$

^{۱۰} Database Culture

^{۱۱} Machine Learning Culture

^{۱۲} Scalability

^{۱۳} Computing Architectures

^{۱۴} Bonferroni's Principle

اگر $1 \ll N \times p$ باشد، مشاهده چنین الگویی نباید دلیلی بر کشف پدیده‌ای واقعی قلمداد شود، زیرا رخداد آن بر اساس تصادف صرف حتمی است.

مثال کاربردی ۱.۱: محاسبه هتل‌های مشکوک به سبک بونفرونی

فرض کنید سازمان امنیت ملی می‌خواهد افرادی را که مظنون به اقدامات هماهنگ تروریستی هستند با ردیابی محل اقامت آنها شناسایی کند. سناریوی زیر در اسلایدهای درس و فصل اول کتاب مطرح شده است:

- تعداد افراد تحت نظر: 10^9 نفر (یک میلیارد نفر).
 - بازه زمانی رصد: ۱۰۰۰ روز.
 - هر فرد به طور میانگین در ۱٪ روزها (یعنی از هر ۱۰۰ روز، ۱ روز) در یک هتل اقامت می‌کند. بنابراین در طول ۱۰۰۰ روز، هر فرد به طور تصادفی در ۱۰ روز به هتل می‌رود.
 - هر هتل گنجایش دقیقاً ۱۰۰ مسافر دارد.
 - چون در هر روز $10^9 \times 0.01 = 10^7$ نفر در هتل‌ها هستند، به تعداد $10^7 / 100 = 10^5$ هتل نیاز است.
 - تعریف مظنون: هر زوج از افراد نامرتب که حداقل در دو روز متمایز در یک هتل یکسان اقامت کرده باشند.
- پرسش: اگر همه افراد کاملاً رفتاری تصادفی داشته باشند و هیچ توطئه‌ای در کار نباشد، چه تعداد «زوج مظنون» در این داده‌ها ظاهر خواهد شد؟
حل گام‌به‌گام:

۱. تعداد کل زوج‌های ممکن:

$$\binom{10^9}{2} = \frac{10^9 \times (10^9 - 1)}{2} \approx 5 \times 10^{17} \text{ زوج} \quad (2.1)$$

۲. احتمال حضور دو فرد A و B در یک روز خاص در هتل یکسان: احتمال اینکه A در هتل باشد 0.01 ، و احتمال اینکه B در هتل باشد نیز 0.01 است. چون 10^5 هتل وجود دارد، احتمال اینکه هر دو در یک هتل خاص اقامت کنند برابر با $1/10^5$ است. بنابراین، احتمال اقامت همزمان در یک هتل در یک روز معین:

$$p_1 = 10^{-2} \times 10^{-2} \times 10^5 = 10^{-9} \quad (3.1)$$

۳. احتمال اقامت در دو روز یکسان: احتمال اینکه این دو نفر در دو روز خاص هم‌هتل شوند $p_1^2 = 10^{-18}$ است. تعداد جفت‌روزهای ممکن از میان ۱۰۰۰ روز برابر است با:

$$\binom{1000}{2} = \frac{1000 \times 999}{2} \approx 5 \times 10^5 \quad (4.1)$$

بنابراین، احتمال اینکه دو فرد تصادفی حداقل در دو روز در یک هتل اقامت داشته باشند:

$$p_2 = \binom{1000}{2} \times (10^{-9})^2 = (5 \times 10^5) \times 10^{-18} = 5 \times 10^{-13} \quad (5.1)$$

۴. امید ریاضی تعداد زوج‌های مظنون کاذب: تعداد زوج‌های مظنون مورد انتظار از حاصل ضرب تعداد کل زوج‌ها در احتمال p_2 حاصل می‌شود:

$$E = (5 \times 10^{17}) \times (5 \times 10^{-13}) = 250,000! \text{ زوج مظنون} \quad (6.1)$$

نتیجه‌گیری تحلیلی: در این داده‌ها، ۲۵۰ هزار جفت فرد کاملاً بی‌گناه شناسایی می‌شوند که تصادفاً دو بار در یک هتل اقامت داشته‌اند. اگر تیم امنیتی بخواهد تمامی این ۲۵۰ هزار زوج را بازجویی کند، دچار فلج عملیاتی خواهد شد. این مثال اثبات می‌کند که در کلان‌داده، یافتن همبستگی بدون پشتوانه آماری و شواهد کمکی، به نتایج فاجعه‌بار منجر می‌شود.

نکته امتحانی و کلیدی

در آزمون‌های درس کلان داده، مسائل مبتنی بر اصل بونفرونی بسیار پرکاربرد هستند. به خاطر بسپارید که کلید حل این مسائل در سه گام خلاصه می‌شود: ۱. محاسبه تعداد کل ترکیبات نامزدها (N) ؛ ۲. محاسبه احتمال رخداد تصادفی رویداد مورد نظر برای یک نامزد (p) ؛ ۳. محاسبه امید ریاضی $E = N \times p$. چنانچه $E \geq 1$ باشد، الگو غیرقابل استناد به عنوان ناهنجاری است.

۴.۱ مفاهیم کمکی و ابزارهای ضروری برای کلان داده

۱.۴.۱ معیار اهمیت کلمات در اسناد: TF-IDF

در بازیابی اطلاعات و کاوش متون، لازم است کلمات کلیدی اسناد را بدون دخالت کلمات پرتکرار و بی‌ارزش (نظیر حروف اضافه و ربط) شناسایی کرد. برای این منظور از شاخص TF-IDF^{۱۵} استفاده می‌شود. فرض کنید مجموعه‌ای از N سند متنی داریم. برای واژه t در سند d :

۱. فراوانی واژه (TF): نسبت تعداد دفعات تکرار واژه t در سند d ($f_{t,d}$) به بیشترین تعداد تکرار هر واژه‌ای در همان سند:

$$TF(t, d) = \frac{f_{t,d}}{\max_{w \in d} f_{w,d}} \quad (۷.۱)$$

این نرمال‌سازی مانع از آن می‌شود که اسناد طولانی‌تر صرفاً به خاطر طولشان امتیاز بالاتری دریافت کنند.

۲. معکوس فراوانی سند (IDF): اگر کلمه t در n_t سند از کل N سند ظاهر شده باشد، شاخص IDF آن به صورت زیر تعریف می‌شود:

$$IDF(t) = \log_p \left(\frac{N}{n_t} \right) \quad (۸.۱)$$

اگر کلمه‌ای در تمام اسناد تکرار شود ($n_t = N$)، آنگاه $\log_p(1) = 0$ شده و وزن آن کلمه صفر می‌گردد. در مقابل، کلماتی که فقط در چند سند خاص ظاهر شوند، IDF بالایی دریافت می‌کنند.

۳. شاخص ترکیبی TF-IDF:

$$TF-IDF(t, d) = TF(t, d) \times IDF(t) \quad (۹.۱)$$

۲.۴.۱ توابع درهم‌سازی و رفتار آن‌ها

توابع درهم‌سازی^{۱۶} ابزار بنیادین در تمام الگوریتم‌های کلان داده هستند. یک تابع درهم‌سازی $h(x)$ کلیدی دلخواه را به مقداری صحیح در بازه $[0, B-1]$ نگاشت می‌کند. در کلان داده، دو ویژگی حیاتی زیر مورد انتظار است:

- توزیع یکنواخت^{۱۷}: کلیدها به طور مساوی میان B سبد پخش شوند تا از تجمع بار جلوگیری گردد.
- شبه تصادفی بودن^{۱۸}: حتی اگر ورودی‌ها دارای الگوهای منظم باشند، مقادیر درهم‌سازی حاصل رفتاری مشابه متغیرهای تصادفی مستقل نشان دهند.

۳.۴.۱ قوانین توانی و مفهوم دنباله طولانی

در بسیاری از پدیده‌های وب و کلان داده (مانند توزیع درجات در گراف وب، تعداد خریدهای کالاها در آمازون، یا تعداد بازدید ویدئوها در یوتیوب)، داده‌ها از توزیع نرمال گاوسی پیروی نمی‌کنند، بلکه پیرو قانون توانی^{۱۹} هستند:

$$y = c \cdot x^k \quad (k < 0) \quad (۱۰.۱)$$

^{۱۵} Term Frequency - Inverse Document Frequency

^{۱۶} Hash Functions

^{۱۷} Uniform Distribution

^{۱۸} Pseudorandomness

^{۱۹} Power Law

که در مقیاس لگاریتمی به یک خط راست تبدیل می‌شود:

$$\log y = \log c + k \log x \quad (11.1)$$

این توزیع بیانگر پدیده **دنباله طولانی**^{۲۰} است؛ یعنی تعداد بسیار کمی از عناصر دارای فراوانی نجومی هستند (اقلیت پرطرفدار)، اما مجموع حجم عظیمی از داده‌ها در دنباله‌ای از عناصر کم‌تکرار قرار دارد. **قانون زیپف و توزیع پارتو**: یکی از شناخته‌شده‌ترین مصادیق قوانین توانی، **قانون زیپف**^{۲۱} است که بیان می‌دارد فراوانی عنصر با رتبه r ام در میان اقلام به صورت زیر رفتار می‌کند:

$$f(r) \propto \frac{1}{r^\alpha} \quad (\alpha \approx 1) \quad (12.1)$$

به عنوان مثال، پربسامدترین واژه در زبان انگلیسی («the») تقریباً دو برابر دومین واژه («of») و سه برابر سومین واژه («and») تکرار می‌شود. قاعده مشهور **پارتو ۸۰-۲۰** (که در آن ۸۰ درصد خریدهای یک فروشگاه تنها از ۲۰ درصد کالاهای پرفروش ناشی می‌شود) حالت خاصی از همین توزیع‌های توانی با نماهای شیب‌دار است. در سامانه‌های کلان داده، پردازش کارآمد داده‌های دنباله طولانی نیازمند الگوریتم‌های متمایز از پردازش اقلام ابرپرطرفدار است.

جمع‌بندی فصل اول

- داده‌کاوی به معنای کشف الگوهای معتبر، جدید، سودمند و قابل فهم در حجم انبوه داده‌ها است.
- اصل بونفرونی هشدار می‌دهد که جستجوی کورکورانه در میان تعداد حالات نجومی، قطعاً به کشف همبستگی‌های تصادفی و کاذب می‌انجامد.
- شاخص TF-IDF ابزاری اساسی برای ارزیابی اهمیت واژگان در اسناد است.
- توابع درهم‌سازی و قوانین توانی در سراسر فصول بعدی برای طراحی الگوریتم‌های مقیاس‌پذیر به کار گرفته خواهند شد.

فصل ۲

نگاشت-کاهش و پشته نرم‌افزاری پردازش توزیع‌شده

۱.۲ مقدمه: چالش‌های معماری در مقیاس وب

در سیستم‌های سنتی، برای پاسخ‌گویی به افزایش حجم محاسبات، راهکار ارتقای عمودی^۱ (افزودن پردازنده، حافظه یا دیسک‌های سریع‌تر به یک سرور واحد) به کار گرفته می‌شد. با این حال، در عصر کلان‌داده، حجم داده‌ها به ده‌ها و صدها ترابایت رسیده است؛ در نتیجه ارتقای عمودی به دلیل محدودیت‌های فیزیکی، گذرگاه حافظه و هزینه‌های نجومی توجیه‌پذیر نیست.

رویکرد نوین، **ارتقای افقی**^۲ است؛ یعنی اتصال هزاران رایانه معمولی و ارزان‌قیمت تجاری^۳ در قالب خوشه‌های محاسباتی^۴. در این معماری:

- ماشین‌ها در قفسه‌هایی^۵ شامل ۳۰ تا ۴۰ گره مستقر شده و با سوئیچ‌های محلی به یکدیگر و با سوئیچ‌های شبکه اصلی به سایر قفسه‌ها متصل می‌شوند.

- نرخ خرابی قطعات سخت‌افزاری (خرابی هارد دیسک، سوختن منبع تغذیه، یا قطع کابل شبکه) بسیار بالا است. اگر میانگین طول عمر یک ماشین ۳ سال (۱۰۰۰ روز) باشد، در یک خوشه با ۱۰۰۰ ماشین، به طور متوسط هر روز حداقل یک ماشین دچار خرابی کامل می‌شود.

بنابراین، دو اصل بنیادین در معماری نرم‌افزارهای کلان‌داده حاکم است:

۱. **تحمل خطای نرم‌افزاری**^۶: سیستم نرم‌افزاری باید به گونه‌ای طراحی شود که خرابی گره‌ها را به عنوان امری روزمره بپذیرد و بدون نیاز به آغاز مجدد کل محاسبات، وظایف گره خراب را به طور خودکار بازتولید کند.

۲. **نزدیک‌سازی محاسبات به داده**^۷: انتقال ترابایت‌ها داده در پهنای باند محدود شبکه خوشه غیرممکن است؛ بنابراین کد برنامه باید به گره‌هایی که قطعات داده روی دیسک آن‌ها ذخیره شده است فرستاده شود.

۲.۲ سیستم‌های فایل توزیع‌شده (DFS / GFS / HDFS)

سیستم‌های فایل توزیع‌شده نظیر GFS^۸ و پیاده‌سازی متن‌باز آن HDFS^۹، زیرساخت بنیادین ذخیره‌سازی داده‌های حجیم به صورت پایدار و توزیع‌شده هستند.

Scale-Up یا Scaling Vertical^۱
Scale-Out یا Scaling Horizontal^۲
Commodity Hardware^۳
Computing Clusters^۴
Racks^۵
Software-level Fault Tolerance^۶
Moving Computation to Data^۷
Google File System^۸
Hadoop Distributed File System^۹

تعریف ۱.۲: تکه یا بلوک داده

در سیستم فایل توزیع‌شده، فایل‌های بسیار بزرگ به قطعات یکنواختی به نام **تکه^a** با اندازه‌ای بین ۶۴ تا ۱۲۸ مگابایت تقسیم می‌شوند. هر تکه به طور پیش‌فرض بر روی **سه گره متمایز** (ترجیحاً در دو قفسه شبکه مختلف جهت تاب‌آوری در برابر قطع برق قفسه) تکثیر^b می‌گردد.

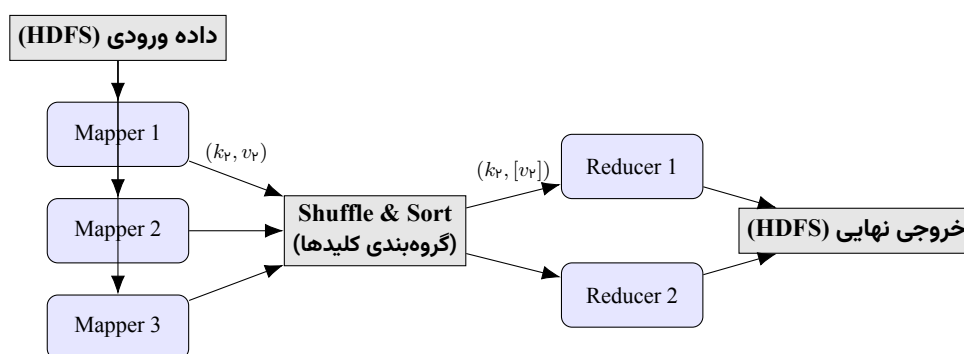
Block یا Chunk^a
Replication^b

معماری HDFS از دو بخش اصلی تشکیل شده است:

- **گره نام (NameNode):** متادیتا و شاخص محل ذخیره‌سازی تکه‌ها را در حافظه اصلی خود نگه می‌دارد. جهت کارایی بالا، هیچ داده خامی از مسیر گره نام عبور نمی‌کند.
- **گره‌های داده (DataNodes):** مسئولیت ذخیره‌سازی واقعی تکه‌های فایل روی دیسک محلی و ارائه مستقیم آن‌ها به کلاینت‌ها را بر عهده دارند.

۳.۲ مدل برنامه‌نویسی نگاشت-کاهش (MapReduce)

مدل نگاشت-کاهش توسط جفری دین و سانجای جماوات در گوگل معرفی شد [۶]. این مدل، پیچیدگی‌های موازی‌سازی، زمان‌بندی، تقسیم بار شبکه و مدیریت خرابی را از دید برنامه‌نویس پنهان ساخته و محاسبات را به دو تابع اصلی تقسیم می‌کند:



شکل ۱.۲: جریان اجرای الگوریتم MapReduce شامل مراحل نگاشت، بُر زدن و مرتب‌سازی، و کاهش

تعریف ۲.۲: امضای توابع نگاشت و کاهش

توابع Map و Reduce به شکل زیر فرمول‌بندی می‌شوند:

$$\text{Map}(k_1, v_1) \rightarrow \text{list}(k_2, v_2) \quad (1.2)$$

$$\text{Reduce}(k_2, \text{list}(v_2)) \rightarrow \text{list}(v_3) \quad (2.2)$$

در مرحله میانی که توسط زیرساخت اجرا می‌شود و به **بُر زدن و مرتب‌سازی^a** مشهور است، تمامی مقادیر با کلید یکسان k_2 گردآوری شده و به صورت یک لیست تحویل یک کاهنده واحد می‌گردند.

Shuffle and Sort^a

مثال کاربردی ۱.۲: شمارش واژگان (Word Count)

مسئله استاندارد برای نشان دادن کارکرد MapReduce، شمارش تکرار هر کلمه در یک مجموعه سند است:

- **ورودی نگاشت:** کلید نام سند و مقدار متن یک سطر از سند است.

• تابع نگاشت:

(۳.۲) منتشر شود. $(w, 1)$ در متن، جفت w برای هر واژه $\text{Map}(\text{DocID}, \text{Text}) \Rightarrow$

• مرحله میانی (سیستم): تمام جفت‌های واژه یکسان دسته‌بندی می‌شوند:

(۴.۲) $(w, [1, 1, 1, \dots, 1])$

• تابع کاهش: مقادیر لیست را جمع می‌زند:

(۵.۲) $\text{Reduce}(w, [v_1, v_2, \dots, v_n]) \Rightarrow \left(w, \sum_{i=1}^n v_i \right)$

۴.۲ بهینه‌سازی‌های اجرایی در نگاشت-کاهش

۱.۴.۲ تابع ترکیب‌کننده (Combiner)

در مثال شمارش واژگان، برای کلماتی مانند «در» یا «از»، یک مپر ممکن است میلیون‌ها جفت $(1, \text{«در»})$ تولید کند. فرستادن این حجم از داده خام روی شبکه باعث انسداد پهنای باند می‌گردد.

تعریف ۳.۲: ترکیب‌کننده

تابع ترکیب‌کننده^a یک مینی-ردیوسر اختیاری است که در حافظه ماشین نگاشت، قبل از ارسال داده‌ها به شبکه اجرا می‌شود. شرط استفاده از Combiner این است که عمل کاهش دارای ویژگی شرکت‌پذیری^b و جابجایی‌پذیری^c باشد (مانند جمع، ضرب، یا محاسبه بیشینه و کمینه؛ اما برای محاسبه میانگین نمی‌توان مستقیماً میانگین گرفت، بلکه باید مجموع و تعداد به طور مجزا فشرده شوند).

Combiner^a
Associative^b
Commutative^c

۲.۴.۲ تابع افرازبندی (Partitioner)

برای تعیین اینکه هر کلید k_p به کدام کاهنده (از میان R پردازنده کاهنده) ارسال شود، از تابع افرازبندی استفاده می‌شود. به صورت پیش‌فرض:

(۶.۲) $\text{Reducer_ID} = \text{hash}(k_p) \pmod{R}$

با تنظیم دستی تابع افرازبندی می‌توان توزیع کلیدها را برای جلوگیری از کجی داده^{۱۰} کنترل کرد.

۳.۴.۲ مدیریت گره‌های کند (Stragglers) با اجرای گمانه‌زانه

در یک اجرای توزیع‌شده با هزاران وظیفه، اگر ۹۹٪ وظایف ظرف ۵ دقیقه تمام شوند، ولی یک وظیفه روی ماشینی با هارد دیسک معیوب گیر کند، کل جاب متوقف می‌ماند. این پدیده عقب‌ماندگی یا گره کند^{۱۱} نام دارد. راهکار سیستم MapReduce اجرای کارهای پشتیبان^{۱۲} در مراحل پایانی است؛ هر یک از کارهای موازی که زودتر به پایان برسد، پذیرفته شده و دیگری متوقف می‌گردد.

Data Skew^{۱۰}
Straggler^{۱۱}
Execution Speculative یا Tasks Backup^{۱۲}

۵.۲ پیاده‌سازی اعمال جبر رابطه‌ای در MapReduce

پایگاه داده‌های رابطه‌ای بر مبنای اعمال جبر رابطه‌ای عمل می‌کنند. این اعمال در MapReduce به صورت مقیاس‌پذیر پیاده‌سازی می‌شوند:

۱.۵.۲ پیوند طبیعی (Natural Join)

فرض کنید دو رابطه $R(A, B)$ و $S(B, C)$ داریم و می‌خواهیم $R \bowtie S$ را روی ویژگی مشترک B محاسبه نماییم.

الگوریتم ۱.۲: پیوند طبیعی در MapReduce

• ورودی: سطرهای رابطه R به فرم (a, b) و سطرهای رابطه S به فرم (b, c) .

• خروجی: تاپل‌های پیوندخورده (a, b, c) .

• تابع نگاشت:

$$(7.2) \quad (b, (R, a)) \Rightarrow \text{خروجی: } R(a, b) \text{ برای هر سطر}$$

$$(8.2) \quad (b, (S, c)) \Rightarrow \text{خروجی: } S(b, c) \text{ برای هر سطر}$$

• تابع کاهش: ورودی کاهنده برای کلید b شامل تمام مقادیر مرتبط با b از هر دو رابطه است:

$$(9.2) \quad (b, [(R, a_1), (R, a_2), \dots, (S, c_1), (S, c_2), \dots])$$

کاهنده لیست را به دو زیرمجموعه بر حسب برچسب R و S تقسیم کرده و حاصل ضرب دکارتی آن‌ها را تولید می‌کند: به ازای هر $a \in R$ و هر $c \in S$ ، تاپل (a, b, c) را منتشر می‌نماید.

۲.۵.۲ گروه‌بندی و تجمیع (Grouping and Aggregation)

فرض کنید رابطه $R(A, B, C)$ را داریم و می‌خواهیم عملگر $\gamma_{A, \text{SUM}(B)}(R)$ (معادل دستور SELECT A, SUM(B) FROM R GROUP BY A) را در نگاشت-کاهش پیاده کنیم:

• تابع نگاشت: برای هر تاپل (a, b, c) ، خروجی جفت (a, b) خواهد بود.

• تابع ترکیب‌کننده (Combiner): در حافظه ماشین می‌رود، مجموع مقادیر b به ازای هر a محاسبه و جمع محلی ارسال می‌شود.

• تابع کاهش: به ازای هر کلید a ، لیست مقادیر $[b_1, b_2, \dots]$ جمع زده شده و جفت $(a, \sum b_i)$ منتشر می‌گردد.

۳.۵.۲ ضرب ماتریس در بردار (Matrix-Vector Multiplication)

مسئله ضرب ماتریس بزرگ M با ابعاد $n \times n$ در بردار v به طول n (که در فصل پنجم برای الگوریتم PageRank حیاتی است):

$$(10.2) \quad x_i = \sum_{j=1}^n m_{ij} v_j$$

حالت اول: بردار v در حافظه اصلی هر می‌رود:

• هر می‌رود یک سطر یا مجموعه‌ای از درایه‌های ماتریس m_{ij} را می‌خواند.

• می‌رود بلافاصله حاصل ضرب $m_{ij} v_j$ را محاسبه کرده و جفت $(i, m_{ij} v_j)$ را به عنوان خروجی صادر می‌کند.

• کاهنده تمام مقادیر با کلید i را جمع می‌زند تا درایه نهایی x_i محاسبه شود.

حالت دوم: بردار v در حافظه جا نمی‌شود (حجم بسیار بالا): ماتریس M و بردار v به بلوک‌های مربعی و نواری تقسیم می‌شوند. در این حالت، نگاشت‌ها بر اساس شناسه بلوک‌ها سازمان‌دهی شده و محاسبات به صورت بلوکی صورت می‌پذیرد.

۴.۵.۲ ضرب دو ماتریس بزرگ در نگاشت-کاهش (Matrix Multiplication)

فرض کنید هدف محاسبه حاصل ضرب دو ماتریس بزرگ M با ابعاد $r \times c$ و N با ابعاد $c \times p$ باشد $P = M \times N$ به طوری که $p_{ij} = \sum_{k=1}^c m_{ik} n_{kj}$. دو الگوریتم استاندارد در مرجع MMDS برای این منظور تدوین شده است: ۱. الگوریتم دو مرحله‌ای نگاشت-کاهش (Two-Pass MapReduce):

۱. گذر اول (محاسبه ضرب‌های جزئی):

- نگاشت: برای هر درایه m_{ik} جفت $(k, (M, i, m_{ik}))$ و برای هر درایه n_{kj} جفت $(k, (N, j, n_{kj}))$ تولید می‌شود.

- کاهش: کاهنده برای کلید مشترک k ، تمام درایه‌های سطر k از N و ستون k از M را در اختیار دارد و به ازای هر جفت، ضرب جزئی $m_{ik} n_{kj}$ را با کلید خروجی (i, j) صادر می‌کند:

$$((i, j), m_{ik} n_{kj}) \quad (11.2)$$

۲. گذر دوم (مجموع ضرب‌های جزئی):

- نگاشت: تابع همانی (خروجی گذر اول را مستقیماً عبور می‌دهد).

- کاهش: به ازای هر کلید (i, j) ، تمام ضرب‌های جزئی دریافتی را با یکدیگر جمع کرده و درایه نهایی $p_{ij} = \sum_{k=1}^c m_{ik} n_{kj}$ را در فایل خروجی ذخیره می‌کند.

۲. الگوریتم تک‌مرحله‌ای (One-Pass MapReduce): اگر اولویت کاهش تعداد گذرها و زمان پایان باشد:

- هر درایه m_{ik} توسط میر به p نسخه تکثیر شده و با کلیدهای $((i, 1), (M, k, m_{ik}))$ تا $((i, p), (M, k, m_{ik}))$ صادر می‌گردد.

- هر درایه n_{kj} نیز به r نسخه با کلیدهای $((1, j), (N, k, n_{kj}))$ تا $((r, j), (N, k, n_{kj}))$ تکثیر می‌شود.

- ردیوسر متناظر با کلید (i, j) ، کل سطر i از M و کل ستون j از N را به یکباره دریافت کرده و p_{ij} را مستقیماً در یک گام محاسبه می‌نماید (این روش ترافیک میر به شبکه را به اندازه ابعاد ضرب افزایش می‌دهد).

نکته امتحانی و کلیدی

در تحلیل هزینه الگوریتم‌های MapReduce در آزمون، دو معیار اساسی وجود دارد: ۱. **هزینه ارتباطات (Communication Cost)**: برابر است با مجموع اندازه فایل‌های خروجی تمام میپرها که باید از طریق شبکه به سمت ردیوسرها منتقل شوند. ۲. **هزینه زمانی سپری‌شده (Elapsed Time)**: بیشترین زمان صرف‌شده در طولانی‌ترین مسیر اجرای یک گره، که نشان‌دهنده گلوگاه اصلی محاسبات موازی است.

جمع‌بندی فصل دوم

- مدل نگاشت-کاهش پردازش داده‌های در مقیاس پتابایت را با تکیه بر سخت‌افزارهای تجاری ارزان و تحمل خطای خودکار میسر می‌سازد.
- سیستم‌های فایل توزیع‌شده (HDFS) با تکثیر بلوک‌ها تضمین پایداری داده را بر عهده دارند.
- توابع Combiner و Partitioner نقش حیاتی در بهینه‌سازی پهنای باند و تعادل بار شبکه ایفا می‌کنند.
- پیوند طبیعی، گروه‌بندی و ضرب‌های ماتریسی به سادگی و کارایی در قالب یک یا چند گام نگاشت-کاهش مدل‌سازی می‌شوند.

فصل ۳

یافتن آیتم‌های مشابه و درهم‌سازی حساس به مکان (LSH)

۱.۳ مقدمه و تعریف مسئله

در تحلیل کلان‌داده، یکی از متداول‌ترین نیازمندی‌ها، یافتن آیتم‌هایی است که به یکدیگر «مشابه» هستند. کاربردهای این مسئله بسیار متنوع و گسترده است:

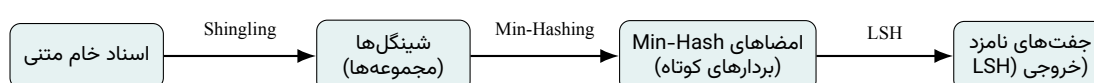
- **تشخیص اسناد تکراری یا تقریباً یکسان^۱:** شناسایی صفحات آینه‌ای در وب، سرقت ادبی در مقالات، یا اخبار بازنشر شده از یک خبرگزاری در سایت‌های مختلف.
 - **سامانه‌های توصیه‌گر^۲:** یافتن کاربرانی که سلیقه یا تاریخچه خرید یکسانی در فروشگاه آنلاین دارند.
 - **جستجوی تصاویر مشابه:** بازیابی تصاویری که علیرغم تغییرات اندک در رزولوشن یا کادربندی، محتوای یکسانی دارند.
 - **شناسایی کلاهبرداری^۳:** یافتن گزارش‌های پزشکی یا خسارت‌های بیمه‌ای با شرح یکسان.
- اگر مجموعه‌ای شامل N آیتم داشته باشیم، مقایسه دوجه‌دوی تمامی جفت‌ها نیازمند انجام:

$$\binom{N}{2} = \frac{N(N-1)}{2} \approx \frac{N^2}{2} \quad (1.3)$$

مقایسه است. چنانچه $N = 10^6$ (یک میلیون سند) باشد، تعداد مقایسه‌ها حدود 5×10^{11} خواهد بود. حتی اگر هر مقایسه تنها ۱ میکروثانیه زمان ببرد، اجرای آن نزدیک به ۶ روز به طول می‌انجامد؛ برای $N = 10^7$ ، این زمان به ۶۰۰ روز افزایش خواهد یافت!

برای حل این چالش بنیادین، یک خط لوله محاسباتی سه‌مرحله‌ای طراحی شده است (شکل ۱.۳):

۱. **شینگلینگ (Shingling):** تبدیل اسناد متنی به مجموعه‌هایی از توکن‌های متوالی کوتاه.
۲. **کمینه‌سازی درهم‌سازی (Min-Hashing):** تبدیل مجموعه‌های بسیار بزرگ به امضاهای کوتاه، به نحوی که تشابه مجموعه‌ها حفظ گردد.
۳. **درهم‌سازی حساس به مکان (LSH):** متمرکز ساختن بررسی‌ها تنها بر روی جفت‌های نامزدی که احتمال شباهت آن‌ها بسیار بالاست، بدون نیاز به مقایسه تمامی جفت‌ها.



شکل ۱.۳: خط لوله استاندارد سه‌مرحله‌ای برای یافتن آیتم‌های مشابه در کلان‌داده

^۱ Near-Duplicate Documents
^۲ Recommender Systems
^۳ Fraud Detection

۲.۳ گام نخست: شینگلینگ اسناد

برای مقایسه دو سند، نمی‌توان صرفاً به برابری دقیق کلمات نگاه کرد، زیرا تغییر ترتیب چند کلمه یا افزودن یک پاراگراف کوتاه نباید مانع از کشف شباهت شود.

تعریف ۱.۳: k -شینگل یا k -گرام

یک k -شینگل^a برای یک سند متنی، هر زیررشته به طول k است که در درون آن سند ظاهر می‌شود. شینگل‌ها می‌توانند بر حسب کاراکتر یا واژه تعریف شوند.

^aShingle- k یا Gram- k

مثال کاربردی ۱.۳: استخراج شینگل‌ها

فرض کنید سند D برابر با رشته «abcdabd» باشد و $k = ۲$ کاراکتر در نظر گرفته شود. مجموعه شینگل‌های موجود در سند عبارت‌اند از:

$$S(D) = \{ab, bc, cd, da, bd\} \quad (۲.۳)$$

توجه نمایید که شینگل «ab» دوبار ظاهر شده است، اما در قالب مجموعه، عناصر تکراری حذف می‌گردند (مگر آنکه از چندمجموعه^a استفاده شود).

^aBag of Shingles

۱.۲.۳ انتخاب اندازه مناسب برای k

- اگر k خیلی کوچک انتخاب شود (مثلاً $k = ۱$ یا $k = ۲$ در زبان انگلیسی)، تقریباً تمام شینگل‌ها در تمام اسناد تکرار خواهند شد؛ در نتیجه همه اسناد به اشتباه شبیه هم ارزیابی می‌شوند.
- قانون تجربی: k باید آن‌قدر بزرگ باشد که احتمال حضور تصادفی یک شینگل خاص در سندی نامرتبط بسیار اندک باشد. برای اسناد معمولی متنی (ایمیل‌ها و مقالات کوتاه) معمولاً $k = ۵$ واژه یا $k = ۹$ کاراکتر انتخاب می‌شود.
- **فشرده‌سازی شینگل‌ها:** برای صرفه‌جویی در مصرف حافظه، هر شینگل با استفاده از یک تابع درهم‌سازی مطمئن به یک عدد صحیح ۴ بیتی (۳۲ بیتی) نگاشت می‌شود؛ بدین ترتیب هر سند به مجموعه‌ای از اعداد ۴ بیتی تبدیل می‌گردد.

۳.۳ معیار تشابه جاکارد و ماتریس مشخصه

تعریف ۲.۳: تشابه جاکارد

تشابه جاکارد^a میان دو مجموعه S و T برابر است با نسبت اندازه اشتراک آن‌ها به اندازه اجتماع آن‌ها:

$$\text{SIM}(S, T) = J(S, T) = \frac{|S \cap T|}{|S \cup T|} \quad (۳.۳)$$

فاصله جاکارد^b نیز به صورت مکمل تشابه تعریف می‌شود:

$$d_J(S, T) = 1 - J(S, T) = \frac{|S \cup T| - |S \cap T|}{|S \cup T|} \quad (۴.۳)$$

Jaccard Similarity^aJaccard Distance^b

قضیه ۱.۳: ویژگی متریک فاصله جاکارد

فاصله جاکارد $d_J(S, T) = 1 - J(S, T)$ یک **متریک ریاضی معتبر** است؛ یعنی هر چهار شرط فضای متریک را ارضا می‌کند:

۱. **نامنفی بودن:** $d_J(S, T) \geq 0$ (چون $0 \leq J(S, T) \leq 1$).

۲. **انطباق بر خود:** $d_J(S, T) = 0 \iff S = T$.

۳. **تقارن:** $d_J(S, T) = d_J(T, S)$.

۴. **نامساوی مثلثی:** برای هر سه مجموعه S_1, S_2, S_3 :

$$d_J(S_1, S_3) \leq d_J(S_1, S_2) + d_J(S_2, S_3) \quad (۵.۳)$$

اثبات نامساوی مثلثی با بهره‌گیری از Min-Hashing: بر اساس قضیه Min-Hashing، فاصله جاکارد دقیقاً برابر است با احتمال عدم تطابق مقدار هش تصادفی دو مجموعه:

$$d_J(S, T) = \Pr[h(S) \neq h(T)] \quad (۶.۳)$$

حال برای هر سه مجموعه S_1, S_2, S_3 ، اگر $h(S_1) \neq h(S_3)$ رخ دهد، لزوماً حداقل یکی از دو رویداد $h(S_1) \neq h(S_2)$ یا $h(S_2) \neq h(S_3)$ باید اتفاق افتاده باشد (زیرا اگر هر دو جفت برابر بودند، بنا بر اصل تعدی باید $h(S_1) = h(S_3)$ می‌شد). بنابراین طبق قاعده اجتماع احتمالات:

$$\Pr[h(S_1) \neq h(S_3)] \leq \Pr[h(S_1) \neq h(S_2)] + \Pr[h(S_2) \neq h(S_3)] \quad (۷.۳)$$

که مستقیماً نامساوی مثلثی $d_J(S_1, S_3) \leq d_J(S_1, S_2) + d_J(S_2, S_3)$ را به اثبات می‌رساند.

۱.۳.۳ ماتریس مشخصه (Characteristic Matrix)

اگر تمام شینگل‌های جهان داده را به عنوان سطرهای یک ماتریس و اسناد مختلف را به عنوان ستون‌های آن در نظر بگیریم، **ماتریس مشخصه**^۴ حاصل می‌شود:

$$M_{ij} = \begin{cases} 1 & \text{وجود داشته باشد } j \text{ در سند } i \text{ اگر شینگل} \\ 0 & \text{در غیر این صورت} \end{cases} \quad (۸.۳)$$

این ماتریس در عمل بی‌نهایت تنگ^۵ است و هرگز به طور کامل در حافظه ذخیره نمی‌شود، اما چارچوب مفهومی بسیار روشنی برای تعریف Min-Hashing فراهم می‌آورد.

Characteristic Matrix^۴
Sparse^۵

۴.۳ گام دوم: کمینه‌سازی درهم‌سازی (Min-Hashing)

هدف از Min-Hashing فشرده‌سازی ستون‌های بسیار طولانی ماتریس مشخصه به بردارها و امضاهای کوتاه با طول n (مثلاً $n = 100$ یا 200) است، به گونه‌ای که تشابه امضاها دقیقاً برابر با تشابه جاکارد مجموعه‌های اصلی باشد.

۱.۴.۳ جایگشت سطرها و تابع Min-Hash

فرض کنید جایگشتی تصادفی π بر روی سطرهای ماتریس مشخصه اعمال کنیم.

تعریف ۳.۳: تابع Min-Hash

مقدار $h_{\pi}(C)$ برای ستون C برابر است با شماره نخستین سطر (در ترتیب جایگشت π) که مقدار درایه ستون C در آن سطر برابر با ۱ باشد.

قضیه ۲.۳: قضیه بنیادین Min-Hashing []

احتمال اینکه مقدار Min-Hash دو ستون C_1 و C_2 با یکدیگر برابر باشد، دقیقاً معادل تشابه جاکارد آن دو ستون است:

$$\Pr[h_{\pi}(C_1) = h_{\pi}(C_2)] = \text{SIM}(C_1, C_2) = J(C_1, C_2) \quad (9.3)$$

اثبات قضیه: سطرهای ماتریس مشخصه نسبت به دو ستون C_1 و C_2 به سه دسته تقسیم می‌شوند:

- نوع X : سطرهایی که در هر دو ستون مقدار ۱ دارند (تعداد: $|C_1 \cap C_2|$). $x = |C_1 \cap C_2|$.
- نوع Y : سطرهایی که در یکی از دو ستون مقدار ۱ و در دیگری ۰ دارند (تعداد: $|C_1 \cup C_2| - |C_1 \cap C_2|$). $y = |C_1 \cup C_2| - |C_1 \cap C_2|$.
- نوع Z : سطرهایی که در هر دو ستون مقدار ۰ دارند.

هنگامی که سطرها را به صورت تصادفی جایگشت می‌دهیم و از بالا به پایین پیمایش می‌کنیم، سطرهای نوع Z تاثیری در مقدار Min-Hash ندارند زیرا مقدار هر دو ستون صفر است. بنابراین، نخستین سطر که مقدار ۱ در آن دیده می‌شود، یا از نوع X خواهد بود یا از نوع Y . مقدار $h_{\pi}(C_1) = h_{\pi}(C_2)$ خواهد بود اگر و تنها اگر نخستین سطر غیرصفر مشاهده‌شده از نوع X باشد. احتمال این رخداد برابر است با نسبت تعداد سطرهای نوع X به کل سطرهای نوع X و Y :

$$\Pr[h_{\pi}(C_1) = h_{\pi}(C_2)] = \frac{x}{x+y} = \frac{|C_1 \cap C_2|}{|C_1 \cup C_2|} = J(C_1, C_2) \quad (10.3)$$

و حکم اثبات می‌شود.

۲.۴.۳ الگوریتم شبیه‌سازی Min-Hash بدون اعمال جایگشت فیزیکی

در کلان‌داده، جایگشت دادن فیزیکی میلیاردها سطر ناممکن است. به جای آن، از n تابع درهم‌سازی مستقل به فرم $h_i(r) = (a_i r + b_i) \pmod{p}$ استفاده می‌شود. الگوریتم ماتریس امضا به شرح زیر عمل می‌کند:

الگوریتم ۱.۳: محاسبه ماتریس امضای Min-Hash

- ورودی: ماتریس مشخصه با R سطر و C ستون، و n تابع درهم‌سازی $h_1, h_2, \dots, h_n$.
- خروجی: ماتریس امضا SIG با ابعاد $n \times C$.
- مقداردهی اولیه: برای تمامی درایه‌های ماتریس امضا، $SIG(i, c) = \infty$ قرار دهید.
- پیمایش سطر به سطر: به ازای هر سطر r از ماتریس مشخصه:
 ۱. مقادیر $h_1(r), h_2(r), \dots, h_n(r)$ را محاسبه کنید.

۲. به ازای هر ستون c که در آن سطر مقدار ۱ دارد:

$$SIG(i, c) = \min(SIG(i, c), h_i(r)) \quad \forall i \in \{1, \dots, n\} \quad (11.3)$$

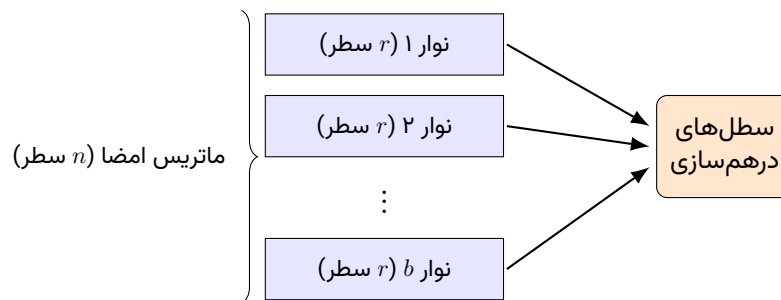
۵.۳ گام سوم: درهم‌سازی حساس به مکان (LSH) برای امضاها

حتی پس از تبدیل اسناد به امضاها، اگر بخواهیم تمامی جفت‌امضاها را با یکدیگر مقایسه کنیم، همچنان با هزینه زمانی مرتبه $O(N^2)$ روبرو هستیم. روش LSH^۶ راهکاری هوشمندانه برای کاهش فضای جستجو است.

۱.۵.۳ روش دسته‌بندی نواری (Banding Technique)

فرض کنید ماتریس امضا دارای n سطر است. این ماتریس را به b نوار^۷ تقسیم می‌کنیم، به طوری که هر نوار شامل دقیقاً r سطر متوالی باشد، یعنی:

$$n = b \times r \quad (12.3)$$



شکل ۲.۳: افراز ماتریس امضا به b نوار از r سطر و هش کردن بخش‌های ستون به سطرها

سازوکار الگوریتم به شرح زیر است:

۱. برای هر نوار، یک تابع درهم‌سازی مجزا در نظر می‌گیریم.
۲. قطعه r -عنصری هر ستون در آن نوار را به یکی از سطرهای متعدد درهم‌سازی نگاشت می‌کنیم.
۳. **قاعده جفت‌های نامزد:** دو سند C_1 و C_2 به عنوان **جفت نامزد**^۸ اعلام می‌شوند اگر و تنها اگر حداقل در یک نوار، در سطر یکسانی قرار گیرند.
۴. تنها جفت‌های نامزد مورد مقایسه کامل شباهت قرار می‌گیرند.

۲.۵.۳ تحلیل ریاضی منحنی اس-شکل (S-Curve Analysis)

فرض کنید تشابه واقعی جاکارد میان دو سند برابر با $s = J(C_1, C_2)$ باشد. می‌خواهیم احتمال نامزد شدن این دو سند $(f(s))$ را محاسبه نماییم:

۱. احتمال اینکه دو ستون در یک سطر خاص از یک نوار توافق داشته باشند: s .
۲. احتمال اینکه در تمام r سطر یک نوار خاص توافق داشته باشند (یعنی در آن نوار در یک سطر بیفتند):

$$P(\text{تطابق در نوار خاص}) = s^r \quad (13.3)$$

^۶ Locality-Sensitive Hashing
^۷ Band
^۸ Candidate Pair

۳. احتمال اینکه در آن نوار خاص تطابق نداشته باشند:

$$1 - s^r \quad (۱۴.۳)$$

۴. احتمال اینکه در هیچ‌یک از b نوار تطابق نداشته باشند (عدم توافق در تمام نوارها):

$$(1 - s^r)^b \quad (۱۵.۳)$$

۵. بنابراین، احتمال اینکه در حداقل یک نوار تطابق داشته باشند (شرط نامزدی):

$$f(s) = 1 - (1 - s^r)^b \quad (۱۶.۳)$$

نمودار تابع $f(s)$ دارای شکل S است. آستانه شباهت بحرانی s^* که در آن شیب منحنی بیشترین مقدار است، با تقریب عالی از رابطه زیر به دست می‌آید:

$$s^* \approx \left(\frac{1}{b}\right)^{1/r} \quad (۱۷.۳)$$

اگر تشابه دو سند $s > s^*$ باشد، احتمال نامزدی آنها به سرعت به ۱ نزدیک می‌شود؛ و اگر $s < s^*$ باشد، احتمال نامزدی به سرعت به ۰ میل می‌کند.

مثال کاربردی ۲.۳: محاسبه احتمال خطای مثبت کاذب و منفی کاذب در LSH

فرض کنید $n = ۱۰۰$ سطر در ماتریس امضا داریم و آن را به $b = ۲۰$ نوار با $r = ۵$ سطر تقسیم کرده‌ایم. آستانه تقریب برابر است با:

$$s^* \approx (1/20)^{1/5} \approx ۰.۵۴۹ \quad (۱۸.۳)$$

حال دو سناریو را بررسی می‌کنیم:

• دو سند با شباهت بالا $s = ۰.۸$:

$$f(0.8) = 1 - (1 - 0.8^5)^{20} = 1 - (1 - 0.32768)^{20} = 1 - (0.67232)^{20} \approx 1 - 0.00035 = 0.99965 \quad (۱۹.۳)$$

یعنی با احتمال ۹۹.۹۶۵٪ این دو سند به درستی شناسایی می‌شوند (تنها ۰.۰۳۵٪ خطای منفی کاذب).

• دو سند با شباهت پایین $s = ۰.۲$:

$$f(0.2) = 1 - (1 - 0.2^5)^{20} = 1 - (1 - 0.00032)^{20} \approx 1 - (1 - 20 \times 0.00032) \approx 0.0064 \quad (۲۰.۳)$$

احتمال اینکه این دو سند نامزد شوند کمتر از ۰.۶۴٪ است (خطای مثبت کاذب بسیار اندک).

۶.۳ معیارهای فاصله و خانواده‌های درهم‌سازی حساس به مکان

تشابه جاکارد تنها یکی از معیارهای فاصله است. در کلان‌داده، انواع معیارهای فاصله به کار می‌روند:

تعریف ۴.۳: اصول موضوعه یک تابع فاصله (متریک)

یک تابع $d(x, y)$ یک متریک معتبر است اگر و تنها اگر در چهار شرط زیر صدق کند:

۱. نامنفی بودن: $d(x, y) \geq ۰$.

۲. تفکیک‌پذیری: $d(x, y) = ۰ \iff x = y$.

۳. تقارن: $d(x, y) = d(y, x)$.

۴. نامساوی مثلثی: $d(x, z) \leq d(x, y) + d(y, z)$.

• فاصله اقلیدسی (L_2) : $d(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$.

• فاصله منهتن (L_1) : $d(x, y) = \sum_i |x_i - y_i|$.

• فاصله کسینوسی: زاویه بین دو بردار، $\theta = \arccos\left(\frac{x \cdot y}{\|x\| \|y\|}\right)$ ، که منجر به روش اَبَرَبَرش‌های تصادفی^{۱۰} یا SimHash می‌شود.

• فاصله همینگ^{۱۱}: تعداد موقعیت‌هایی که دو رشته با طول برابر در آن‌ها متفاوت‌اند.

نکته امتحانی و کلیدی

در آزمون‌ها، نحوه تنظیم پارامترهای b و r با توجه به آستانه مورد نظر s بسیار حائز اهمیت است:

- اگر هدف کاهش منفی کاذب^a باشد (نمی‌خواهیم هیچ جفت مشابهی جا بیفتد)، b را افزایش و r را کاهش می‌دهیم (پایین آوردن آستانه).
- اگر هدف کاهش مثبت کاذب^b و کاهش زمان محاسبات باشد، r را افزایش می‌دهیم تا شرط سخت‌گیرانه‌تری برای هم‌سطلی اعمال گردد.

False Negatives^a
False Positives^b

جمع‌بندی فصل سوم

- مقایسه دوبه‌دوی اسناد در کلان‌داده به دلیل مرتبه زمانی $O(N^2)$ غیرممکن است.
- شینگلینگ اسناد را به مجموعه‌هایی مناسب برای ارزیابی تشابه جاکارد تبدیل می‌کند.
- روش Min-Hash با تکیه بر جایگشت‌های تصادفی، مجموعه‌های حجیم را به امضاهایی کوتاه فشرده می‌سازد به نحوی که $\Pr[h(C_1) = h(C_2)] = J(C_1, C_2)$.
- الگوریتم LSH با تقسیم ماتریس امضا به b نوار از r سطر و ایجاد منحنی، شکل-S فضای جستجو را تنها به نامزدهای بسیار محتمل محدود می‌سازد.

بخش دوم

کاوش جریان‌های داده و وب

فصل ۴

کاوش جریان‌های داده (Data Streams)

۱.۴ مقدمه و مدل محاسباتی جریان داده

در سیستم‌های سنتی پایگاه داده، فرض بر این است که داده‌ها ابتدا روی دیسک به صورت پایدار ذخیره می‌شوند و سپس پرس‌وجوهای تحلیلی روی داده‌های ایستا اجرا می‌گردند. اما در بسیاری از کاربردهای نوین کلان‌داده، داده‌ها به صورت پیوسته، با سرعتی سرسام‌آور و در حجم نامحدود تولید می‌شوند؛ به این نوع داده‌ها **جریان داده**^۱ گفته می‌شود. نمونه‌های بارز جریان داده عبارت‌اند از:

- کلیک‌های کاربران در وبسایت‌های پرترافیک مانند گوگل یا آمازون.
- داده‌های ارسالی از میلیون‌ها حسگر در شبکه‌های اینترنت اشیا (IoT).
- بسته‌های اطلاعاتی عبوری از روترهای شبکه اینترنت (نظیر پایش حملات DDoS و ناهنجاری‌های شبکه).
- پیام‌های منتشرشده در شبکه‌های اجتماعی نظیر توییتر.

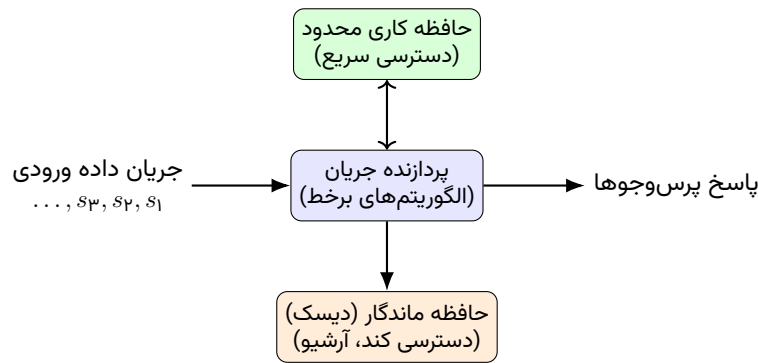
تعریف ۱.۴: مدل پردازش جریان داده

در مدل محاسباتی جریان داده (شکل ۱.۴):

۱. داده‌ها به شکل دنباله‌ای بی‌پایان از تاپل‌ها وارد سیستم می‌شوند.
۲. نرخ ورود داده‌ها ممکن است بسیار فراتر از توان ذخیره‌سازی فوری روی دیسک باشد.
۳. سیستم پردازش جریان دارای یک **حافظه کاری**^a با ظرفیت محدود است.
۴. هر عنصر تنها یک بار (یا با تعداد دفعات بسیار محدود) قابل مشاهده است؛ اگر عنصری پردازش نشود، برای همیشه از دست می‌رود.

Working Memory^a

Data Stream^۱



شکل ۱.۴: معماری سیستم مدیریت جریان داده: پردازش فوری با تکیه بر حافظه کاری محدود

پرس‌وجوها در جریان داده به دو دسته تقسیم می‌شوند:

- **پرس‌وجوهای دائمی^۲:** پرس‌وجوهایی که همواره فعال هستند و به ازای هر عنصر ورودی، پاسخی به روز تولید می‌کنند (مثلاً: «اگر دمای هر حسگر از ۸۰ درجه بیشتر شد، هشدار بده»).
- **پرس‌وجوهای موردی^۳:** پرس‌وجوهایی که در یک لحظه خاص توسط تحلیل‌گر مطرح می‌شوند (مثلاً: «میانگین طول پیام‌های ارسال‌شده در ۱۰ دقیقه گذشته چقدر بوده است؟»).

۲.۴ نمونه‌گیری از جریان‌های داده

هنگامی که نمی‌توان تمام جریان را ذخیره کرد، نمونه‌گیری یک راهکار طبیعی است. دو مسئله اساسی در نمونه‌گیری وجود دارد:

۱.۲.۴ نمونه‌گیری با نسبت ثابت (Fixed Proportion Sampling)

فرض کنید می‌خواهیم ۱۰٪ از پرس‌وجوهای یک موتور جستجو را نمونه‌برداری کنیم تا رفتار کاربران را بررسی نماییم. اگر به ازای هر پرس‌وجو یک عدد تصادفی بین ۰ تا ۹ تولید کنیم و در صورت صفر بودن آن را برگزینیم، هر کاربر دارای تعداد پرس‌وجوهای متفاوتی در نمونه خواهد بود، اما اگر بخواهیم بفهمیم چه کسری از کاربران تنها یک بار پرس‌وجو کرده‌اند، این نمونه‌گیری نتایج را به شدت مخدوش می‌کند.

راهکار اصولی: نمونه‌گیری بر مبنای **کلید کاربر**. با استفاده از یک تابع درهم‌سازی:

$$h(\text{UserID}) \pmod{10} = 0 \quad (1.4)$$

اگر باقیمانده صفر شد، تمام پرس‌وجوهای آن کاربر ذخیره می‌شود و در غیر این صورت هیچ‌یک از پرس‌وجوهای او ذخیره نمی‌گردد. این کار نسبت نمونه ۱۰٪ را تضمین کرده و در عین حال ساختار کامل رفتار هر کاربر نمونه را حفظ می‌کند.

۲.۲.۴ نمونه‌گیری با اندازه ثابت: الگوریتم مخزنی (Reservoir Sampling)

فرض کنید می‌خواهیم در هر لحظه نمونه‌ای تصادفی و یکنواخت به اندازه دقیقاً s از میان n عنصر نخست جریان که تاکنون مشاهده شده‌اند در اختیار داشته باشیم، در حالی که مقدار نهایی n از پیش معلوم نیست و حافظه ما فقط گنجایش s عنصر را دارد.

الگوریتم ۱.۴: الگوریتم نمونه‌گیری مخزنی (Reservoir Sampling)

- **ورودی:** جریان داده $x_1, x_2, \dots$ و ظرفیت مخزن s .
- **گام اول:** s عنصر نخست جریان $(x_1, \dots, x_s)$ را مستقیماً در مخزن قرار دهید.
- **گام دوم:** برای هر عنصر ورودی بعدی x_n (که $n > s$):

۱. با احتمال $\frac{s}{n}$ عنصر x_n را برای ورود به مخزن بپذیرید.
۲. در صورت پذیرش، یکی از s عنصر موجود در مخزن را به صورت کاملاً تصادفی و یکنواخت (با احتمال $1/s$) انتخاب کرده و عنصر جدید را جایگزین آن نمایید.
۳. با احتمال $1 - \frac{s}{n}$ عنصر جدید را بدون تغییر مخزن دور بریزید.

قضیه ۱.۴: صحت الگوریتم مخزنی

پس از رسیدن n عنصر از جریان، احتمال حضور هر عنصر x_i ($1 \leq i \leq n$) در مخزن دقیقاً برابر با $\frac{s}{n}$ است.

اثبات با استقرای ریاضی:

- **پایه استقرا:** برای $n = s$ ، تمامی s عنصر اول در مخزن قرار دارند و احتمال حضور هر یک $s/s = 1$ است که درست است.
- **فرض استقرا:** فرض کنیم حکم برای $n - 1$ عنصر برقرار باشد؛ یعنی بعد از $n - 1$ عنصر، هر عنصری با احتمال $\frac{s}{n-1}$ در مخزن قرار دارد.
- **گام استقرا:** حال عنصر n -ام می‌رسد:

۱. برای خود عنصر n -ام، الگوریتم مستقیماً آن را با احتمال $\frac{s}{n}$ در مخزن قرار می‌دهد.
۲. برای هر عنصر قبلی x_i ($i < n$): این عنصر در مخزن باقی می‌ماند اگر و تنها اگر قبل از ورود عنصر n -ام در مخزن بوده باشد، و با ورود عنصر جدید حذف نشود:

$$\begin{aligned}
 \Pr[x_i \text{ حذف نشود} \mid n] &= \Pr[x_i \text{ در مخزن در گام } n-1] \times \Pr[x_i \text{ حذف نشود} \mid n] \\
 &= \left(\frac{s}{n-1}\right) \times \left(1 - \frac{s}{n} + \frac{s}{n} \left(1 - \frac{1}{s}\right)\right) \\
 &= \left(\frac{s}{n-1}\right) \times \left(1 - \frac{s}{n} + \frac{s}{n} - \frac{1}{n}\right) \\
 &= \left(\frac{s}{n-1}\right) \times \left(\frac{n-1}{n}\right) = \frac{s}{n}
 \end{aligned} \tag{۲.۴}$$

بنابراین حکم برای تمامی n عنصر اثبات می‌شود.

۳.۴ فیلتر کردن جریان‌های داده: فیلتر بلوم (Bloom Filter)

فرض کنید مجموعه‌ای از m کلید مجاز یا نامطلوب S داریم (مثلاً ۱ میلیارد آدرس ایمیل اسپم یا های URL فیشینگ). جریان عظیمی از تاپل‌ها وارد می‌شود و می‌خواهیم در چند میکروثانیه بررسی کنیم آیا کلید ورودی در S هست یا خیر. اندازه S ممکن است به گیگابایت‌ها برسد و در حافظه کش سرور جا نگیرد.

تعریف ۲.۴: فیلتر بلوم [۱]

- یک **فیلتر بلوم**^a یک ساختار داده احتمالی و بسیار فشرده برای آزمون عضویت در مجموعه است که:
- دارای یک آرایه بیتی B به طول n (در ابتدا همگی ۰) است.
 - از k تابع درهم‌سازی مستقل $h_1, h_2, \dots, h_k$ استفاده می‌کند که بازه خروجی آنها $[0, n-1]$ است.
 - **درج عضو** $x \in S$: برای تمام $i \in \{1, \dots, k\}$ بیت $B[h_i(x)]$ برابر با ۱ قرار داده می‌شود.
 - **پرس‌وجوی عضویت** y : اگر برای تمامی $i \in \{1, \dots, k\}$ مقدار $B[h_i(y)] = 1$ باشد، پاسخ «بله» است؛ وگرنه «خیر».

^aBloom Filter

ویژگی حیاتی فیلتر بلوم:

- **عدم وجود منفی کاذب (Zero False Negatives):** اگر عنصری در S باشد، بیت‌های مربوط به آن قطعاً ۱ شده‌اند و امکان ندارد پاسخ منفی دریافت کند.
- **احتمال مثبت کاذب (False Positives):** ممکن است عنصری در S نباشد، اما تصادفاً تمامی بیت‌های متناظر با توابع هش آن توسط سایر عناصر ۱ شده باشند.

۱.۳.۴ تحلیل احتمال خطای مثبت کاذب و مقدار بهینه k

فرض کنید m عنصر در فیلتر با اندازه n بیت و با k تابع هش درج شده باشند:

۱. احتمال اینکه یک بیت خاص پس از یک بار اعمال هش همچنان ۰ باقی بماند برابر است با $1 - \frac{1}{n}$.
۲. پس از درج m عنصر با k تابع هش (مجموعاً $k \cdot m$ درج بیت)، احتمال اینکه یک بیت هنوز ۰ باشد:

$$p_0 = \left(1 - \frac{1}{n}\right)^{km} \approx e^{-km/n} \quad (۳.۴)$$

۳. بنابراین، کسر بیت‌هایی از آرایه که مقدار ۱ به خود گرفته‌اند:

$$p_1 = 1 - e^{-km/n} \quad (۴.۴)$$

۴. احتمال اینکه برای یک عنصر ناموجود y ، تمامی k بیت هش‌شده برابر ۱ باشند (نرخ مثبت کاذب):

$$P(\text{Positive False}) = (p_1)^k = \left(1 - e^{-km/n}\right)^k \quad (۵.۴)$$

با مشتق‌گیری نسبت به k و مساوی صفر قرار دادن آن، **تعداد بهینه توابع درهم‌سازی** برای کمینه‌سازی خطای مثبت کاذب به دست می‌آید:

$$k_{\text{opt}} = \frac{n}{m} \ln 2 \approx 0.693 \times \frac{n}{m} \quad (۶.۴)$$

و با جایگذاری این مقدار، کمترین احتمال مثبت کاذب ممکن برابر است با:

$$P_{\text{opt}} = \left(\frac{1}{2}\right)^k = (0.6185)^{n/m} \quad (۷.۴)$$

مثال کاربردی ۱.۴: محاسبه پارامترهای فیلتر بلوم

فرض کنید $m = 10^9$ کلید داریم و می‌خواهیم نرخ مثبت کاذب کمتر از ۱٪ (۰/۰۱) باشد.

$$\bullet \text{ از رابطه } k \approx 7 \Rightarrow 2^k \geq 100 \Rightarrow P \approx (0.5)^k \leq 0.01$$

$$\bullet \text{ از رابطه } \frac{n}{m} \approx 10.7 \Rightarrow k = \frac{n}{m} \ln 2 \Rightarrow \frac{n}{m} = \frac{7}{\ln 2}$$

• بنابراین با اختصاص حدود ۱۰ بیت (تنها ۲۵.۱ بایت) به ازای هر کلید، خطای مثبت کاذب به زیر ۱ درصد می‌رسد!

۴.۴ شمارش عناصر متمایز: الگوریتم فلژوله-مارتین (Flajolet-Martin)

مسئله: در یک جریان از شناسه‌ها، چه تعداد عنصر یکتا و متمایز وجود دارد؟ اگر بخواهیم شناسه‌ها را در جدول درهم‌سازی ذخیره کنیم، با افزایش تنوع عناصر با کمبود شدید حافظه مواجه می‌شویم. الگوریتم فلژوله-مارتین (FM) [۷] تنها با مصرف حافظه‌ای در حد چند کلمه، این مقدار را تخمین می‌زند.

الگوریتم ۲.۴: الگوریتم فلزوله-مارتین برای شمارش عناصر متمایز

• **تابع هش:** تابعی مانند $h(a)$ که عناصر را به رشته‌ای از بیت‌ها به طول L نگاشت می‌کند (به طوری که $2^L > N$).

• **تابع صفرهای انتهایی $r(a)$:** تعداد صفرهای متوالی در سمت راست نمایش دودویی $h(a)$. (مثلاً اگر $h(a) = 101000$ باشد، $r(a) = 3$ است).

• **نگهداری بیشینه صفرهای انتهایی:**

$$R = \max_{a \in \text{Stream}} r(a) \quad (۸.۴)$$

• **تخمین تعداد عناصر متمایز:**

$$\hat{D} = 2^R / \phi \quad \text{که در آن } \phi \approx 0.77351 \quad (۹.۴)$$

بینش شهودی و تحلیل آماری: احتمال اینکه هش یک عنصر با حداقل i صفر متوالی تمام شود برابر با 2^{-i} است. اگر D عنصر متمایز در جریان وجود داشته باشد، احتمال اینکه هیچ‌کدام از آن‌ها با i صفر خاتمه نیابند:

$$(1 - 2^{-i})^D \approx e^{-D \cdot 2^{-i}} \quad (۱۰.۴)$$

• اگر $2^i \ll D$ باشد، احتمال مشاهده نکردن صفر انتهایی به سمت صفر میل می‌کند (قطعاً چنین عنصری دیده می‌شود).

• اگر $2^i \gg D$ باشد، این احتمال نزدیک به ۱ است (تقریباً محال است چنین عنصری دیده شود).

• نقطه گذار دقیقاً در $2^R \approx D$ اتفاق می‌افتد.

۱.۴.۴ تکنیک کاهش واریانس: میانگین میانه‌ها (Median of Means)

چون 2^R همواره توانی از دو است، واریانس این تخمین‌گر بالاست. برای رفع این نقیصه:

۱. از چندین تابع هش مستقل استفاده می‌کنیم.

۲. توابع هش را به گروه‌های کوچکی افراز می‌کنیم.

۳. در هر گروه، میانگین حسابی^۴ مقادیر 2^R را محاسبه می‌کنیم تا واریانس کاهش یابد.

۴. میان گروه‌ها، میانه^۵ مقادیر میانگین را برمی‌گزینیم تا مقادیر پرت و نامتعارف بی‌اثر شوند.

۵.۴ شمارش بیت‌ها در پنجره لغزان: الگوریتم DGIM

فرض کنید جریانی از صفرها و یک‌ها داریم و می‌خواهیم در هر لحظه تعداد بیت‌های ۱ موجود در پنجره لغزان^۶ به طول N گذشته را بدانیم. ذخیره کل N بیت نیازمند N بیت حافظه است که برای پنجره‌های بزرگ (مثلاً $N = 10^9$) بسیار پرهزینه است. الگوریتم DGIM (ابداع‌شده توسط داتار، گیونیس، ایندیک و موتوانی [۵]) این مسئله را در فضای $O(\log^2 N)$ و با خطای حداکثر ۵۰٪ حل می‌کند.

تعریف ۳.۴: ساختار سطل‌های DGIM

الگوریتم پنجره را به صورت مجموعه‌ای از سطل‌ها^۸ خلاصه می‌کند:

۱. هر سطل شامل دو مقدار است: (۱) زمان انتهایی سطل (یک برچسب زمانی $O(\log N)$ بیتی)؛ (۲) اندازه سطل

Average^۴Median^۵Sliding Window^۶

(تعداد بیت‌های ۱ موجود در آن).

۲. اندازه هر سطل همواره توانی از ۲ است $(1, 2, 4, 8, \dots)$.

۳. قاعده ظرفیت: به ازای هر اندازه سطل، حداکثر دو سطل (یک یا دو سطل) در پنجره مجاز است و هرگز سه سطل با اندازه یکسان نباید وجود داشته باشد.

۴. سطل‌ها هرگز هم‌پوشانی ندارند و سطل‌های با اندازه بزرگ‌تر، برچسب‌های زمانی قدیمی‌تری دارند.

Buckets^a

الگوریتم ۳.۴: به‌روزرسانی و پاسخ به پرس‌وجو در الگوریتم DGIM

عملیات به‌روزرسانی هنگام ورود بیت جدید:

۱. اگر بیت جدید ۰ باشد، کاری انجام نمی‌شود (تنها برچسب زمان جاری پیش می‌رود).

۲. اگر بیت جدید ۱ باشد:

- سطلی با اندازه ۱ و با زمان جاری ایجاد می‌شود.
- اگر اکنون سه سطل با اندازه ۱ داشته باشیم، دو سطل قدیمی‌تر در هم ادغام شده و یک سطل با اندازه ۲ تشکیل می‌دهند.
- این ادغام به صورت زنجیره‌ای به سمت اندازه‌های بزرگ‌تر ادامه می‌یابد تا قانون «حداکثر دو سطل از هر اندازه» برقرار بماند.

۳. هر سطلی که زمان انتهای آن قدیمی‌تر از N واحد زمانی گذشته باشد، از انتهای پنجره حذف می‌گردد.

پاسخ به پرس‌وجوی «تعداد اها در k موقعیت اخیر» $(k \leq N)$:

- تمامی سطل‌هایی را که کاملاً در بازه k قرار دارند پیدا کرده و اندازه‌های آن‌ها را به طور کامل جمع بزنید.
- برای آخرین سطل که تا حدی با ابتدای پنجره تداخل دارد، دقیقاً نیمی از اندازه آن را به حاصل جمع اضافه کنید.

قضیه ۲.۴: کران خطای الگوریتم DGIM

حداکثر درصد خطای الگوریتم DGIM هرگز از ۵۰٪ تجاوز نمی‌کند.

اثبات: فرض کنید آخرین سطل با اندازه 2^j دارای تداخل باشد. تخمین ما از این سطل برابر با 2^{j-1} است؛ لذا حداکثر خطای مطلق ممکن برابر با $2^{j-1} - 1$ است. از طرفی، چون قبل از این سطل حداقل یک سطل از هر یک از اندازه‌های $1, 2, 4, \dots, 2^{j-1}$ وجود دارد، مجموع واقعی اها حداقل برابر با $2^j = \sum_{i=0}^{j-1} 2^i + 1$ خواهد بود. بنابراین نسبت خطا:

$$\text{Error} \leq \frac{2^{j-1}}{2^j} = 50\% \quad (11.4)$$

(با افزایش سقف تعداد سطل‌ها از ۲ به r ، می‌توان خطا را به دلخواه تا حد $1/r$ کاهش داد).

۶.۴ تخمین گشتاورهای جریان: الگوریتم آلون-ماتیاس-سگدی (AMS)

در تحلیل آماری جریان، گشتاور k -ام^v به صورت زیر تعریف می‌شود:

$$F_k = \sum_{v \in V} m_v^k \quad (12.4)$$

^v k -th Moment

که در آن m_v فراوانی رخداد مقدار v در جریان است:

• **گشتاور صفرم** (F_0): تعداد عناصر متمایز است (توسط الگوریتم فلژوله-مارتین حل شد).

• **گشتاور یکم** (F_1): طول کل جریان ($N = \sum m_v$).

• **گشتاور دوم** (F_2): **عدد شگفتی** (Surprise Number) نام دارد و مقیاسی از میزان کجی (چولگی) یا ناهمگونی توزیع است. اگر همه عناصر فراوانی یکسان داشته باشند، F_2 کمینه است؛ و اگر یک عنصر کل جریان را تسخیر کند، F_2 بیشینه (N^2) خواهد بود.

الگوریتم ۴.۴: AMS برای محاسبه گشتاور دوم F_2

الگوریتم Alon-Matias-Szegedy یک برآوردگر نااریب شگفت‌انگیز برای F_2 در فضای محدود است:

۱. یک موقعیت تصادفی p در بازه زمانی $[1, N]$ انتخاب کرده و عنصر دیده‌شده در آن لحظه (v) را ذخیره کنید.

۲. متغیری به نام c تعریف کرده و تعداد دفعات مشاهده مجدد v را از موقعیت p تا انتهای جریان بشمارید (در ابتدا $c = 1$).

۳. در پایان، مقدار برآوردگر را به صورت زیر محاسبه نمایید:

$$X = N(2c - 1) \quad (13.4)$$

قضیه ۳.۴: نااریب بودن برآوردگر AMS

امید ریاضی متغیر تصادفی X در الگوریتم AMS دقیقاً برابر با گشتاور دوم واقعی F_2 است:

$$\mathbb{E}[X] = F_2 \quad (14.4)$$

اثبات: فرض کنید عنصر v با فراوانی کلی m_v در جریان وجود دارد. احتمال اینکه موقعیت تصادفی p بر روی یکی از m_v وقوع قرار گیرد برابر با $1/N$ است. اگر بر روی i -امین وقوع از آخر قرار گیرد، متغیر شمارش برابر با i خواهد شد ($i \in \{1, \dots, m_v\}$). بنابراین:

$$\mathbb{E}[X] = \sum_{v \in V} \frac{1}{N} \sum_{i=1}^{m_v} N(2i - 1) = \sum_{v \in V} \sum_{i=1}^{m_v} (2i - 1) \quad (15.4)$$

چون مجموع m عدد فرد اول برابر با مربع کامل m^2 است ($\sum_{i=1}^m (2i - 1) = 1 + 3 + 5 + \dots + (2m - 1) = m^2$):

$$\mathbb{E}[X] = \sum_{v \in V} m_v^2 = F_2 \quad (16.4)$$

با نگهداری چندین نمونه متغیر تصادفی X و محاسبه میانگین میانه‌های آن‌ها، واریانس تخمین به سرعت کاهش می‌یابد.

۷.۴ پنجره‌های با کاهش نمایی (Decaying Windows)

در بسیاری از کاربردهای تحلیل جریان (نظیر روندهای توییت‌ر یا معاملات بورس)، رخدادهای اخیر از اهمیت بسیار بالاتری نسبت به رویدادهای دوردست برخوردارند. در **پنجره با کاهش نمایی**^۸، وزن هر رویداد با گذشت زمان به صورت نمایی مستهلک می‌شود:

$$w(t) = e^{-\lambda(t_{\text{now}} - t)} \approx (1 - c)^{t_{\text{now}} - t} \quad (17.4)$$

مزیت بی‌نظیر این رویکرد در محاسبات کلان‌داده این است که برای به‌روزرسانی مجموع اوزان در هنگام ورود یک عنصر جدید، نیازی به نگهداری هیچ تاریخچه‌ای نیست؛ تنها کافی است مقدار مجموع پیشین را در ضریب تخفیف $(1 - c)$ ضرب کرده و عدد ۱ را به آن بیفزاییم:

$$S_{\text{new}} = S_{\text{old}} \times (1 - c) + 1 \quad (18.4)$$

این فرمول در زمان و فضای ثابت $O(1)$ اجرا شده و بار محاسباتی سرورهای تحلیل بلادرنگ را به صفر نزدیک می‌کند.

جمع‌بندی فصل چهارم

- در جریان‌های داده، امکان ذخیره‌سازی کل تاریخچه وجود ندارد؛ پردازش باید یک‌گذره و با حافظه زیرخطی انجام شود.
- نمونه‌گیری مخزنی به انتخاب تصادفی یکنواخت از میان عناصر با اندازه نامعلوم جریان امکان می‌دهد.
- فیلتر بلوم با حداقل مصرف حافظه، عضویت در مجموعه‌ها را با صفر درصد منفی کاذب بررسی می‌کند.
- الگوریتم Flajolet-Martin شمارش عناصر متمایز را از روی بیشینه صفرهای متوالی درهم‌سازی تخمین می‌زند.
- الگوریتم DGIM شمارش عناصر پنجره لغزان را در فضای $O(\log^2 N)$ با خطای تضمین‌شده کنترل می‌نماید.

فصل ۵

تحلیل پیوندها و رتبه‌بندی وب (Link Analysis & PageRank)

۱.۵ مقدمه: گراف وب و چالش بازیابی اطلاعات

در روزهای آغازین وب، موتورهای جستجوی اولیه (نظیر Altavista و Lycos) اسناد را صرفاً بر پایه تطابق متنی و شمارش کلمات کلیدی (مشابه شاخص TF-IDF) رتبه‌بندی می‌کردند. این رویکرد به سرعت با شکست مواجه شد؛ زیرا طراحان سایت‌ها با تکرار هزاران باره کلمات کلیدی پر جستجو با رنگ سفید در پس‌زمینه سفید، نتایج موتورهای جستجو را فریب می‌دادند (پدیده هرزنامه متنی وب^۱).

انقلاب واقعی زمانی رخ داد که لری پیج و سرگئی برین، بنیان‌گذاران گوگل، دریافتند که وب صرفاً مجموعه‌ای از اسناد متنی مجزا نیست، بلکه یک **گراف جهت‌دار عظیم** است که در آن:

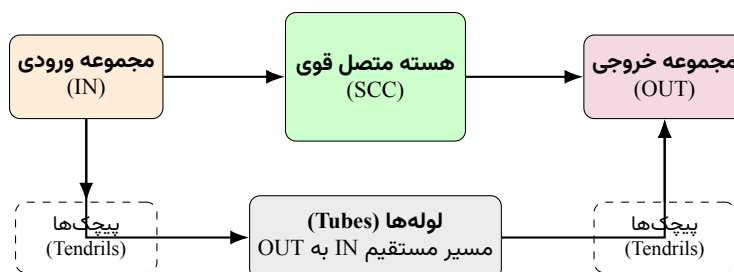
• گره‌ها صفحات وب هستند.

• یال‌های جهت‌دار، ابرپیوندها^۲ میان صفحات هستند.

ایده بنیادین این بود: یک پیوند از صفحه A به صفحه B ، به منزله یک **رای اعتماد**^۳ از سوی نویسنده صفحه A به کیفیت و اعتبار صفحه B است. با این حال، ارزش همه آرا یکسان نیست؛ رای صفحه‌ای معتبر و شناخته‌شده (نظیر دانشگاه استنفورد یا یاهو) باید وزن بسیار بیشتری نسبت به رای یک وبلاگ تازه‌تأسیس داشته باشد. این بینش ریاضی، اساس الگوریتم **PageRank** [۱۱] را تشکیل می‌دهد.

۲.۵ ساختار کلان گراف وب: مدل پاپیونی (Bow-Tie Model)

تحقیقات تجربی بر روی ساختار گراف وب نشان داده است که این گراف دارای یک ساختار منسجم به نام **مدل پاپیونی**^۴ است (شکل ۱.۵):



شکل ۱.۵: مدل پاپیونی ساختار گراف جهانی وب (Web Graph Bow-Tie Architecture)

اجزای ساختار پاپیونی عبارت‌اند از:

- Web Term Spam^۱
- Hyperlinks^۲
- Vote of Confidence^۳
- Bow-Tie Architecture^۴

۱. **هسته متصل قوی (SCC):** بخشی از گراف که میان هر دو صفحه آن، یک مسیر جهت‌دار دوطرفه وجود دارد (قلب تپنده وب).
۲. **بخش ورودی (IN):** صفحاتی که می‌توان از آن‌ها به SCC رسید، اما هیچ مسیری از SCC به آن‌ها وجود ندارد (مانند سایت‌های شخصی جدید).
۳. **بخش خروجی (OUT):** صفحاتی که از طریق SCC قابل دسترسی هستند، اما هیچ پیوندی به عقب باز نمی‌گردانند (مانند صفحات قوانین یا دانلود فایل).
۴. **پیچک‌ها و لوله‌ها (Tendrils & Tubes):** لوله‌ها مسیرهای مستقیمی هستند که بدون عبور از SCC، مستقیماً بخش IN را به OUT متصل می‌کنند؛ پیچک‌ها مسیرهای بن‌بست آویزان به بخش‌های جانبی هستند.

۳.۵ فرمول‌بندی ریاضی الگوریتم PageRank

تعریف ۱.۵: رتبه‌بندی صفحه (PageRank) اولیه

فرض کنید گراف وب دارای N صفحه باشد. رتبه صفحه j که با r_j نشان داده می‌شود، برابر است با مجموع سهم رتبه‌های صفحاتی که به j پیوند داده‌اند:

$$r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i} \quad (1.5)$$

که در آن d_i بیانگر **درجه خروجی^a** صفحه i است؛ یعنی صفحه i رتبه خود را به طور مساوی میان تمامی لینک‌های خروجی‌اش توزیع می‌کند.

Out-degree^a

۱.۳.۵ بیان ماتریسی و بردار ویژه

برای فرمول‌بندی مسئله در قالب جبر خطی، **ماتریس انتقال تصادفی^۵** M با ابعاد $N \times N$ را تعریف می‌کنیم:

$$M_{ji} = \begin{cases} \frac{1}{d_i} & \text{وجود داشته باشد } j \text{ به } i \text{ اگر پیوند جهت‌دار از} \\ 0 & \text{در غیر این صورت} \end{cases} \quad (2.5)$$

توجه فرمایید که ستون‌های ماتریس M جمع‌شان برابر با ۱ است (ماتریس ستون-تصادفی). معادله توزیع رتبه به فرم ماتریسی زیر بازنویسی می‌شود:

$$r = M \cdot r \quad (3.5)$$

این معادله نشان می‌دهد که بردار PageRank (r)، در واقع **بردار ویژه راست متناظر با مقدار ویژه $\lambda = 1$** برای ماتریس انتقال M است. طبق نظریه زنجیره‌های مارکوف، این بردار همان **توزیع پایدار^۶** یک گردش تصادفی^۷ بی‌نهایت در گراف وب است.

۲.۳.۵ الگوریتم تکرار توانی (Power Iteration)

برای محاسبه بردار رتبه در مقیاس وب، از روش تکرار توانی استفاده می‌شود:

Stochastic Transition Matrix^۵
Stationary Distribution^۶
Random Walk^۷

الگوریتم ۱.۵: محاسبه PageRank با روش تکرار توانی

• **مقداردهی اولیه:** بردار رتبه به صورت یکنواخت مقداردهی می‌شود:

$$r^{(0)} = \left[\frac{1}{N}, \frac{1}{N}, \dots, \frac{1}{N} \right]^T \quad (۴.۵)$$

• **تکرار گام‌ها:** در هر مرحله $t + 1$:

$$r^{(t+1)} = M \cdot r^{(t)} \quad (۵.۵)$$

• **شرط توقف:** تا زمانی که تغییر بردار کمتر از آستانه خطا شود: $\|r^{(t+1)} - r^{(t)}\|_1 < \epsilon$ (معمولاً ۵۰ تا ۱۰۰ تکرار برای همگرایی کامل کافی است).

قضیه ۱.۵: نرخ همگرایی تکرار توانی و دومین مقدار ویژه

ماتریس تصادفی گوگل $A = \beta M + (1 - \beta) \frac{1}{n} ee^T$ یک ماتریس تصادفی ستونی مارکوف، مثبت، کاهش‌ناپذیر و نامتناوب است. طبق قضیه پرون-فرو Frobenius-Perron:

۱. بزرگ‌ترین مقدار ویژه این ماتریس دقیقاً یکتا و برابر با $\lambda_1 = 1$ است.

۲. قدرمطلق دومین مقدار ویژه ماتریس از ضریب میرایی تبعیت می‌کند: $|\lambda_2| \leq \beta$.

۳. پس از k گام تکرار توانی، مولفه خطا با نرخ هندسی β^k منقبض می‌شود:

$$\|r^{(k)} - r^*\| \leq C \cdot \beta^k \quad (۶.۵)$$

با انتخاب متداول $\beta = ۰.۸۵$ ، پس از ۵۰ تکرار مقدار خطا به زیر $۰.۰۰۰۳ \approx ۰.۸۵^{۵۰}$ رسیده و مستقل از تعداد میلیاردها صفحه وب، همگرایی تضمین می‌گردد.

۴.۵ ناهنجاری‌های گراف وب و راهکارهای اصلاحی

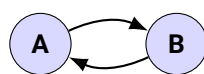
الگوریتم ساده فوق در مواجهه با ساختار واقعی وب با دو آسیب ساختاری مرگبار روبرو می‌شود:

۱.۴.۵ ۱. تله‌های عنکبوتی (Spider Traps)

تله عنکبوتی به مجموعه‌ای از صفحات گفته می‌شود که پیوندهایی میان خود دارند، اما هیچ پیوند خروجی به سایر صفحات وب ندارند (شکل ۲.۵). در هنگام گردش تصادفی، وب‌گرد وارد این حلقه شده و دیگر هرگز نمی‌تواند از آن خارج شود. در نتیجه تکرار توانی، تمامی رتبه صفحات وب جذب این تله شده و رتبه سایر صفحات به سمت صفر میل می‌کند!

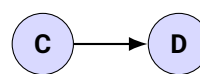
۲.۴.۵ ۲. بن‌بست‌ها (Dead Ends)

بن‌بست صفحه‌ای است که هیچ پیوند خروجی ندارد ($d_i = 0$). در این حالت، ستون متناظر در ماتریس M کاملاً صفر خواهد بود؛ در نتیجه مجموع مقادیر بردار رتبه در هر تکرار کاهش می‌یابد و اصطلاحاً رتبه وب «نشت» کرده و در نهایت صفر می‌شود.



تله عنکبوتی:

رتبه درون حلقه محبوس می‌شود



بن‌بست:

صفحه D لینک خروجی ندارد

شکل ۲.۵: دو چالش اساسی گردش تصادفی: تله عنکبوتی و بن‌بست

۵.۵ الگوریتم نهایی گوگل با تله‌پورتیشن (مالیات‌گیری)

برای رفع تله‌های عنکبوتی و بن‌بست‌ها، مدل گردشگر تصادفی با جهش^۸ ابداع شد.

تعریف ۲.۵: فرمول نهایی PageRank

فرض می‌کنیم وب‌گرد در هر گام:

- با احتمال β ، یکی از لینک‌های خروجی صفحه جاری را به صورت تصادفی دنبال می‌کند.
- با احتمال $1 - \beta$ ، تصمیم می‌گیرد جهش کند (تله‌پورت شود) و به صورت کاملاً تصادفی و یکنواخت به یکی از N صفحه موجود در سراسر وب برود.

معادله به‌روزرسانی رتبه به فرم زیر اصلاح می‌گردد:

$$r = \beta Mr + \frac{1 - \beta}{N} \mathbf{1} \quad (۷.۵)$$

که در آن $\mathbf{1}$ بردار ستونی از تماماً یک‌ها به طول N است. ضریب β معمولاً در بازه ۰.۸ تا ۰.۸۵ تنظیم می‌شود.

اگر صفحه‌ای بن‌بست باشد ($d_i = ۰$)، فرض می‌شود که با احتمال مساوی $1/N$ به تمام صفحات وب پیوند دارد؛ در نتیجه ماتریس اصلاح‌شده کاملاً تصادفی و غیرقابل تقلیل^۹ شده و بر اساس قضیه پرن-فروبنیوس^{۱۰}، همگرایی به بردار ویژه یکتای مثبت تضمین می‌گردد.

مثال کاربردی ۱.۵: محاسبه عددی PageRank با تله‌پورتیشن

فرض کنید گرافی با ۳ صفحه A, B, C داریم که در آن: $A \rightarrow B, C$ ، $B \rightarrow C$ ، و $C \rightarrow A$. ماتریس انتقال M :

$$M = \begin{bmatrix} 0 & 0 & 1 \\ 1/2 & 0 & 0 \\ 1/2 & 1 & 0 \end{bmatrix} \quad (۸.۵)$$

با فرض $\beta = ۰.۸$ ، بردار اولیه $r^{(۰)} = [1/3, 1/3, 1/3]^T$ است. سهم تله‌پورتیشن برابر با $۰.۰۶۷ = \frac{1-\beta}{3} \approx \frac{۰.۲}{3}$ خواهد بود. در گام اول تکرار:

$$r^{(1)} = ۰.۸ \begin{bmatrix} 0 & 0 & 1 \\ 1/2 & 0 & 0 \\ 1/2 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix} + \begin{bmatrix} ۰.۰۶۷ \\ ۰.۰۶۷ \\ ۰.۰۶۷ \end{bmatrix} = \begin{bmatrix} ۰.۲۶۷ + ۰.۰۶۷ \\ ۰.۱۳۳ + ۰.۰۶۷ \\ ۰.۴۰۰ + ۰.۰۶۷ \end{bmatrix} = \begin{bmatrix} ۰.۳۳۴ \\ ۰.۲۰۰ \\ ۰.۴۶۷ \end{bmatrix} \quad (۹.۵)$$

با ادامه تکرارها، صفحه C بالاترین رتبه را کسب می‌کند زیرا از هر دو صفحه A و B لینک دریافت می‌نماید.

۶.۵ الگوریتم‌های پیشرفته: رتبه‌بندی حساس به موضوع و TrustRank

۱.۶.۵ PageRank وابسته به موضوع (Topic-Sensitive PageRank)

اگر کاربری واژه «جاوا» را جستجو کند، آیا به دنبال جزیره جاوا در اندونزی است یا زبان برنامه‌نویسی جاوا؟ در Topic-Sensitive PageRank، به جای آنکه احتمال تله‌پورتیشن به صورت یکنواخت میان تمام صفحات تقسیم شود، وب‌گرد تنها به مجموعه‌ای از صفحات معتبر مرتبط با آن موضوع خاص (مجموعه S) جهش می‌کند:

$$r = \beta Mr + (1 - \beta) \frac{e_S}{|S|} \quad (۱۰.۵)$$

که در آن e_S برداری است که درایه‌های متعلق به مجموعه S برابر با ۱ و سایر درایه‌ها صفر است.

Random Surfer with Teleportation^۸
Irreducible^۹
Perron-Frobenius Theorem^{۱۰}

۲.۶.۵ مبارزه با مزارع اسپم با TrustRank

یک مزرعه اسپم^{۱۱} شامل یک صفحه هدف t و میلیون‌ها صفحه جعلی ساخته‌شده توسط اسپمر است که همگی به t لینک می‌دهند تا رتبه آن را به طور مصنوعی بالا ببرند. الگوریتم TrustRank برای مقابله با این پدیده:

۱. مجموعه‌ای از صفحات با اعتماد قطعی (دامنه‌های دانشگاهی edu و دولتی gov) توسط داوران انسانی انتخاب می‌شوند.

۲. الگوریتم تله‌پورتیشن تنها جهش به این صفحات معتبر را مجاز می‌داند.

۳. صفحات اسپم چون دور از صفحات معتبر هستند، نمره اعتماد (Trust) نزدیک به صفر دریافت می‌کنند.

۷.۵ الگوریتم قطب‌ها و مراجع (HITS)

الگوریتم HITS که توسط جان کلینبرگ ابداع شد [۹]، دیدگاهی دوگانه نسبت به اهمیت صفحات ارائه می‌دهد:

تعریف ۳.۵: قطب‌ها و مراجع

- **مرجع (Authority):** صفحه‌ای با محتوای عمیق و ارزشمند درباره یک موضوع (دارای نمره a).
- **قطب (Hub):** صفحه‌ای که خود محتوای خاصی ندارد، اما لینک‌های فوق‌العاده‌ای به مراجع معتبر ارائه می‌دهد (نظیر صفحات فهرست پیوندها یا دایرکتوری‌ها، دارای نمره h).

رابطه بازگشتی میان قطب و مرجع:

$$a = A^T \cdot h \quad (۱۱.۵)$$

$$h = A \cdot a \quad (۱۲.۵)$$

که در آن A ماتریس مجاورت گراف است ($A_{ij} = 1$ اگر پیوند از i به j باشد). با ترکیب این دو رابطه:

$$a = A^T(Aa) = (A^T A)a, \quad h = A(A^T h) = (AA^T)h \quad (۱۳.۵)$$

یعنی بردار مراجع، بردار ویژه ماتریس AA^T و بردار قطب‌ها، بردار ویژه ماتریس AA^T است!

نکته امتحانی و کلیدی

مقایسه کلیدی میان PageRank و HITS در آزمون‌ها:

۱. **استقلال از پرس‌وجو:** PageRank مستقل از پرس‌وجو بوده و به صورت آفلاین برای کل وب محاسبه می‌شود؛ اما HITS وابسته به پرس‌وجو است و بر روی زیرگراف حاصل از نتایج اولیه جستجو محاسبه می‌گردد.

۲. **حساسیت به اسپم:** HITS به شدت نسبت به دستکاری لینک‌ها (افزودن لینک‌های متقابل قطبی) آسیب‌پذیرتر از PageRank است.

جمع‌بندی فصل پنجم

- وب یک گراف جهت‌دار با معماری پایبونی است.
- الگوریتم PageRank با مدل‌سازی گردش تصادفی، اعتبار ساختاری صفحات را می‌سنجد.
- تله‌های عنکبوتی و بن‌بست‌ها با سازوکار تله‌پورتیشن تصادفی (β) درمان می‌شوند.
- الگوریتم‌های PageRank Topic-Sensitive و TrustRank رتبه‌بندی را برای موضوعات و جلوگیری از اسپم شخصی‌سازی می‌کنند.

• الگوریتم HITS صفحات را به دو طبقه مراجع و قطب‌ها تفکیک می‌نماید.

فصل ۶

مجموعه‌آیتم‌های پرتکرار و قوانین انجمنی

۱.۶ مقدمه و مدل سبد بازار (Market-Basket Model)

یکی از کلاسیک‌ترین و در عین حال پرکاربردترین الگوها در کلان‌داده، کشف وابستگی‌ها میان اقلامی است که معمولاً به صورت هم‌زمان رخ می‌دهند. این چارچوب تحلیلی تحت عنوان **مدل سبد بازار**^۱ شناخته می‌شود. در مدل سبد بازار:

- مجموعه‌ای بزرگ از آیتم‌ها^۲ وجود دارد (مانند کالاها در فروشگاه آمازون).
 - مجموعه‌ای بسیار بزرگ از **سبدها**^۳ وجود دارد که هر سبد، زیرمجموعه‌ای کوچک از آیتم‌ها است (مانند سبد خرید یک مشتری در یک بار مراجعه).
 - فرض بر این است که تعداد کل آیتم‌ها ممکن است در حد میلیون‌ها، و تعداد سبدها در حد صدها میلیون باشد، اما تعداد آیتم‌های درون هر سبد معمولاً اندک (مثلاً کمتر از ۲۰ قلم) است.
- کاربردهای این مدل فراتر از خرده‌فروشی است:
- **کاوش مفاهیم و کلمات هم‌نشین:** آیتم‌ها کلمات و سبدها اسناد متنی هستند (کشف عبارات مرکب).
 - **زیست‌انفورماتیک:** آیتم‌ها نشانگرهای زیستی و سبدها پرونده ژنتیکی بیماران مبتلا به یک بیماری خاص هستند.
 - **تحلیل سرقت ادبی:** آیتم‌ها جملات و سبدها مقالات هستند.

۲.۶ تعاریف پایه: پشتیبانی، اطمینان و قوانین انجمنی

تعریف ۱.۶: پشتیبانی یک مجموعه‌آیتم

پشتیبانی^a یک مجموعه‌آیتم I که با $s(I)$ نشان داده می‌شود، برابر است با تعداد سبدهایی که I زیرمجموعه‌ای از آن‌ها است:

$$s(I) = |\{B \in \text{Baskets} : I \subseteq B\}| \quad (1.6)$$

اگر یک **آستانه پشتیبانی**^b مانند s داشته باشیم، هر مجموعه‌آیتم I که شرط $s(I) \geq s$ را ارضا کند، یک **مجموعه‌آیتم پرتکرار**^c نامیده می‌شود.

Support^a
Support Threshold^b
Frequent Itemset^c

Market-Basket Model^۱
Items^۲
Baskets^۳

تعریف ۲.۶: قوانین انجمنی، اطمینان و بهره

یک **قانون انجمنی**^a به فرم $I \rightarrow j$ بیان می‌شود؛ یعنی «اگر سبدی شامل مجموعه‌آیتم I باشد، آنگاه این سبد با احتمال بالایی شامل آیتم j نیز خواهد بود».

• **اطمینان (Confidence):** نسبت احتمال وقوع توأم به احتمال وقوع شرط:

$$\text{conf}(I \rightarrow j) = \frac{s(I \cup \{j\})}{s(I)} \quad (۲.۶)$$

• **بهره (Interest):** تفاوت میان اطمینان قانون و احتمال وقوع تصادفی آیتم j در کل سبدها:

$$\text{interest}(I \rightarrow j) = \text{conf}(I \rightarrow j) - P(j) = \text{conf}(I \rightarrow j) - \frac{s(\{j\})}{|Baskets|} \quad (۳.۶)$$

قوانینی جذاب و ارزشمند هستند که بهره آن‌ها مقدار مثبت بزرگی داشته باشد (همبستگی مثبت قوی).

Association Rule^a

مثال کاربردی ۱.۶: محاسبه اطمینان و بهره

فرض کنید در یک فروشگاه آنلاین با ۱۰,۰۰۰ سبد:

• سبدهای حاوی پوشک: ۱۰۰۰ $s(\text{پوشک}) = ۰.۱$ $(P(\text{پوشک}))$.

• سبدهای حاوی شیر خشک: ۲۰۰۰ $s(\text{شیر}) = ۰.۲$ $(P(\text{شیر}))$.

• سبدهای حاوی هر دو: ۸۰۰ $s(\{\text{پوشک}, \text{شیر}\})$.

برای قانون شیر $\rightarrow$ پوشک:

$$\text{conf}(\text{شیر} \rightarrow \text{پوشک}) = \frac{۸۰۰}{۱۰۰۰} = ۰.۸ \quad (۸۰\%) \quad (۴.۶)$$

$$\text{interest}(\text{شیر} \rightarrow \text{پوشک}) = ۰.۸ - ۰.۲ = +۰.۶ \quad (۵.۶)$$

چون بهره مثبت و بالاست، این قانون همبستگی قوی خرید همزمان را اثبات می‌کند.

۳.۶ اصل یکنوایی و الگوریتم A-Priori

چالش محاسباتی اصلی در یافتن مجموعه‌آیتم‌های پرتکرار، رشد نمایی ترکیبات است. اگر N آیتم داشته باشیم، تعداد کل زیرمجموعه‌های ممکن ۲^N است. کلید غلبه بر این انفجار ترکیبیاتی، **اصل یکنوایی**^f است.

قضیه ۱.۶: اصل یکنوایی پشتیبانی

اگر یک مجموعه‌آیتم I پرتکرار باشد، آنگاه تمامی زیرمجموعه‌های آن نیز قطعاً پرتکرار هستند. به بیان عکس (عکس نقیض): اگر مجموعه‌آیتمی نامتکرار باشد، هیچ اُبرمجموعه‌ای از آن نمی‌تواند پرتکرار باشد.

$$J \subseteq I \implies s(I) \leq s(J) \quad (۶.۶)$$

Principle Apriori یا Property Monotonicity^f

۱.۳.۶ مراحل اجرای الگوریتم A-Priori

الگوریتم A-Priori در گذرها^۵ متوالی بر روی داده‌های سبدها عمل می‌کند:

الگوریتم ۱.۶: الگوریتم استاندارد A-Priori []

• گذر اول (Pass 1):

۱. با یک بار خواندن سبدها از روی دیسک، فراوانی هر آیتم منفرد شمرده می‌شود.
۲. آیتم‌هایی که فراوانی آن‌ها $\geq s$ باشد، مجموعه آیتم‌های ۱-تایی پرتکرار (L_1) را تشکیل می‌دهند.
- تولید نامزدهای ۲-تایی (C_2): تنها جفت‌هایی $\{i, j\}$ به عنوان نامزد در نظر گرفته می‌شوند که هر دوی $i \in L_1$ و $j \in L_1$ باشند.
- گذر دوم (Pass 2): سبدها مجدداً از دیسک خوانده می‌شوند. برای هر سبد، تمامی جفت‌های درون آن که در C_2 عضویت دارند پیدا شده و شمارنده آن‌ها یکی افزایش می‌یابد. در پایان گذر دوم، جفت‌های با شمارش $\geq s$ به عنوان L_2 انتخاب می‌شوند.
- گذر k -ام (Pass k): مجموعه نامزدهای C_k از روی L_{k-1} تولید می‌شود، سبدها خوانده شده و L_k به دست می‌آید. این فرایند تا زمانی ادامه می‌یابد که هیچ نامزد جدیدی تولید نشود.

۴.۶ الگوریتم‌های بهینه‌سازی مصرف حافظه

در الگوریتم A-Priori، گلوگاه اصلی حافظه در **گذر دوم** (شمارش تعداد جفت‌های نامزد C_2) رخ می‌دهد. الگوریتم‌های زیر برای حل این گلوگاه ابداع شدند:

۱.۴.۶ الگوریتم پارک-چن-یو (PCY)

مشاهده کلیدی: در گذر اول الگوریتم A-Priori، تنها کاری که انجام می‌شود شمارش آیتم‌های تکی است که حافظه بسیار ناچیزی نیاز دارد؛ در نتیجه بخش اعظم حافظه اصلی کامپیوتر در گذر اول بیکار می‌ماند!

ایده الگوریتم PCY:

۱. در گذر اول، از حافظه آزاد باقیمانده به عنوان یک جدول درهم‌سازی بزرگ از سطرها^۶ استفاده می‌کنیم.
۲. هنگام خواندن هر سبد در گذر اول، علاوه بر شمارش آیتم‌های تکی، به ازای هر جفت $\{i, j\}$ در سبد، مقدار درهم‌سازی $h(i, j)$ را محاسبه کرده و شمارنده آن سطر را یک واحد افزایش می‌دهیم.
۳. در پایان گذر اول، اگر مقدار یک سطر کمتر از آستانه s باشد، امکان ندارد هیچ جفت پرتکراری به آن سطر نگاشت شده باشد.
۴. جدول درهم‌سازی به یک نقشه بیتی^۷ فشرده تبدیل می‌شود (۱ بیت برای هر سطر: ۱ اگر مجموع سطر $\geq s$ باشد، وگرنه ۰).

۵. در گذر دوم: یک جفت $\{i, j\}$ به عنوان نامزد شمارش می‌شود اگر و تنها اگر:

$$i \in L_1 \text{ و } j \in L_1 \text{ و } \text{Bitmap}[h(i, j)] = 1 \quad (۷.۶)$$

این شرط سه‌گانه، تعداد جفت‌های نامزد در گذر دوم را به شدت کاهش می‌دهد.

^۵ Passes
^۶ Hash Table of Buckets
^۷ Bitmap

۲.۴.۶ الگوریتم‌های چندمرحله‌ای و چنددرهم‌سازی (Multistage & Multihash)

- **الگوریتم چندمرحله‌ای (Multistage):** یک گذر میانی (گذر ۵.۱) اضافه می‌کند که در آن، تنها جفت‌هایی که از فیلتر سطل اول عبور کرده‌اند با یک تابع درهم‌سازی مستقل دوم هاش می‌شوند؛ بدین ترتیب نقشه بیتی دوم تولید شده و تعداد نامزدهای گذر نهایی باز هم کاهش می‌یابد.
- **الگوریتم چنددرهم‌سازی (Multihash):** در همان گذر اول، حافظه را به دو یا چند جدول درهم‌سازی مستقل با دو تابع هاش مجزا تقسیم می‌کند.

۵.۶ الگوریتم‌های با تعداد گذر محدود (یک یا دو گذره)

در کلان‌داده، خواندن مکرر ترابایت‌ها داده از دیسک زمان‌بر است. الگوریتم‌های زیر یافتن مجموعه‌آیتم‌های پرتکرار را در ۱ یا ۲ گذر میسر می‌سازند:

۱.۵.۶ الگوریتم ساواسره-امینسکی-ناواته (SON)

الگوریتم SON ساختاری کاملاً موازی دارد و بهترین الگوریتم برای پیاده‌سازی در **MapReduce** است:

الگوریتم ۲.۶: الگوریتم SON در MapReduce

۱. گذر اول (گام MapReduce اول):

- داده‌ها به m قطعه مساوی (Chunk) تقسیم می‌شوند.
 - هر میر یک قطعه را خوانده و با استفاده از الگوریتم A-Priori یا PCY، مجموعه‌آیتم‌های پرتکرار محلی با آستانه متناسب s/m را پیدا می‌کند.
 - **لم بنیادین:** هر مجموعه‌آیتمی که در کل داده‌ها پرتکرار باشد، حداقل در یکی از m قطعه باید دارای پشتیبانی $s/m \geq$ باشد.
 - کاهنده اجتماع تمامی مجموعه‌آیتم‌های پرتکرار محلی را به عنوان نامزدهای جهانی منتشر می‌سازد.
۲. **گذر دوم (گام MapReduce دوم):** میرها سبدها را خوانده و پشتیبانی دقیق نامزدهای جهانی را شمارش می‌کنند؛ کاهنده نامزدهایی را که مجموعاً $s \geq$ هستند به عنوان پاسخ قطعی و نهایی برمی‌گزیند.

ویژگی منحصر به فرد SON: دقیقاً در ۲ گذر اجرا می‌شود و هیچ منفی کاذب یا مثبت کاذبی تولید نمی‌کند!

۲.۵.۶ الگوریتم توینون (Toivonen's Algorithm) و مرز منفی

الگوریتم Toivonen با یک گذر بر روی نمونه و یک گذر بر روی کل داده‌ها عمل می‌کند:

۱. نمونه‌ای تصادفی با اندازه کسری p از سبدها برداشته می‌شود.
۲. مجموعه‌آیتم‌های پرتکرار در نمونه با آستانه‌ای اندکی کمتر از $p \cdot s$ استخراج می‌گردند.
۳. **مرز منفی (Negative Border)** تشکیل می‌شود: مجموعه‌آیتم‌هایی که خود در نمونه پرتکرار نشده‌اند، اما تمامی زیرمجموعه‌های بلافاصل آن‌ها پرتکرار بوده‌اند.
۴. در گذر دوم بر روی کل داده‌های اصلی، پشتیبانی تمامی آیتم‌های پرتکرار نمونه و تمامی آیتم‌های مرز منفی به طور هم‌زمان شمرده می‌شود.
۵. **شرط موفقیت:** اگر هیچ‌یک از اعضای مرز منفی در داده‌های اصلی به آستانه s نرسند، نتایج نمونه برای داده‌های اصلی کاملاً دقیق و بدون خطا است! (اگر عضوی از مرز منفی پرتکرار شود، آزمون باطل شده و با نمونه جدید تکرار می‌گردد).

نکته امتحانی و کلیدی

نکات امتحانی پرکاربرد:

۱. **فرمول آدرس‌دهی ماتریس مثلثی:** برای ذخیره شماره‌های جفت‌های نامزد در یک آرایه یک‌بعدی فشرده (بدون اتلاف فضای قطر و بخش بالایی)، درایه (i, j) با شرط $i < j$ در خانه زیر ذخیره می‌شود:

$$k = (i-1) \left(n - \frac{i}{2} \right) + j - i \quad (۸.۶)$$

مقایسه روش ماتریس مثلثی در برابر سه‌تایی‌ها (Triples): هر خانه ماتریس مثلثی ۴ بایت (یک عدد صحیح) فضا می‌گیرد؛ اما روش سه‌تایی‌ها برای هر جفت نیازمند ۱۲ بایت $((i, j, \text{count}))$ است. بنابراین اگر بیش از یک سوم کل جفت‌های ممکن نامزد باشند، ماتریس مثلثی فشرده‌تر است؛ اما اگر جفت‌های نامزد بسیار تنگ باشند، روش سه‌تایی‌ها کاراتر خواهد بود.

۲. **اثبات صفر بودن منفی‌های کاذب در الگوریتم SON:** اگر مجموعه‌آیتم I در کل داده‌ها دارای پشتیبانی حداقل s باشد، غیرممکن است که در تمام m قطعه دارای پشتیبانی کمتر از s/m باشد؛ زیرا در این صورت مجموع کل پشتیبانی اکیداً کمتر از $s \times (s/m) = m \times (s/m)$ می‌شد که تناقض است. بنابراین I قطعاً حداقل در یک قطعه کشف شده و هیچ آیتم پرتکراری از دست نمی‌رود.

۳. **نقش حیاتی مرز منفی در الگوریتم Toivonen:** اگر کوچک‌ترین آیتمی که در کل داده‌ها به حد نصاب رسیده اما در نمونه کشف نشده وجود داشته باشد، طبق اصل یکنوایی تمام زیرمجموعه‌های بلافاصل آن باید در نمونه کشف شده باشند؛ پس آن آیتم حتماً عضوی از مرز منفی است و در گذر دوم شمرده می‌شود.

جمع‌بندی فصل ششم

- مدل سبد بازار همبستگی‌های هم‌نشینی را از طریق معیارهای پشتیبانی، اطمینان و بهره ارزیابی می‌کند.
- اصل یکنوایی با حذف ابرمجموعه‌های اقلام نامتکرار، مانع انفجار ترکیبیاتی می‌شود.
- الگوریتم PCY با به کارگیری حافظه بلااستفاده برای سطل‌های درهم‌سازی، گلوگاه گذر دوم را مرتفع می‌سازد.
- الگوریتم SON چارچوبی بهینه برای موازی‌سازی کاوش در بستر MapReduce تنها در دو گذر است.
- الگوریتم Toivonen به کمک مرز منفی، بررسی صحت نتایج نمونه‌گیری را تضمین می‌نماید.

بخش سوم

خوشه‌بندی و سامانه‌های وب

فصل ۷

خوشه‌بندی در فضاهای با بعد بالا (High-Dimensional Clustering)

۱.۷ مقدمه و چالش‌های خوشه‌بندی کلان‌داده

خوشه‌بندی^۱ فرآیند افراز مجموعه‌ای از اشیاء یا نقاط داده به گروه‌هایی (خوشه‌ها) است، به گونه‌ای که شباهت میان اعضای درون یک خوشه حداکثر، و شباهت میان اعضای خوشه‌های متمایز حداقل باشد. در مبحث کلان‌داده، خوشه‌بندی با دو چالش بنیادین مواجه است:

۱. **نفرین ابعاد (Curse of Dimensionality)**: با افزایش تعداد ابعاد (مثلاً در فضاهای ۱۰۰۰ بعدی متنی یا ویژگی‌های تصویری):

- تقریباً تمامی جفت‌نقاط دارای فواصل اقلیدسی تقریباً برابری از یکدیگر می‌شوند.
- حجم فضا به صورت نمایی رشد کرده و داده‌ها به شدت تنگ می‌شوند؛ به گونه‌ای که تقریباً تمامی نقاط بر روی پوسته یا مرز فضا قرار می‌گیرند.
- تعاریف سنتی فاصله، کارایی تفکیک‌کنندگی خود را از دست می‌دهند.

۲. **محدودیت حافظه اصلی (RAM Bottleneck)**: مجموعه داده‌ها شامل میلیون‌ها یا میلیارد‌ها نقطه هستند و امکان بارگذاری تمامی نقاط در حافظه برای اجرای الگوریتم‌های تکراری وجود ندارد. داده‌ها باید روی دیسک قرار داشته باشند و الگوریتم تنها با یک یا دو گذر از روی دیسک، خوشه‌ها را شناسایی کند.

روش‌های خوشه‌بندی عموماً به دو دسته کلی تقسیم می‌شوند:

- **روش‌های سلسله‌مراتبی^۲**: ساخت درختی از خوشه‌ها به صورت پایین به بالا یا بالا به پایین.
- **روش‌های تخصیص نقطه^۳**: انتساب هر نقطه به مناسب‌ترین خوشه بر اساس توابع هدف و مراکز خوشه‌ها (نظیر K-Means).

۲.۷ خوشه‌بندی سلسله‌مراتبی تراکمی (Agglomerative)

در این روش، ابتدا هر نقطه داده به عنوان یک خوشه تک‌عضوی در نظر گرفته می‌شود. سپس در هر مرحله، دو خوشه‌ای که کمترین فاصله را دارند با یکدیگر ادغام می‌شوند تا در نهایت تمامی نقاط در یک خوشه واحد تجمیع گردند (یا تعداد خوشه‌ها به k برسد).

۱.۲.۷ معیارهای فاصله میان دو خوشه

تعریف فاصله میان دو خوشه C_1 و C_2 تعیین‌کننده شکل نهایی خوشه‌ها است:

Clustering^۱
Hierarchical Clustering^۲
Point Assignment^۳

• پیوند یگانه (Single Linkage): کمترین فاصله میان اعضا:

$$d(C_1, C_2) = \min_{x \in C_1, y \in C_2} d(x, y) \quad (1.7)$$

تمایل به تولید خوشه‌های کشیده و زنجیره‌ای دارد.

• پیوند کامل (Complete Linkage): بیشترین فاصله میان اعضا:

$$d(C_1, C_2) = \max_{x \in C_1, y \in C_2} d(x, y) \quad (2.7)$$

تمایل به تولید خوشه‌های فشرده و کروی دارد.

• فاصله مراکز (Centroid Distance): فاصله میان میانگین‌های دو خوشه $d(\mu_1, \mu_2)$.

۳.۷ الگوریتم K-Means و نسخه بهینه K-Means++

الگوریتم K-Means یکی از پرکاربردترین روش‌های تخصیص نقطه در فضای اقلیدسی است:

الگوریتم ۱.۷: الگوریتم استاندارد K-Means

• ورودی: مجموعه نقاط X و تعداد خوشه‌ها k .

• گام ۱ (مقداردهی اولیه): k نقطه به عنوان مراکز اولیه خوشه‌ها $(c_1, \dots, c_k)$ انتخاب می‌شوند.

• گام ۲ (تخصیص نقاط): هر نقطه $x \in X$ به نزدیک‌ترین مرکز خوشه تخصیص می‌یابد:

$$\text{Cluster}(x) = \arg \min_{j \in \{1, \dots, k\}} \|x - c_j\|^2 \quad (3.7)$$

• گام ۳ (به‌روزرسانی مراکز): مرکز هر خوشه برابر با میانگین هندسی تمامی نقاط تخصیص‌یافته به آن بازتعریف می‌شود:

$$c_j = \frac{1}{|C_j|} \sum_{x \in C_j} x \quad (4.7)$$

• تکرار: گام‌های ۲ و ۳ تا زمان تثبیت مراکز یا کمترین تغییر در تابع هزینه تکرار می‌شوند.

۱.۳.۷ الگوریتم K-Means++ برای مقداردهی اولیه هوشمند

انتخاب تصادفی ساده مراکز اولیه در K-Means سنتی ممکن است منجر به گیر افتادن در کمینه‌های محلی بسیار ضعیف شود. الگوریتم K-Means++ مراکز را با توزیع احتمال هوشمندانه برمی‌گزیند:

۱. اولین مرکز c_1 به صورت کاملاً تصادفی و یکنواخت از میان نقاط داده انتخاب می‌شود.

۲. برای هر نقطه داده x ، کوتاه‌ترین فاصله آن تا نزدیک‌ترین مرکز انتخاب‌شده قبلی، یعنی $D(x)$ ، محاسبه می‌شود.

۳. مرکز بعدی با احتمالی متناسب با مربع فاصله انتخاب می‌گردد:

$$P(x) = \frac{D(x)^2}{\sum_{y \in X} D(y)^2} \quad (5.7)$$

۴. گام‌های ۲ و ۳ تکرار می‌شوند تا هر k مرکز اولیه انتخاب گردند.

این الگوریتم تضمین اثبات‌شده ریاضی ارائه می‌دهد که امید هزینه خوشه نهایی هرگز از $O(\log k)$ برابر بهینه جهانی تجاوز نمی‌کند.

۴.۷ خوشه‌بندی کلان‌داده در فضای اقلیدسی: الگوریتم BFR

الگوریتم BFR^۴ [۳] نسخه بهینه‌سازی شده K-Means برای کلان‌داده‌هایی است که بسیار فراتر از گنجایش حافظه اصلی هستند.

۱.۴.۷ فرضیات بنیادین BFR

۱. داده‌ها در فضای اقلیدسی تعریف شده‌اند.

۲. خوشه‌ها پیرو توزیع نرمال چندمتغیره حول مرکز هستند و ابعاد داده نسبت به یکدیگر استقلال دارند (ماتریس کوواریانس قطری).

۲.۴.۷ خلاصه‌سازی داده‌ها با آماره‌های سه‌گانه $(N, SUM, SUMSQ)$

برای اینکه نیاز به ذخیره تمامی نقاط در حافظه نباشد، هر خوشه توسط سه بردار خلاصه آماری توصیف می‌شود:

(۶.۷) تعداد نقاط موجود در خوشه : N

$$SUM_i = \sum_{j=1}^N x_{ji} \quad (i \text{ مجموع مختصات در بعد } i) \quad (۷.۷)$$

$$SUMSQ_i = \sum_{j=1}^N x_{ji}^2 \quad (i \text{ مجموع مربع مختصات در بعد } i) \quad (۸.۷)$$

از روی این سه آماره، مرکز خوشه c و انحراف معیار σ در هر بعد i بلافاصله قابل استخراج است:

$$c_i = \frac{SUM_i}{N}, \quad \sigma_i = \sqrt{\frac{SUMSQ_i}{N} - c_i^2} \quad (۹.۷)$$

۳.۴.۷ سه مجموعه داده در حافظه اصلی

الگوریتم BFR نقاط بارگذاری شده در حافظه را به سه دسته افراز می‌کند:

- **مجموعه دفع شده (Discard Set - DS):** نقاطی که به اندازه کافی به یکی از k مرکز اصلی نزدیک هستند. این نقاط به آماره‌های $(N, SUM, SUMSQ)$ آن خوشه افزوده شده و خود نقطه بلافاصله از حافظه پاک می‌شود.
- **مجموعه فشرده (Compressed Set - CS):** ریزخوشه‌هایی از نقاط که به هیچ‌یک از k خوشه اصلی نزدیک نیستند، اما به یکدیگر نزدیک‌اند. این ریزخوشه‌ها نیز با آماره‌های سه‌گانه خلاصه می‌شوند.
- **مجموعه نگه‌داشته شده (Retained Set - RS):** نقاط پرت و ایزوله‌ای که نه به خوشه‌های اصلی نزدیک‌اند و نه ریزخوشه‌ای تشکیل می‌دهند. این نقاط به صورت تکی در حافظه باقی می‌مانند تا شاید با ورود داده‌های جدید به خوشه‌ای ملحق شوند.

۴.۴.۷ معیار فاصله ماهالانوبیس (Mahalanobis Distance)

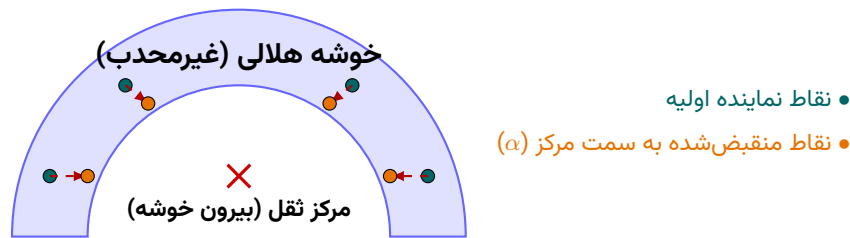
برای تصمیم‌گیری درباره انتساب یک نقطه x به یک خوشه با مرکز c و انحراف معیارهای σ_i ، از فاصله ماهالانوبیس^۵ استفاده می‌شود که واریانس هر بعد را لحاظ می‌کند:

$$d_{\text{Maha}}(x, c) = \sqrt{\sum_{i=1}^d \left(\frac{x_i - c_i}{\sigma_i} \right)^2} \quad (۱۰.۷)$$

اگر فاصله ماهالانوبیس نقطه از مرکز کمتر از آستانه‌ای مشخص (معمولاً بین ۳ تا ۴ برابر انحراف معیار) باشد، نقطه به مجموعه DS تخصیص می‌یابد.

۵.۷ خوشه‌بندی اشکال دلخواه و غیرکروی: الگوریتم CURE

الگوریتم‌های K-Means و BFR فرض می‌کنند خوشه‌ها کروی یا بیضوی همگرا هستند. اما در مسائل واقعی، خوشه‌ها ممکن است اشکال هلالی، تو در تو یا غیرمحدب داشته باشند (شکل ۱.۷). الگوریتم CURE^۶ برای حل این چالش ابداع شده است.



شکل ۱.۷: نمایش نقاط نماینده و مکانیزم انقباض در الگوریتم CURE برای پوشش شکل‌های دلخواه

الگوریتم ۲.۷: الگوریتم CURE

۱. **نمونه‌گیری اولیه:** نمونه‌ای تصادفی از داده‌ها که در حافظه اصلی جا می‌شود برداشته می‌شود.
۲. **خوشه‌بندی سلسله‌مراتبی نمونه:** نمونه به کمک روش سلسله‌مراتبی خوشه‌بندی می‌شود.
۳. **انتخاب نقاط نماینده (Representative Points):** برای هر خوشه، به جای تعریف یک مرکز منفرد، c نقطه نماینده انتخاب می‌شود (نقاطی که بیشترین فاصله را از یکدیگر در درون خوشه دارند).
۴. **فشردگی و انقباض (Shrinkage):** هر نقطه نماینده به اندازه ضریب α (معمولاً $\alpha = 0.2$) به سمت مرکز ثقل خوشه کشیده می‌شود:

$$p_{\text{new}} = p + \alpha(\mu - p) \quad (11.7)$$

این انقباض اثر نقاط پرت و نویزها را به شدت خنثی می‌کند.

۵. **گذر نهایی روی کل داده‌ها:** هر نقطه داده از روی دیسک به خوشه‌ای تخصیص داده می‌شود که نزدیک‌ترین نقطه نماینده به آن را در اختیار دارد.

۶.۷ خوشه‌بندی در فضاهای غیر اقلیدسی: مفهوم شبه‌مرکز (Clustroid)

در بسیاری از مسائل کلان‌داده (نظیر گراف‌ها، فاصله‌های جاکارد یا فواصل کسینوسی)، مفهوم میانگین حسابی مختصات وجود ندارد زیرا نقاط بردار اقلیدسی نیستند. برای مدیریت این وضعیت:

تعریف ۱.۷: شبه‌مرکز (Clustroid)

در یک فضای غیر اقلیدسی، **شبه‌مرکز^a** نقطه‌ای از خود مجموعه داده است که کمترین فاصله را تا سایر نقاط خوشه دارد. بر اساس معیار بهینه‌سازی، یکی از سه تعریف زیر به کار می‌رود:

۱. نقطه‌ای که **مجموع فواصل** تا سایر نقاط خوشه را کمینه کند: $\arg \min_{c \in C} \sum_{x \in C} d(c, x)$.
۲. نقطه‌ای که **مجموع مجذور فواصل** تا سایر نقاط را کمینه کند: $\arg \min_{c \in C} \sum_{x \in C} d^2(c, x)$.
۳. نقطه‌ای که **بیشینه فاصله** تا سایر نقاط (شعاع خوشه) را کمینه کند: $\arg \min_{c \in C} \max_{x \in C} d(c, x)$.

Clustroid^a

معیارهای ارزیابی اندازه خوشه در فضای غیر اقلیدسی:

• شعاع خوشه (Cluster Radius): بیشینه فاصله یا میانگین مربعات فاصله از شبه‌مرکز:

$$R(C) = \sqrt{\frac{1}{|C|} \sum_{x \in C} d^2(x, c)} \quad (12.7)$$

• قطر خوشه (Cluster Diameter): بیشینه فاصله میان هر دو نقطه از خوشه یا جذر میانگین مربعات فواصل دوبره‌دو:

$$D(C) = \sqrt{\frac{1}{|C|(|C| - 1)} \sum_{x \in C} \sum_{y \in C, y \neq x} d^2(x, y)} \quad (13.7)$$

الگوریتم‌های خوشه‌بندی غیر اقلیدسی (نظیر الگوریتم GRGPF) در هر مرحله دو خوشه‌ای را ادغام می‌کنند که پس از ادغام، کمترین شعاع یا قطر حاصل شود.

جمع‌بندی فصل هفتم

- نفرین ابعاد موجب همگرایی تمامی فواصل دوبره‌دو در فضاهای ابعادی بزرگ می‌گردد.
- الگوریتم K-Means++ با انتخاب احتمالاتی بر پایه مجذور فواصل، همگرایی کیفی مراکز را تضمین می‌کند.
- الگوریتم BFR داده‌های بزرگ را از طریق خلاصه آماری سه‌گانه $(N, SUM, SUMSQ)$ و معیار فاصله ماهالانوبیس خوشه‌بندی می‌نماید.
- الگوریتم CURE با تکیه بر چندین نقطه نماینده منقبض‌شده، امکان کشف خوشه‌های با اشکال پیچیده و غیرمحدب را فراهم می‌سازد.

فصل ۸

تبلیغات برخط در وب (Online Advertising & AdWords)

۱.۸ مقدمه: الگوریتم‌های برخط و اقتصاد موتورهای جستجو

درآمد اصلی غول‌های فناوری وب (به‌ویژه گوگل و متا) از تبلیغات هدفمند اینترنتی حاصل می‌شود. تفاوت بنیادین تبلیغات سنتی (نظیر بیلبرد یا آگهی روزنامه) با تبلیغات وب در دو عامل خلاصه می‌شود:

۱. **هدفمندی مستقیم**^۱: نمایش تبلیغ متناسب با نیاز دقیق لحظه‌ای کاربر بر اساس پرس‌وجوی جستجو^۲.
 ۲. **الگوریتم‌های برخط**^۳: ورودی‌های مسئله (پرس‌و‌جوه‌ای کاربران) یکی‌یکی در طول زمان وارد می‌شوند و الگوریتم باید بلافاصله، بدون اطلاع از آینده و به صورت بازگشت‌ناپذیر تصمیم بگیرد که کدام آگهی را نمایش دهد.
- در الگوریتم‌های سنتی، کل ورودی از ابتدا معلوم است (الگوریتم آفلاین)، اما در الگوریتم‌های برخط، تصمیم‌گیری در شرایط نااطمینانی صورت می‌پذیرد.

تعریف ۱.۸: نسبت رقابتی (Competitive Ratio)

کارایی یک الگوریتم برخط در مقایسه با یک الگوریتم بهینه آفلاین (که با برخورداری از ویژگی «پیش‌گویی»، تمام آینده را می‌داند) با معیاری به نام **نسبت رقابتی**^۴ ارزیابی می‌شود:

$$CR = \min_{I \text{ ورودی‌های ممکن}} \frac{\text{سود الگوریتم برخط}(I)}{\text{سود الگوریتم بهینه آفلاین}(I)} \quad (1.8)$$

نسبت رقابتی مقداری بین ۱ و ∞ است. هرچه این نسبت به ۱ نزدیک‌تر باشد، عملکرد الگوریتم برخط بهتر است.

Competitive Ratio^۵

۲.۸ مسئله تطابق دوبخشی برخط (Online Bipartite Matching)

مسئله نمایش تبلیغات را می‌توان در قالب یک گراف دوبخشی $G = (U, V, E)$ مدل‌سازی کرد:

- بخش U : تبلیغ‌دهندگان^۴ که از قبل در سیستم ثبت‌نام کرده‌اند و ثابت هستند.
- بخش V : پرس‌و‌جوه‌ای کاربران^۵ که یکی‌یکی به صورت آنلاین وارد می‌شوند.
- یال (u, v) : نشان می‌دهد که تبلیغ‌دهنده u تمایل به نمایش آگهی برای پرس‌وجوی v دارد.

Targeted Advertising^۱
Search Query^۲
Online Algorithms^۳
Advertisers^۴
Queries^۵

هدف: یافتن بزرگ‌ترین تطابق^۶؛ یعنی بیشترین تعداد یال‌های بدون رأس مشترک.

۱.۲.۸ الگوریتم حریصانه (Greedy Algorithm) و تحلیل نسبت رقابتی

ساده‌ترین استراتژی برخط، الگوریتم حریصانه است: به محض ورود یک پرس‌وجو v ، آن را به هر تبلیغ‌دهنده متصل به آن که تاکنون آگهی به او تخصیص داده نشده، اختصاص می‌دهیم.

قضیه ۱.۸: کران نسبت رقابتی الگوریتم حریصانه

نسبت رقابتی الگوریتم حریصانه برای تطابق دوبخشی برخط حداقل برابر با $\frac{1}{2}$ (۵۰٪) است و این کران تنگ است.

اثبات قضیه: فرض کنید تطابق تولیدشده توسط الگوریتم حریصانه M_{greedy} و تطابق بهینه آفلاین M_{opt} باشد. می‌دانیم هر یال $(u, v) \in M_{\text{opt}}$ باید حداقل با یکی از یال‌های M_{greedy} در رأس u یا v اشتراک داشته باشد (در غیر این صورت الگوریتم حریصانه می‌توانست یال (u, v) را به تطابق خود بیفزاید که با حریصانه بودن آن در تناقض است). چون هر یال در M_{greedy} حداکثر می‌تواند دو سر دو یال مختلف در M_{opt} را لمس کند:

$$|M_{\text{opt}}| \leq 2 \times |M_{\text{greedy}}| \implies \frac{|M_{\text{greedy}}|}{|M_{\text{opt}}|} \geq \frac{1}{2} \quad (۲.۸)$$

بنابراین الگوریتم حریصانه همیشه حداقل نیمی از سود حالت ایده‌آل را کسب می‌کند.

۳.۸ الگوریتم BALANCE با پیشنهادهای برابر

در جهان واقعی، هر تبلیغ‌دهنده دارای یک **بودجه روزانه**^۷ مشخص (مثلاً B دلار) است. اگر پیشنهاد قیمت همه تبلیغ‌دهندگان برای کلمات کلیدی یکسان باشد (مثلاً ۱ دلار به ازای هر کلیک)، استراتژی حریصانه دخواه ممکن است بودجه برخی تبلیغ‌دهندگان را زودتر از موعد تمام کرده و پرس‌و‌جوه‌های بعدی را بدون مشتری باقی بگذارد.

الگوریتم ۱.۸: الگوریتم تعادل (BALANCE)

هنگامی که پرس‌وجوی q وارد می‌شود:

۱. تمامی تبلیغ‌دهندگانی را که برای q پیشنهاد داده‌اند و هنوز بودجه آن‌ها تمام نشده است پیدا کنید.
۲. آگهی را به تبلیغ‌دهنده‌ای اختصاص دهید که **بیشترین بودجه باقیمانده** را دارد (در صورت تساوی، یکی را تصادفی انتخاب کنید).

قضیه ۲.۸: نسبت رقابتی الگوریتم BALANCE

برای مسئله تخصیص آگهی با پیشنهادهای برابر و بودجه‌های متناهی، نسبت رقابتی الگوریتم BALANCE برابر است با:

$$CR = 1 - \frac{1}{e} \approx 0.۶۳۲ \quad (۶۳.۲\%) \quad (۳.۸)$$

این عدد نشان‌دهنده بهبود چشمگیر نسبت به الگوریتم حریصانه ساده (۵۰٪) است.

۴.۸ مسئله عمومی AdWords در گوگل (پیشنهادهای نابرابر)

در سازوکار واقعی سیستم AdWords:

- هر تبلیغ‌دهنده i برای پرس‌وجوی q پیشنهاد قیمت متفاوتی به نام b_{iq} ارائه می‌دهد.

- هر تبلیغ‌دهنده بودجه کل روزانه متفاوتی مانند B_i دارد.
- هدف، بیشینه‌سازی کل درآمد ارزی کسب‌شده از کلیک‌هاست.

۱.۴.۸ تابع امتیازدهی عمومی AdWords

اگر صرفاً به بیشترین پیشنهاد (b_{iq}) توجه کنیم، بودجه تبلیغ‌دهندگان بزرگ خیلی سریع مصرف شده و در ساعات پایانی روز درآمد افت می‌کند. از این رو، الگوریتم عمومی از یک تابع تنزیل غیرخطی استفاده می‌نماید. فرض کنید m_i مقدار پولی باشد که تبلیغ‌دهنده i تاکنون خرج کرده است. کسر بودجه مصرف‌شده برابر با $f_i = m_i/B_i$ است. هنگام ورود پرس‌وجوی q ، تبلیغ‌دهنده برنده بر اساس بیشینه امتیاز ترکیب‌شده انتخاب می‌شود:

$$\Psi_i(q) = b_{iq} \times (1 - e^{-(1-f_i)}) \quad (۴.۸)$$

• اگر تبلیغ‌دهنده‌ای تقریباً تمام بودجه‌اش دست‌نخورده باشد ($f_i \approx 0$)، ضریب تصحیح نزدیک به $1 - 1/e \approx 0.۶۳$ است.

• هرچه بودجه به اتمام نزدیک‌تر شود ($f_i \rightarrow 1$)، ضریب $1 - e^0 = 0$ شده و اولویت آن به شدت کاهش می‌یابد تا فضا برای سایرین باز شود.

اثبات شده است که این الگوریتم تعمیم‌یافته نیز نسبت رقابتی $1 - 1/e$ را تضمین می‌نماید.

۲.۴.۸ نرخ کلیک (CTR) و رتبه‌بندی آگهی‌ها

موتور جستجو تنها در صورتی درآمد کسب می‌کند که کاربر واقعاً بر روی آگهی کلیک نماید (مدل CPC^۸). بنابراین، ارزش انتظاری یک آگهی برابر است با:

$$\text{Revenue Expected} = \text{Bid} \times \text{CTR} \quad (۵.۸)$$

که در آن CTR^۹ نسبت تعداد کلیک‌ها به تعداد نمایش آگهی است. آگهی‌ها بر اساس حاصل‌ضرب پیشنهاد در نرخ کلیک تاریخی (و نمره کیفیت متن آگهی) رتبه‌بندی می‌شوند.

۳.۴.۸ سازوکار حراج قیمت دوم (Vickrey-Clarke-Groves Auction)

گوگل برای ترغیب تبلیغ‌دهندگان به اعلام ارزش واقعی خود، از حراج قیمت دوم تعمیم‌یافته^{۱۰} استفاده می‌کند: برنده حراج، بالاترین پیشنهاد را ثبت کرده است، اما مبلغی که در ازای هر کلیک می‌پردازد برابر با پیشنهاد خودش نیست؛ بلکه برابر است با حداقل مبلغی که برای شکست دادن پیشنهاد رقیب رتبه پایین‌تر لازم بوده است.

قضیه ۳.۸: راست‌گویی به عنوان استراتژی مسلط در حراج قیمت دوم

در حراج قیمت دوم، اعلام ارزش واقعی کالا توسط پیشنهاددهنده ($b = v$) یک استراتژی اکیداً مسلط^{۱۱} است و هیچ عاملی نمی‌تواند با بیش‌برآورد یا کم‌برآورد سود خود را افزایش دهد.

Dominant Strategy^۸

اثبات نظریه بازی‌ها: فرض کنید بالاترین پیشنهاد سایر رقبا برابر با p باشد:

۱. اگر بازیکن بیش‌اظهاری کند ($b > v$): تنها زمانی نتیجه تغییر می‌کند که پیشنهاد رقیب در بازه $v < p < b$ قرار گیرد. در این حالت، بازیکن برنده حراج می‌شود اما باید مبلغ p را بپردازد که از ارزش واقعی کالا (v) بیشتر است؛ یعنی سود خالص منفی ($v - p < 0$) نصیبش می‌گردد.

۲. اگر بازیکن کم‌اظهاری کند ($b < v$): تنها زمانی نتیجه تغییر می‌کند که پیشنهاد رقیب در بازه $b < p < v$ قرار گیرد. در این حالت، بازیکن فرصت برنده شدن در حراجی با سود خالص مثبت ($v - p > 0$) را از دست می‌دهد.

^۸ Cost Per Click^۸

^۹ Click-Through Rate^۹

^{۱۰} Generalized Second-Price (GSP) Auction^{۱۰}

بنابراین، در تمامی حالات، ارائه پیشنهاد برابر با ارزش واقعی ($b = v$) بهترین نتیجه ممکن را برای شرکت‌کننده تضمین می‌نماید.

نکته امتحانی و کلیدی

نکات پرکاربرد امتحانی در مبحث تبلیغات وب:

۱. کران پایینی الگوریتم حریصانه همیشه $1/2$ است و بدترین حالت آن زمانی رخ می‌دهد که یک تبلیغ‌دهنده مشترک میان دو پرس‌وجو، با اولین پرس‌وجو اشباع شود و پرس‌وجوی دوم بی‌مشتري بماند.
۲. عدد طلایی $1 - 1/e$ حد بالای تئوریک الگوریتم‌های برخط بدون اطلاع از توزیع ورودی‌ها است.

جمع‌بندی فصل هشتم

- الگوریتم‌های برخط باید بدون آگاهی از رخدادهای آینده و به صورت آنی تصمیم‌گیری کنند.
- نسبت رقابتی عملکرد الگوریتم برخط را با سناریوی بهینه آفلاین مقایسه می‌نماید.
- الگوریتم حریصانه نسبت رقابتی 50% و الگوریتم BALANCE نسبت رقابتی $63/2\%$ ($1 - 1/e$) را تضمین می‌کنند.
- سیستم AdWords با ترکیب پیشنهادها، کسری بودجه، نرخ کلیک و حراج قیمت دوم، پایداری اقتصادی موتورهای جستجو را رقم می‌زند.

فصل ۹

سامانه‌های توصیه‌گر (Recommender Systems)

۱.۹ مقدمه و پدیده دنباله طولانی

در دنیای فیزیکی، فروشگاه‌ها (نظیر کتاب‌فروشی‌ها یا سینماها) به دلیل محدودیت فضای قفسه‌ها، ناگزیرند تنها محبوب‌ترین و پرفروش‌ترین کالاها را عرضه کنند (پدیده فرهنگ عامه‌پسند یا پرفروش‌ها)^۱. اما در اقتصاد دیجیتال، وب‌سایت‌هایی مانند آمازون، نتفلیکس یا اسپاتیفای دارای هزینه‌های انبارداری و تکثیر نزدیک به صفر هستند؛ در نتیجه می‌توانند میلیون‌ها کالای خاص و کمیاب را نیز ارائه نمایند. پدیده دنباله طولانی^۲ نشان می‌دهد که اگرچه تقاضا برای هر کالای خاص موجود در انتهای دنباله اندک است، اما **مجموع تقاضای تمامی اقلام دنباله طولانی** می‌تواند از تقاضای کالاها پرفروش سنتی فراتر رود. با این حال، در میان میلیون‌ها گزینه، کاربر چگونه کالای مورد علاقه خود را پیدا کند؟ اینجاست که **سامانه‌های توصیه‌گر**^۳ به عنوان موتور محرک اقتصاد وب ظاهر می‌شوند.

۲.۹ ماتریس سودمندی (Utility Matrix)

ستون فقرات ریاضی هر سامانه توصیه‌گر، **ماتریس سودمندی**^۴ است:

تعریف ۱.۹: ماتریس سودمندی

ماتریس M با ابعاد $|U| \times |I|$ تعریف می‌شود که در آن سطرها کاربران (U)، ستون‌ها آیتم‌ها یا کالاها (I)، و درایه‌های r_{ui} بیانگر درجه علاقه یا امتیاز کاربر u به آیتم i هستند (مثلاً امتیازی بین ۱ تا ۵ ستاره).

ویژگی بارز ماتریس سودمندی در کلان‌داده، **تنک بودن فوق‌العاده**^۵ آن است: یک کاربر معمولی ممکن است از میان ده‌ها هزار فیلم موجود در نتفلیکس، تنها چند ده فیلم را تماشا کرده و امتیاز داده باشد؛ بنابراین بیش از ۹۹٪ درایه‌های ماتریس خالی (مجهول) هستند.

هدف بنیادین سامانه توصیه‌گر: پیش‌بینی دقیق مقادیر درایه‌های خالی ماتریس سودمندی و پیشنهاد آیتم‌هایی که بالاترین امتیاز تخمینی را دارند به هر کاربر.

۳.۹ روش‌های مبتنی بر محتوا (Content-Based Recommendation)

در این رویکرد، توصیه بر اساس ویژگی‌های ذاتی کالاها و تاریخچه علایق گذشته همان کاربر انجام می‌پذیرد.

۱.۳.۹ پروفایل آیتم‌ها و پروفایل کاربر

۱. **پروفایل آیتم (Item Profile):** هر کالا با یک بردار ویژگی توصیف می‌شود. برای مثال، برای یک فیلم، بردار شامل ژانرها (اکشن، کمدی)، نام کارگردان، بازیگران، و وزن کلمات کلیدی خلاصه فیلم (به کمک معیار TF-IDF) است.

^۱Blockbuster Culture^۱

^۲Long Tail^۲

^۳Recommender Systems^۳

^۴Utility Matrix^۴

^۵Extreme Sparsity^۵

۲. **پروفایل کاربر (User Profile):** سلیقه کاربر u با ترکیب بردارهای ویژگی آیت‌هایی که به آن‌ها امتیاز بالا داده است، به دست می‌آید:

$$w_u = \frac{\sum_{i \in I_u} r_{ui} \cdot x_i}{\sum_{i \in I_u} r_{ui}} \quad (1.9)$$

که در آن x_i بردار ویژگی آیت i و r_{ui} امتیاز کاربر به آن است.

۳. **تخمین امتیاز:** میزان تناسب آیت جدید j برای کاربر u با استفاده از **تشابه کسینوسی** میان بردار کاربر و بردار آیت سنجیده می‌شود:

$$\text{Score}(u, j) = \cos(w_u, x_j) = \frac{w_u \cdot x_j}{\|w_u\| \|x_j\|} \quad (2.9)$$

نکته امتحانی و کلیدی

مزایا و معایب فیلترینگ محتوا محور:

- **مزیت:** نیازی به داده‌های سایر کاربران ندارد؛ برای آیت‌های تازه اضافه شده به سیستم مشکل راه‌اندازی سرد ایجاد نمی‌کند.
- **عیب بزرگ:** **بیش‌تخصصی شدن^a** و ایجاد «حباب فیلتر»؛ سامانه هرگز کالایی خارج از دایره واژگان یا سبک گذشته کاربر را به او پیشنهاد نمی‌دهد (فقدان شانس کشف غیرمنتظره یا Serendipity).

Overspecialization^a

۴.۹ فیلترینگ مشارکتی (Collaborative Filtering)

فیلترینگ مشارکتی نیازی به استخراج محتوا و ویژگی‌های متنی آیت‌ها ندارد. در عوض، بر پایه این فرض عمل می‌کند: «افرادی که در گذشته سلیقه و امتیازدهی یکسانی داشته‌اند، در آینده نیز نظرات مشابهی خواهند داشت».

۱.۴.۹ کسینوس مرکزی (همبستگی پیرسون)

برای مقایسه دو کاربر یا دو آیت، نمی‌توان از تشابه کسینوسی معمولی استفاده کرد؛ زیرا کاربران سخت‌گیر میانگین نمراتشان پایین (مثلاً ۲ از ۵) و کاربران سهل‌گیر میانگین نمراتشان بالاست (مثلاً ۴ از ۵). راهکار علمی، استفاده از **کسینوس مرکزی^۶** یا ضریب همبستگی پیرسون است که در آن میانگین امتیازات هر کاربر از نمرات او کسر می‌شود.

مثال کاربردی ۱.۹: محاسبه کسینوس مرکزی برای دو کاربر

فرض کنید امتیاز دو کاربر A و B به چهار فیلم به شرح زیر باشد (علامت - به معنای عدم مشاهده است):

کاربر	فیلم ۱	فیلم ۲	فیلم ۳	فیلم ۴
A	۴	۵	۳	-
B	۲	۳	-	۵

میانگین امتیازات کاربر A : $\bar{r}_A = (4 + 5 + 3) / 3 = 4$. میانگین امتیازات کاربر B : $\bar{r}_B = (2 + 3 + 5) / 3 = 3.33$.
با نرمال‌سازی و کسر میانگین (تنها بر روی فیلم‌های مشترک ۱ و ۲):

$$\tilde{A} = [4 - 4, 5 - 4] = [0, 1] \quad (3.9)$$

$$\tilde{B} = [2 - 3.33, 3 - 3.33] = [-1.33, -0.33] \quad (4.9)$$

کسینوس مرکزی:

$$\text{sim}(A, B) = \frac{0 \times (-1/33) + 1 \times (-0/33)}{\sqrt{0^2 + 1^2} \sqrt{(-1/33)^2 + (-0/33)^2}} = \frac{-0/33}{1 \times 1/33} \approx -0/24 \quad (5.9)$$

(نمره منفی نشان‌دهنده اختلاف سلیقه در فیلم‌های مشترک پس از حذف اثر سخت‌گیری است).

۲.۴.۹ پیش‌بینی امتیاز کاربر به آیتم (فیلترینگ کاربر-کاربر)

برای پیش‌بینی امتیاز کاربر u به آیتم i :

۱. مجموعه N شامل k کاربر شبیه‌ترین به u را که به آیتم i امتیاز داده‌اند، انتخاب می‌کنیم.

۲. امتیاز تخمینی با میانگین‌گیری وزنی بر اساس تشابه کسینوس مرکزی به دست می‌آید:

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N} \text{sim}(u, v) \cdot (r_{vi} - \bar{r}_v)}{\sum_{v \in N} |\text{sim}(u, v)|} \quad (6.9)$$

۳.۴.۹ فیلترینگ آیتم-آیتم در برابر کاربر-کاربر

در عمل (به‌ویژه در آمازون)، فیلترینگ آیتم-آیتم ترجیح داده می‌شود، زیرا:

- ویژگی‌ها و ماهیت کالاها بسیار پایدارتر از سلیقه متغیر انسان‌ها در طول زمان است.
- شباهت میان آیتم‌ها را می‌توان به صورت آفلاین محاسبه و پیش‌محاسبه کرد؛ در نتیجه زمان پاسخگویی برخط بسیار سریع‌تر خواهد بود.

۵.۹ مدل‌های عامل پنهان و فاکتورسازی ماتریسی (Matrix Factorization)

پیشرفته‌ترین روش در فیلترینگ مشارکتی (که برنده جایزه یک میلیون دلاری نتفلیکس شد)، تجزیه ماتریس سودمندی به مؤلفه‌های پنهان^۷ است.

تعریف ۲.۹: تجزیه UV

فرض کنید ماتریس سودمندی تنک M دارای ابعاد $m \times n$ باشد. هدف، تقریب این ماتریس از حاصل ضرب دو ماتریس با رتبه پایین d (که $d \ll \min(m, n)$ است، مثلاً $d = 20$ یا 50) است:

$$M \approx P \cdot Q^T \quad (7.9)$$

که در آن:

- ماتریس P با ابعاد $m \times d$ بردار عوامل پنهان کاربران است (p_u) .
- ماتریس Q با ابعاد $n \times d$ بردار عوامل پنهان آیتم‌ها است (q_i) .

امتیاز تخمینی برابر است با ضرب داخلی: $\hat{r}_{ui} = p_u^T \cdot q_i$

۱.۵.۹ تابع هدف بهینه‌سازی و منظم‌سازی (Regularization)

برای جلوگیری از بیش‌برازش بر روی داده‌های محدود مشاهده‌شده، تابع هزینه به صورت مجموع مربعات خطا به همراه ترم منظم‌سازی L_2 فرمول‌بندی می‌شود:

$$\min_{P, Q} \sum_{(u, i) \in \text{Observed}} (r_{ui} - p_u^T q_i)^2 + \lambda \left(\sum_u \|p_u\|_2^2 + \sum_i \|q_i\|_2^2 \right) \quad (8.9)$$

که در آن λ ضریب منظم‌سازی است.

۲.۵.۹ بهینه‌سازی با گرادیان کاهشی تصادفی (SGD)

برای هر درایه مشاهده‌شده r_{ui} ، خطای پیش‌بینی برابر است با:

$$e_{ui} = r_{ui} - p_u^T q_i \quad (۹.۹)$$

قاعده به‌روزرسانی پارامترها با نرخ یادگیری γ :

$$p_u \leftarrow p_u + \gamma(e_{ui} \cdot q_i - \lambda p_u) \quad (۱۰.۹)$$

$$q_i \leftarrow q_i + \gamma(e_{ui} \cdot p_u - \lambda q_i) \quad (۱۱.۹)$$

این به‌روزرسانی بسیار ساده، سریع و با یک یا چند گذر روی کلان‌داده قابل اجرا است.

۳.۵.۹ مدل پایه با اثرات انحراف (Baseline Predictors)

بسیاری از تغییرات امتیازات ناشی از سلیقه نیست، بلکه ناشی از رفتارهای سیستماتیک است:

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u^T q_i \quad (۱۲.۹)$$

که در آن:

- μ : میانگین کل امتیازات در سرتاسر پایگاه داده.
- b_u : انحراف کاربر u از میانگین (سخت‌گیر یا سهل‌گیر بودن).
- b_i : انحراف کیفیت آئتم i از میانگین (فیلمی فوق‌العاده یا ضعیف).

مقایسه روش‌های بهینه‌سازی SGD و ALS در آزمون

- **گرادیان کاهشی تصادفی (SGD)**: در هر گام بر روی یک جفت کاربر-آئتم عمل می‌کند. پیاده‌سازی آن بسیار ساده و همگرایی اولیه آن سریع است، اما موازی‌سازی آن به دلیل هم‌پوشانی متغیرها در خوشه‌های توزیع‌شده چالش‌برانگیز است.
- **کمترین مربعات متناوب (Alternating Least Squares - ALS)**: در هر مرحله یکی از ماتریس‌های P یا Q ثابت نگه داشته شده و مسئله به یک دستگاه معادلات خطی مجزا برای هر سطر تبدیل می‌شود. مزیت بی‌نظیر ALS در سیستم‌های توزیع‌شده این است که کاملاً به صورت موازی (Embarrassingly Parallel) بر روی خوشه‌های MapReduce یا Spark اجرا می‌گردد.

جمع‌بندی فصل نهم

- سامانه‌های توصیه‌گر با بهره‌گیری از پدیده دنباله طولانی، گزینه‌های متناسب با سلیقه فردی را پیشنهاد می‌دهند.
- فیلترینگ محتوای محور بر شباهت ویژگی‌های کالا تکیه دارد اما دچار محدودیت بیش‌تخصیصی شدن است.
- فیلترینگ مشارکتی با بهره‌گیری از کسینوس مرکزی، انحرافات مقیاسی کاربران را خنثی می‌نماید.
- مدل‌های فاکتورسازی ماتریسی (تجزیه UV) با استخراج ابعاد پنهان و بهینه‌سازی SGD، بالاترین دقت پیش‌بینی را در مقیاس وسیع ارائه می‌دهند.

بخش چهارم

تحلیل شبکه‌های اجتماعی و کاهش بعد

فصل ۱۰

کاوش گراف‌های شبکه‌های اجتماعی (Social-Network Graphs)

۱.۱۰ مقدمه و ویژگی‌های توپولوژیک شبکه‌های اجتماعی

با ظهور پلتفرم‌هایی نظیر لینکدین، توییتر و فیسبوک، گراف‌های شبکه‌های اجتماعی به یکی از مهم‌ترین منابع کلان‌داده تبدیل شده‌اند. در این گراف‌ها، گره‌ها بیانگر افراد (یا نهادها) و یال‌ها بیانگر روابط دوستی، تعامل، پیام‌رسانی یا همکاری هستند.

شبکه‌های اجتماعی در مقایسه با گراف‌های تصادفی ساده (مدل اردوش-رنی)، ویژگی‌های توپولوژیک منحصر به فردی دارند:

۱. **پدیده جهان کوچک (Small-World Phenomenon):** میانگین طول کوتاه‌ترین مسیر میان هر دو فرد به طرز شگفت‌انگیزی کوتاه است (معروف به قانون «شش درجه جدایی»).

۲. **توزیع درجه قانون توانی:** توزیع درجات گره‌ها نامتقارن و دارای دنباله سنگین است؛ اکثر افراد روابط محدودی دارند، اما تعداد انگشت‌شماری «ابرگره» دارای میلیون‌ها ارتباط هستند.

۳. **ضریب خوشه‌بندی بالا (High Clustering Coefficient):** اگر فرد A با B و C دوست باشد، احتمال دوستی میان B و C بسیار بالاست (بسته‌شدن سه‌گانه^۱).

۴. **ساختار جوامع (Community Structure):** شبکه از گروه‌های مترادفی تشکیل شده است که درون هر گروه ارتباطات فراوان، اما میان گروه‌ها پیوندهای محدودی وجود دارد.

۲.۱۰ تشخیص جوامع و مرکزیت بینابینی یال‌ها

چگونه می‌توان جوامع درون یک شبکه بزرگ را به صورت الگوریتمی کشف کرد؟

تعریف ۱.۱۰: مرکزیت بینابینی یال (Edge Betweenness)

مرکزیت بینابینی یک یال e برابر است با تعداد کوتاه‌ترین مسیرها میان تمامی جفت‌گره‌های شبکه که از روی یال e عبور می‌کنند:

$$C_B(e) = \sum_{s \neq t} \frac{\sigma_{st}(e)}{\sigma_{st}} \quad (1.10)$$

که در آن σ_{st} کل تعداد کوتاه‌ترین مسیرهای میان گره‌های s و t ، و $\sigma_{st}(e)$ تعداد مسیرهایی است که از یال e می‌گذرند.

بینش شهودی: یال‌هایی که درون یک جامعه قرار دارند دارای بینابینی پایینی هستند زیرا مسیرهای جایگزین متعددی درون جامعه وجود دارد؛ در مقابل، یال‌هایی که مانند یک «پل» دو جامعه مجزا را به هم متصل می‌کنند، دارای بالاترین مرکزیت بینابینی خواهند بود.

^۱ Triadic Closure

۱.۲.۱۰ الگوریتم گیروان-نیومن (Girvan-Newman Algorithm)

الگوریتم Girvan-Newman یک روش خوشه‌بندی سلسله‌مراتبی تقسیمی (بالا به پایین) است:

الگوریتم ۱.۱۰: الگوریتم گیروان-نیومن (Girvan-Newman)

۱. محاسبه بینابینی تمام یال‌ها: به ازای هر گره مبدأ X :

- **پیمایش رو به پایین (BFS):** یک درخت جستجوی اول سطح (DAG) تشکیل دهید. به گره ریشه مقدار ۱ اختصاص دهید. به هر گره در سطوح بعدی، عددی برابر با مجموع مقادیر والدین مستقیم آن نسبت داده می‌شود (شمارش تعداد کوتاه‌ترین مسیرها از ریشه).

- **پیمایش رو به بالا (محاسبه جریان):** از برگ‌های درخت آغاز کنید. هر گره یک واحد جریان به عنوان وزن خود در نظر می‌گیرد و جریان عبوری از هر یال ورودی به سمت والد P متناسب با سهم والد در تعداد مسیرها $\left(\frac{\text{Score}(P)}{\text{Score}(Y)}\right)$ محاسبه می‌شود. هر گره مجموع جریان‌های دریافتی از فرزندان را به علاوه ۱ کرده و به والدین بالادستی ارسال می‌نماید.

مجموع جریان‌های تمام درخت‌های BFS بر روی تمامی گره‌ها تقسیم بر ۲ (به دلیل بدون جهت بودن یال‌ها)، مقدار دقیق بینابینی هر یال را مشخص می‌کند.

۲. حذف یال بحرانی: یالی که دارای بیشترین مرکزیت بینابینی است از گراف حذف می‌گردد.

۳. به‌روزرسانی و تکرار: بینابینی یال‌های باقیمانده مجدداً محاسبه شده و فرآیند تکرار می‌شود تا ساختار خوشه‌ای مطلوب با بیشینه مدولاریتی حاصل شود.

۳.۱۰ معیار پیمانه‌ای (Modularity)

برای پاسخ به این پرسش که «تقسیم‌بندی شبکه به جوامع تا چه اندازه باکیفیت و معنادار است؟»، از شاخص پیمانه‌ای^۲ (Q) استفاده می‌شود.

تعریف ۲.۱۰: پیمانه‌ای (Modularity)

پیمانه‌ای کسر یال‌های درون جوامع را با کسر مورد انتظار همان یال‌ها در یک شبکه تصادفی با توزیع درجات یکسان مقایسه می‌کند:

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j) \quad (2.10)$$

که در آن:

- A_{ij} : درایه ماتریس مجاورت (۱ اگر یال باشد، وگرنه ۰).
- k_i, k_j : درجه گره‌های i و j .
- m : مجموع کل یال‌های گراف ($m = \frac{1}{2} \sum k_i$).
- c_i : شناسه جامعه گره i .
- $\delta(c_i, c_j)$: تابع دلتا (۱ اگر هر دو گره در یک جامعه باشند، وگرنه ۰).

مقدار Q همواره در بازه $[-0.5, 1]$ قرار دارد. مقادیر مثبت بزرگ‌تر از ۰/۳ نشان‌دهنده یک ساختار جامعه‌ای بسیار قوی و طبیعی در شبکه هستند.

۴.۱۰ خوشه‌بندی طیفی گراف‌ها (Spectral Clustering)

خوشه‌بندی طیفی یکی از قدرتمندترین ابزارهای جبر خطی در تحلیل گراف است که با استفاده از مقادیر و بردارهای ویژه ماتریس گراف عمل می‌کند.

۱.۴.۱۰ ماتریس لاپلاسیان گراف (Graph Laplacian)

برای گراف بدون جهت $G = (V, E)$:

۱. **ماتریس مجاورت** (A) : ماتریس متقارن با ابعاد $n \times n$.

۲. **ماتریس درجه** (D) : ماتریس قطری که درایه‌های قطر اصلی آن برابر با درجه هر گره است: $D_{ii} = d_i$.

۳. **ماتریس لاپلاسیان گراف** (L) :

$$L = D - A \quad (۳.۱۰)$$

قضیه ۱.۱۰: خواص بنیادین ماتریس لاپلاسیان

۱. ماتریس L همواره متقارن و نیمه‌معین مثبت^a است؛ یعنی تمامی مقادیر ویژه آن حقیقی و نامنفی هستند:

$$0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$$

۲. کوچک‌ترین مقدار ویژه همواره صفر است ($\lambda_1 = 0$) و بردار ویژه متناظر با آن بردار تماماً یک است:

$$L \cdot \mathbf{1} = (D - A)\mathbf{1} = \mathbf{0} \quad (۴.۱۰)$$

۳. **تعداد مؤلفه‌های همبند**: چندگانگی^b مقدار ویژه صفر دقیقاً برابر با تعداد مؤلفه‌های همبند گراف است.

Positive Semi-Definite^a
Multiplicity^b

۲.۴.۱۰ بردار فیدلر (Fiedler Vector) و افراز گراف

اگر گراف همبند باشد، دومین مقدار ویژه اکیداً مثبت است ($\lambda_2 > 0$). بردار ویژه متناظر با λ_2 ، **بردار فیدلر**^۳ نامیده می‌شود.

بردار فیدلر مسئله بهینه‌سازی پیوسته برش گراف را حل می‌کند:

$$\min_{x \perp \mathbf{1}, \|x\|=1} x^T L x = \min_{(i,j) \in E} \sum (x_i - x_j)^2 \quad (۵.۱۰)$$

اگر دو گره با یالی به هم متصل باشند، مؤلفه‌های متناظر آن‌ها در بردار فیدلر تمایل دارند مقادیری بسیار نزدیک به هم داشته باشند.

الگوریتم ۲.۱۰: افراز گراف به کمک بردار فیدلر

۱. ماتریس لاپلاسیان $L = D - A$ را تشکیل دهید.

۲. دومین بردار ویژه کوچک‌تر (x_2) را محاسبه نمایید.

۳. گره‌ها را بر اساس مقدار درایه متناظرشان در x_2 مرتب کنید.

۴. گره‌هایی را که درایه آن‌ها در x_2 مثبت است در یک بخش (A)، و گره‌های با درایه منفی را در بخش دیگر (B) قرار دهید.

جمع‌بندی فصل دهم

- شبکه‌های اجتماعی دارای ویژگی‌های ساختاری متمایز نظیر پدیده جهان کوچک، توزیع قانون توانی و خوشه‌بندی بالا هستند.
- الگوریتم گیروان-نیومن با حذف متوالی یال‌های با بالاترین بینابینی، ساختار جوامع را استخراج می‌کند.
- معیار پیمانه‌ای (Modularity) کیفیت افراز گراف را می‌سنجد.
- خوشه‌بندی طیفی با بهره‌گیری از ماتریس لاپلاسیین ($L = D - A$) و بردار فیدلر، برش بهینه گراف را در ابعاد طیفی میسر می‌سازد.

فصل ۱۱

روش‌های کاهش بعد (Dimensionality Reduction)

۱.۱۱ مقدمه و انگیزه کاهش بعد

در عصر کلان‌داده، اشیاء غالباً با بردارهای بسیار طولانی در فضاهای با ابعاد نجومی (مثلاً 10^4 تا 10^6 بعد) مدل‌سازی می‌شوند؛ مانند ماتریس اسناد در برابر واژگان در پردازش متن، یا ماتریس کاربران در برابر کالاها در وب‌سایت‌های تجارت الکترونیک.

کاهش بعد^۱ فرآیند نگاشت نقاط داده از فضای اصلی d -بعدی به فضایی با بعد بسیار کوچک‌تر $k \ll d$ است، به گونه‌ای که ساختار بنیادین، تنوع هندسی و فواصل میان نقاط داده تا حد ممکن حفظ گردد. دلایل کلیدی کاهش بعد:

۱. کاهش هزینه‌های محاسباتی و حافظه: جایگزینی محاسبات سنگین در ابعاد بالا با عملیات سریع در ابعاد فشرده.
۲. حذف نویز و افزونگی: ابعاد داده معمولاً به شدت با یکدیگر همبسته هستند؛ کاهش بعد بخش‌های نویزی و بی‌اهمیت داده را فیلتر می‌کند.
۳. استخراج معنا و مفاهیم پنهان^۲: ابعاد ترکیبی جدید اغلب مفاهیم کیفی معناداری را آشکار می‌سازند (مانند کشف سبک‌های ادبی یا ژانرهای سینمایی پنهان).
۴. امکان مصورسازی: کاهش ابعاد به ۲ یا ۳ بعد جهت رسم نمودارها و درک شهودی توسط انسان.

۲.۱۱ تجزیه مقادیر منفرد (SVD - Singular Value Decomposition)

تجزیه مقادیر منفرد (SVD) یکی از شاهکارهای جبر خطی کاربردی است که بر روی هر ماتریس مستطیلی دلخواه قابل اعمال است.

قضیه ۱.۱۱: قضیه تجزیه مقادیر منفرد

هر ماتریس حقیقی A با ابعاد $m \times n$ و رتبه r را می‌توان دقیقاً به حاصل ضرب سه ماتریس تجزیه کرد:

$$A = U \cdot \Sigma \cdot V^T \quad (1.11)$$

که در آن:

• U : ماتریس متعامد با ابعاد $m \times r$ که در شرط $U^T U = I$ صدق می‌کند. ستون‌های U ، بردارهای منفرد چپ^۳ (بردارهای ویژه ماتریس متقارن AA^T) هستند.

• Σ : ماتریس قطری با ابعاد $r \times r$ با درایه‌های مثبت بر روی قطر اصلی:

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0 \quad (2.11)$$

^۱ Dimensionality Reduction
^۲ Latent Concepts

این مقادیر، **مقادیر منفرد**^b نامیده می‌شوند و برابر با جذر مقادیر ویژه ماتریس‌های $A^T A$ یا AA^T هستند. V : ماتریس متعامد با ابعاد $n \times r$ که در شرط $V^T V = I$ صدق می‌کند. ستون‌های V ، **بردارهای منفرد راست**^c (بردارهای ویژه ماتریس متقارن $A^T A$) هستند.

Left Singular Vectors^a
Singular Values^b
Right Singular Vectors^c

۱.۲.۱۱ تفسیر هندسی و فیزیکی SVD

در تحلیل داده‌ها، ماتریس A (مثلاً ماتریس کاربران در برابر فیلم‌ها) را می‌توان به صورت نگاشتی از فضای اشیاء به یک «فضای مفاهیم پنهان» تفسیر کرد:

- هر بعد در فضای پنهان متناظر با یک سطر/ستون در Σ است (مثلاً مفهوم ژانر علمی-تخیلی یا رمانس).
- سطر i -ام ماتریس U بیانگر وابستگی کاربر i به مفاهیم پنهان است.
- مقدار منفرد σ_k نشان‌دهنده «قدرت و میزان واریانس پوشش داده‌شده توسط مفهوم k -ام» است.
- سطر j -ام ماتریس V (ستون j -ام V^T) بیانگر میزان تعلق فیلم j به مفاهیم پنهان است.

۲.۲.۱۱ تقریب با رتبه پایین و قضیه اکارت-یانگ-میرسکی

اگر تنها k مقدار منفرد نخست (بزرگ‌ترین مقادیر) را نگه داریم و سایر را صفر کنیم ($k < r$):

$$A_k = U_k \cdot \Sigma_k \cdot V_k^T = \sum_{i=1}^k \sigma_i u_i v_i^T \quad (۳.۱۱)$$

قضیه ۲.۱۱: قضیه اکارت-یانگ-میرسکی (Eckart-Young-Mirsky)

ماتریس A_k ، **بهترین تقریب ممکن با رتبه k برای ماتریس A** در هر دو نرم فروبنیوس و نرم طیفی است:

$$\|A - A_k\|_F = \min_{\text{rank}(B) \leq k} \|A - B\|_F = \sqrt{\sum_{i=k+1}^r \sigma_i^2} \quad (۴.۱۱)$$

این قضیه تضمین می‌کند که خطای کاهش بعد توسط SVD حداقل ممکن در میان تمامی روش‌های خطی است. **معیار حفظ انرژی برای انتخاب رتبه بهینه k :** مجموع مجذور مقادیر منفرد ماتریس، بیانگر کل انرژی یا واریانس فروبنیوس موجود در داده‌ها است:

$$\text{Energy Total} = \|A\|_F^2 = \sum_{i=1}^r \sigma_i^2 \quad (۵.۱۱)$$

در مسائل داده‌کاوی واقعی، رتبه تقریب k معمولاً به گونه‌ای برگزیده می‌شود که حداقل ۸۰ تا ۹۰ درصد از کل انرژی داده‌ها حفظ گردد:

$$\frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2} \geq 0.90 \quad (۶.۱۱)$$

۳.۱۱ تحلیل مؤلفه‌های اصلی (Principal Component Analysis - PCA)

تحلیل مؤلفه‌های اصلی (PCA) به دنبال یافتن راستاهایی در فضا است که بیشترین پراکندگی و واریانس داده‌ها را در امتداد خود ثبت می‌کنند.

الگوریتم ۱.۱۱: مراحل اجرای PCA

۱. **مرکززدایی داده‌ها (Zero-Centering):** میانگین هر ستون را محاسبه کرده و از تمام درایه‌های آن ستون کم می‌کنیم تا میانگین هر بعد صفر شود:

$$X_{\text{centered}} = X - \mathbf{1}\mu^T \quad (۷.۱۱)$$

۲. **ماتریس کوواریانس:** ماتریس کوواریانس نمونه C را محاسبه می‌کنیم:

$$C = \frac{1}{n} X_{\text{centered}}^T X_{\text{centered}} \quad (۸.۱۱)$$

۳. **تجزیه طیفی:** مقادیر و بردارهای ویژه ماتریس کوواریانس را استخراج می‌کنیم:

$$C \cdot v_i = \lambda_i \cdot v_i \quad (۹.۱۱)$$

۴. **انتخاب مؤلفه‌های اصلی:** بردارهای ویژه $v_1, \dots, v_k$ متناظر با بزرگ‌ترین مقادیر ویژه، جهات مؤلفه‌های اصلی را مشخص می‌کنند.

رابطه شگفت‌انگیز میان PCA و SVD: اگر بر روی ماتریس مرکززدایی‌شده X_{centered} مستقیماً تجزیه SVD اعمال شود $(X_{\text{centered}} = U\Sigma V^T)$:

$$C = \frac{1}{n} (V\Sigma U^T)(U\Sigma V^T) = V \left(\frac{\Sigma^2}{n} \right) V^T \quad (۱۰.۱۱)$$

این تساوی نشان می‌دهد که **بردارهای منفرد راست V در SVD دقیقاً همان مؤلفه‌های اصلی در PCA هستند**، و واریانس در طول هر مؤلفه برابر با $\lambda_i = \sigma_i^2/n$ است! در عمل، محاسبات PCA از طریق SVD انجام می‌شود تا از تشکیل ماتریس کوواریانس سنگین اجتناب گردد.

۴.۱۱ تجزیه ماتریسی: CUR حفظ تنک بودن و تفسیرپذیری

اگرچه SVD از نظر ریاضی تقریب بهینه‌ای ارائه می‌دهد، اما در پردازش کلان‌داده با دو نقص عملیاتی بزرگ همراه است:

۱. **تخریب ساختار تنک بودن:** ماتریس اولیه A بسیار تنک است (اکثر درایه‌ها صفرند)، اما ماتریس‌های حاصل از S (U و V) کاملاً چگال (Dense) هستند که ذخیره‌سازی آن‌ها در حافظه غیرممکن است.

۲. **فقدان تفسیرپذیری فیزیکی:** سطرهای U و V ترکیبات خطی عجیبی از کلمات یا کاربران هستند؛ در حالی که متخصصان داده تمایل دارند داده‌ها را بر حسب سطرها و ستون‌های واقعی توصیف کنند.

روش **CUR^۳** برای غلبه بر این دو مشکل طراحی شد:

$$A \approx C \cdot U \cdot R \quad (۱۱.۱۱)$$

که در آن:

- C : شامل c ستون واقعی انتخاب‌شده از خود ماتریس A است.
- R : شامل r سطر واقعی انتخاب‌شده از خود ماتریس A است.
- U : یک ماتریس میانی کوچک با ابعاد $c \times r$ است که به عنوان پل میان C و R عمل می‌کند.

۱.۴.۱۱ الگوریتم انتخاب سطرها و ستون‌ها بر مبنای امتیاز اهرمی

انتخاب ستون‌ها در CUR به صورت تصادفی ساده نیست، بلکه با احتمالی متناسب با مربع نرم ستون انجام می‌شود:

الگوریتم ۲.۱۱: انتخاب ستون‌ها و سطرها در تجزیه CUR

۱. به ازای هر ستون j از ماتریس A ، احتمال انتخاب را متناسب با انرژی فروبنیوس آن ستون تعریف می‌کنیم:

$$p_j = \frac{\sum_i A_{ij}^2}{\|A\|_F^2} \quad (۱۲.۱۱)$$

۲. تعداد c ستون را با جایگذاری بر اساس توزیع احتمال p_j انتخاب کرده و در ماتریس C قرار می‌دهیم.

۳. برای جلوگیری از اربیبی آماری، هر ستون انتخاب‌شده j در ماتریس C بر ضریب $\sqrt{c \cdot p_j}$ تقسیم می‌شود.

۴. به طور کاملاً متقارن، r سطر از A با احتمالات متناسب با انرژی سطر انتخاب شده و در ماتریس R قرار می‌گیرند و بر $\sqrt{r \cdot p_i}$ تقسیم می‌شوند.

۵. **محاسبه ماتریس میانی U :** فرض کنید ماتریس W با ابعاد $c \times r$ اشتراک سطرها و ستون‌های انتخاب‌شده در ماتریس اصلی باشد. ماتریس U برابر با شبه‌وارون مور-پنروز ماتریس W تعریف می‌شود:

$$U = W^+ \quad (۱۳.۱۱)$$

۵.۱۱ تصویرسازی‌های تصادفی و لم جانسون-لیندن اشتراوس (JL Lemma)

اگر مجموعه‌ای از n نقطه در یک فضای بی‌نهایت با بعد بالا $\mathbb{R}^d$ داشته باشیم، آیا می‌توان بدون از دست دادن فواصل میان نقاط، بعد فضا را کاهش داد؟

قضیه ۳.۱۱: لم جانسون-لیندن اشتراوس (Johnson-Lindenstrauss Lemma)

برای هر مجموعه شامل n نقطه در فضای اقلیدسی $\mathbb{R}^d$ و هر خطای مجاز $0 < \epsilon < 1$ ، یک نگاشت خطی تصادفی $f: \mathbb{R}^d \rightarrow \mathbb{R}^k$ وجود دارد، به گونه‌ای که با بعد هدف:

$$k = O\left(\frac{\log n}{\epsilon^2}\right) \quad (۱۴.۱۱)$$

فواصل اقلیدسی میان تمامی جفت نقاط با تقریب $1 \pm \epsilon$ حفظ می‌شود:

$$(1 - \epsilon)\|x - y\|^2 \leq \|f(x) - f(y)\|^2 \leq (1 + \epsilon)\|x - y\|^2 \quad \forall x, y \in X \quad (۱۵.۱۱)$$

نکته امتحانی و کلیدی

نکته اعجاب‌انگیز لم JL که پای ثابت سؤالات امتحانی است: بعد هدف k هیچ ارتباطی با بعد اولیه d ندارد! چه داده‌ها در فضای ۱۰۰ بعدی باشند و چه در فضای یک میلیون بعدی، اگر $n = 10^4$ نقطه داشته باشیم و خطای ۱۰٪ بخواهیم، نگاشت به فضایی با حدود چند صد بعد، تمامی فواصل را تضمین‌شده حفظ می‌کند. ماتریس تصویرسازی تصادفی به سادگی با کشیدن درایه‌ها از یک توزیع نرمال استاندارد گاوسی $\mathcal{N}(0, 1)$ یا متغیرهای برنولی متعادل (± 1) تولید می‌شود.

جمع‌بندی فصل یازدهم

- کاهش بعد راهکاری اساسی برای غلبه بر نفرین ابعاد و کشف ساختارهای پنهان در کلان داده است.
- قضیه SVD هر ماتریس را به مقادیر منفرد تجزیه کرده و طبق قضیه اکارت-یانگ بهترین تقریب با رتبه پایین را ارائه می‌دهد.
- الگوریتم PCA از طریق مؤلفه‌های اصلی، واریانس بیشینه داده‌های نرمال‌شده را استخراج می‌کند.

- تجزیه CUR با برگزیدن سطرها و ستون‌های واقعی، تنک بودن و تفسیرپذیری را حفظ می‌نماید.
- لم جانسون-لیندن اشتراوس اثبات می‌کند که تصویرسازی تصادفی به ابعادی از مرتبه $O(\frac{\log n}{\epsilon^2})$ ، کلیه فواصل اقلیدسی را مستقل از بعد اولیه مصون نگه می‌دارد.

بخش پنجم

یادگیری در مقیاس بزرگ و یادگیری عمیق

فصل ۱۲

یادگیری ماشین در مقیاس بزرگ (Large-Scale Machine Learning)

۱.۱۲ مقدمه و چالش‌های یادگیری ماشین در کلان‌داده

در کاربردهای سنتی یادگیری ماشین، معمولاً با مجموعه داده‌هایی با اندازه متوسط روبرو هستیم که به راحتی در حافظه اصلی یک کامپیوتر بارگذاری می‌شوند. در این شرایط، مدل‌های پیچیده غیرخطی با زمان‌های آموزش چندساعته قابل استفاده هستند. اما در مسائل کلان‌داده، شرایط به کلی دگرگون می‌شود:

- تعداد نمونه‌های آموزشی (N) در مقیاس صدها میلیون یا میلیاردها رکورد است.
- تعداد ابعاد و ویژگی‌ها (d) ممکن است به میلیون‌ها برسد (مانند پیش‌بینی کلیک در تبلیغات یا فیلترینگ هرزنامه).
- الگوریتم‌های با مرتبه زمانی $O(N^2)$ یا $O(N^3)$ (نظیر روش‌های کرنلی سنتی در SVM) عملاً غیرقابل استفاده هستند.
- **قانون طلایی یادگیری ماشین در مقیاس بزرگ:** الگوریتم آموزش باید دارای مرتبه زمانی خطی $O(N)$ باشد و ترجیحاً بتواند با تنها یک گذر (یا تعداد بسیار معدودی گذر) از روی داده‌ها همگرا شود.

۲.۱۲ روش نزدیک‌ترین همسایه در مقیاس بزرگ (k -NN)

در روش نزدیک‌ترین همسایه^۱، هیچ مرحله آموزش اولیه‌ای وجود ندارد (یادگیری تنبل^۲). برای طبقه‌بندی یک نقطه جدید x :

۱. k نقطه از داده‌های آموزشی که کمترین فاصله را تا x دارند پیدا می‌شوند.

۲. برچسب x با رأی اکثریت این k همسایه تعیین می‌گردد.

چالش در کلان‌داده: جستجوی خطی در میان میلیاردها نقطه آموزشی برای هر پرس‌وجو به شدت کند است. راهکار: بهره‌گیری از تکنیک‌های فصل سوم، به‌ویژه **درهم‌سازی حساس به مکان (LSH)** به عنوان یک شاخص پیشرفته؛ بدین ترتیب، بدون نیاز به مقایسه با تمام نقاط، نزدیک‌ترین همسایه‌ها در زمان زیرخطی $O(\log N)$ یا حتی زمان ثابت $O(1)$ بازیابی می‌شوند.

۳.۱۲ طبقه‌بندی خطی و الگوریتم پرسپترون (Perceptron)

در طبقه‌بندی دودویی، داده‌ها به صورت جفت‌های (x_i, y_i) داده شده‌اند که در آن $x_i \in \mathbb{R}^d$ بردار ویژگی و $y_i \in \{-1, +1\}$ برچسب کلاس است. هدف، یافتن یک ابرصفحه جداکننده^۳ است:

$$f(x) = \text{sign}(w \cdot x + b) \quad (۱.۱۲)$$

که در آن w بردار وزن‌ها و b مقدار انحراف (بایاس) است.

^۱ k -Nearest Neighbors (k -NN)

^۲ Lazy Learning

^۳ Separating Hyperplane

الگوریتم ۱.۱۲: الگوریتم پرسپترون

۱. بردار وزن را صفر قرار دهید: $w = 0$ و $b = 0$.

۲. داده‌های آموزشی را به صورت متوالی پیمایش کنید. برای هر نمونه (x_i, y_i) :

• مقدار پیش‌بینی را محاسبه کنید: $\hat{y} = \text{sign}(w \cdot x_i + b)$.

• اگر پیش‌بینی اشتباه بود ($y_i(w \cdot x_i + b) \leq 0$):

$$w \leftarrow w + \eta \cdot y_i \cdot x_i \quad (2.12)$$

$$b \leftarrow b + \eta \cdot y_i \quad (3.12)$$

(که در آن $\eta > 0$ نرخ یادگیری است).

۳. تکرار تا زمان همگرایی کامل یا رسیدن به سقف تکرارها.

قضیه ۱.۱۲: قضیه همگرایی پرسپترون (نوویکف)

اگر داده‌ها به صورت خطی تفکیک‌پذیر^a باشند، یعنی ابرصفحه‌ای با حاشیه اطمینان $\gamma > 0$ وجود داشته باشد به طوری که $y_i(w^* \cdot x_i) \geq \gamma$ به ازای $\|w^*\| = 1$ ، آنگاه الگوریتم پرسپترون حداکثر پس از:

$$M \leq \left(\frac{R}{\gamma} \right)^2 \quad (4.12)$$

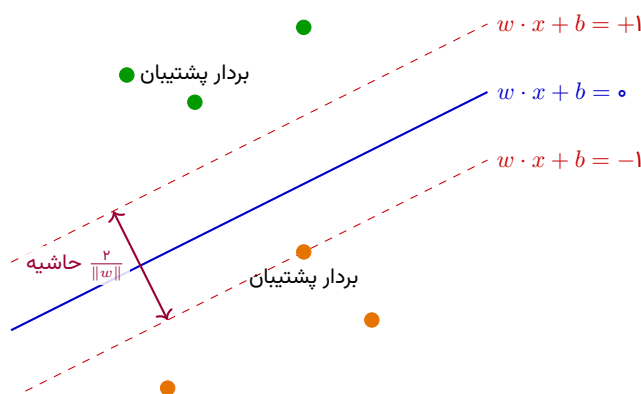
خطا همگرا می‌شود، که در آن $R = \max_i \|x_i\|$ شعاع کوچک‌ترین کره دربرگیرنده داده‌ها است.

Linearly Separable^a

اما اگر داده‌ها خطی تفکیک‌پذیر نباشند، پرسپترون هرگز همگرا نشده و تا ابد دچار نوسان می‌شود.

۴.۱۲ ماشین‌های بردار پشتیبان در مقیاس بزرگ (SVM)

ماشین بردار پشتیبان^f نه تنها به دنبال یک ابرصفحه جداکننده است، بلکه بهترین و مطمئن‌ترین ابرصفحه را جستجو می‌کند؛ یعنی ابرصفحه‌ای که بیشترین فاصله حاشیه^g را از نزدیک‌ترین نقاط داده‌های آموزشی داشته باشد (شکل ۱.۱۲).



شکل ۱.۱۲: ابرصفحه با حاشیه بیشینه در ماشین‌های بردار پشتیبان (SVM)

فاصله حاشیه هندسی برابر با $\frac{2}{\|w\|}$ است. بنابراین، بیشینه‌سازی حاشیه معادل است با کمینه‌سازی $\|w\|^2$.

Support Vector Machine (SVM)^f
Margin^g

۱.۴.۱۲ فرم نرم حاشیه و تابع هزینه لولا (Hinge Loss)

در مسائل واقعی کلان‌داده، داده‌ها نویزی هستند و تفکیک‌پذیری کامل خطی ناممکن است. با تعریف متغیرهای لغزش^۶ $\xi_i \geq 0$ ، به داده‌ها اجازه نقض حاشیه با پرداخت جریمه داده می‌شود:

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i \quad \text{شرط} \quad y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad (۵.۱۲)$$

مقدار بهینه متغیر لغزش دقیقاً برابر با تابع هزینه لولا^۷ است:

$$\xi_i = \max(0, 1 - y_i(w \cdot x_i + b)) \quad (۶.۱۲)$$

بدین ترتیب، مسئله مقید SVM به یک مسئله بهینه‌سازی نامقید با تابع هدف منظم‌شده تبدیل می‌گردد:

$$\min_{w, b} \mathcal{L}(w, b) = \frac{\lambda}{2} \|w\|^2 + \frac{1}{N} \sum_{i=1}^N \max(0, 1 - y_i(w \cdot x_i + b)) \quad (۷.۱۲)$$

که در آن $\lambda = \frac{1}{C \cdot N}$ ضریب منظم‌سازی است.

۵.۱۲ بهینه‌سازی در مقیاس بزرگ با گرادیان کاهش تصادفی (SGD)

الگوریتم گرادیان کاهش استاندارد (Batch Gradient Descent) در هر گام نیازمند محاسبه گرادیان روی تک‌تک N نمونه آموزشی است که برای $N = 10^9$ کاملاً غیرممکن است. در مقابل، گرادیان کاهش تصادفی^۸ در هر گام تنها یک نمونه تصادفی را بررسی کرده و وزن‌ها را فوراً به‌روزرسانی می‌کند.

الگوریتم ۲.۱۲: الگوریتم SGD Pegasos برای آموزش SVM در مقیاس بزرگ

• ورودی: داده‌های آموزشی، پارامتر منظم‌سازی λ ، و تعداد کل گام‌ها T .

• مقداردهی اولیه: $w_1 = 0$.

• حلقه تکرار: برای $t = 1, 2, \dots, T$

۱. یک نمونه تصادفی (x_t, y_t) از میان داده‌ها برگزینید.

۲. نرخ یادگیری در گام جاری را تنظیم کنید: $\eta_t = \frac{1}{\lambda \cdot t}$.

۳. شرط حاشیه را بررسی نمایید:

– حالت اول: نمونه به درستی و خارج از حاشیه قرار دارد ($y_t(w_t \cdot x_t) \geq 1$):

$$w_{t+1} = (1 - \eta_t \lambda) w_t = \left(1 - \frac{1}{t}\right) w_t \quad (۸.۱۲)$$

(تنها انقباض وزنی برای منظم‌سازی اعمال می‌شود).

– حالت دوم: نقض حاشیه یا خطای طبقه‌بندی ($y_t(w_t \cdot x_t) < 1$):

$$w_{t+1/2} = (1 - \eta_t \lambda) w_t + \eta_t y_t x_t \quad (۹.۱۲)$$

۴. تصویرسازی بر روی گوی بهینه (Projection Step): جهت تضمین تثوریک نرخ همگرایی، بردار وزن به درون گوی به شعاع $1/\sqrt{\lambda}$ تصویر می‌شود:

$$w_{t+1} = \min \left(1, \frac{1/\sqrt{\lambda}}{\|w_{t+1/2}\|} \right) w_{t+1/2} \quad (۱۰.۱۲)$$

^۶ Slack Variables

^۷ Hinge Loss

^۸ Stochastic Gradient Descent (SGD)

نکته امتحانی و کلیدی

ویژگی‌های خیره‌کننده SGD برای کلان‌داده:

۱. **استقلال زمان همگرایی از تعداد کل نمونه‌ها:** اثبات شده است که تعداد گام‌های لازم برای رسیدن به خطای ϵ از مرتبه $O(1/\epsilon)$ است و هیچ وابستگی به اندازه کل مجموعه داده N ندارد!
۲. **آموزش در یک گذر (Streaming ML):** داده‌ها می‌توانند به صورت جریانی مستقیماً از روی دیسک یا شبکه خوانده شوند؛ هر نمونه پس از یک بار به‌روزرسانی دور ریخته می‌شود، بدون آنکه نیازی به ذخیره مجدد آن باشد.

۶.۱۲ موازی‌سازی یادگیری ماشین در نگاشت-کاهش

برای توزیع آموزش مدل‌های یادگیری بر روی خوشه‌های چند هزار ماشینی:

- داده‌ها میان M مپر تقسیم می‌شوند.
- هر مپر یک مدل مجزا یا گرادیان‌های محلی بر روی بسته خود (Mini-batch) را محاسبه می‌کند.
- در مرحله کاهش، وزن‌ها یا گرادیان‌ها تجمیع شده و میانگین آن‌ها گرفته می‌شود (الگوریتم Server Parameter AllReduce). یا

جمع‌بندی فصل دوازدهم

- یادگیری ماشین در کلان‌داده نیازمند الگوریتم‌های با پیچیدگی خطی و کمترین تعداد گذر از روی دیسک است.
- تکنیک LSH اجرای سریع روش NN_k را در ابعاد بالا ممکن می‌سازد.
- الگوریتم پرسپترون با تفکیک‌پذیری خطی همگرا می‌شود اما در صورت وجود نویز دچار ناپایداری است.
- ماشین بردار پشتیبان با پیشینه‌سازی حاشیه و استفاده از تابع هزینه لولا، طبقه‌بندی مقاومی ارائه می‌دهد.
- الگوریتم SGD امکان آموزش برخط مدل‌های خطی را در مقیاس میلیاردها نمونه با مرتبه زمانی مستقل از N میسر می‌سازد.

فصل ۱۳

شبکه‌های عصبی و یادگیری عمیق (Neural Networks & Deep Learning)

۱.۱۳ مقدمه و گذار از پرسپترون به یادگیری عمیق

در فصل دوازدهم، دیدیم که یک پرسپترون ساده یا ماشین بردار پشتیبان خطی، قادر به تفکیک داده‌هایی است که با یک ابرصفحه مسطح جدا شوند. با این حال، بسیاری از پدیده‌های دنیای واقعی (نظیر تابع XOR، الگوهای تصویری، پردازش زبان طبیعی و ساختارهای پیچیده شبکه) ذاتاً غیرخطی هستند. ایده شبکه‌های عصبی مصنوعی^۱ الهام‌گرفته از ساختار زیستی مغز انسان است: اگر چندین نرون ساده در قالب لایه‌های متوالی سازمان‌دهی شوند، سیستم قادر به یادگیری نگاشت‌های غیرخطی فوق‌العاده پیچیده خواهد بود.

قضیه ۱.۱۳: قضیه تقریب عام (Universal Approximation Theorem)

یک شبکه عصبی پیش‌خور^a حتی با تنها یک لایه پنهان با تعداد کافی نرون و توابع فعال‌ساز غیرخطی، قادر است هر تابع پیوسته دلخواهی را بر روی یک مجموعه فشرده با هر درجه دقت مطلوبی تقریب بزند.

Feedforward Neural Network^a

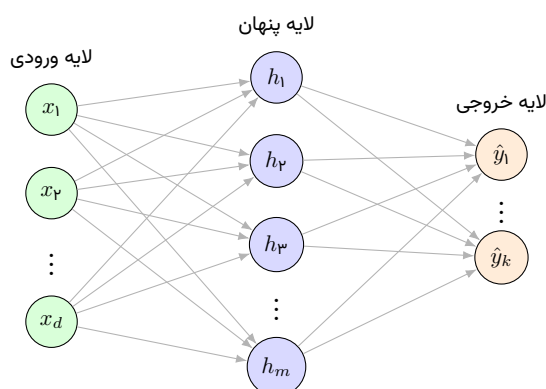
اما چرا «یادگیری عمیق» با چندین لایه پنهان بر شبکه‌های کم‌عمق برتری دارد؟ شبکه‌های عمیق با سلسله‌مراتب لایه‌ای خود، **بازنمایی ویژگی‌ها**^۲ را به صورت خودکار انجام می‌دهند: لایه‌های ابتدایی الگوهای پایه (نظیر لبه‌ها در تصویر)، لایه‌های میانی ترکیبات ساختاری (نظیر چشم و بینی)، و لایه‌های پایانی مفاهیم انتزاعی کامل (نظیر چهره یک فرد) را استخراج می‌کنند؛ امری که نیاز به مهندسی دستی ویژگی‌ها در کلان‌داده را مرتفع می‌سازد.

۲.۱۳ معماری پرسپترون چندلایه (MLP)

یک پرسپترون چندلایه^۳ از سه بخش اصلی تشکیل شده است (شکل ۱.۱۳):

۱. **لایه ورودی (Input Layer):** بردارهای ویژگی خام $x = [x_1, \dots, x_d]^T$ را دریافت می‌کند.
۲. **لایه‌های پنهان (Hidden Layers):** پردازش‌های میانی غیرخطی را انجام می‌دهند.
۳. **لایه خروجی (Output Layer):** پیش‌بینی نهایی (کلاس‌ها یا مقادیر پیوسته) را صادر می‌نماید.

Artificial Neural Networks (ANN)^۱
Representation Learning^۲
Multilayer Perceptron (MLP)^۳



شکل ۱.۱۳: معماری استاندارد پرسپترون چندلایه (Feedforward Multilayer Perceptron)

۱.۲.۱۳ انتشار رو به جلو (Forward Propagation)

در لایه l ، خروجی‌های لایه قبلی ($a^{(l-1)}$) در ماتریس وزن‌ها $W^{(l)}$ ضرب شده و با بردار بایاس $b^{(l)}$ جمع می‌گردند؛ سپس تابع فعال‌ساز $g(\cdot)$ اعمال می‌شود:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)} \quad (۱.۱۳)$$

$$a^{(l)} = g(z^{(l)}) \quad (۲.۱۳)$$

که در آن $a^{(0)} = x$ بردار ورودی اولیه است.

۳.۱۳ توابع فعال‌ساز (Activation Functions)

تابع فعال‌ساز غیرخطی عنصری حیاتی در شبکه عصبی است؛ اگر تابع فعال‌ساز خطی باشد ($g(z) = z$)، ترکیب صدها لایه پنهان نیز چیزی جز یک تبدیل خطی ساده نخواهد بود!

۱. تابع سیگموئید (Sigmoid):

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)) \quad (۳.۱۳)$$

خروجی را به بازه $(0, 1)$ می‌برد که برای احتمالات دودویی عالی است؛ اما برای شبکه‌های عمیق دچار محوشدگی گرادیان^۴ می‌شود زیرا مشتق آن برای مقادیر بزرگ یا کوچک نزدیک به صفر است.

۲. تانژانت هذلولوی (Tanh):

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \quad (۴.۱۳)$$

خروجی متقارن حول صفر در بازه $(-1, +1)$ تولید می‌کند و معمولاً همگرایی سریع‌تری نسبت به سیگموئید دارد.

۳. یک‌سوساز خطی (ReLU - Rectified Linear Unit):

$$\text{ReLU}(z) = \max(0, z) \quad (۵.۱۳)$$

انقلاب یادگیری عمیق: تابع ReLU به دلایل زیر به استاندارد طلایی لایه‌های پنهان تبدیل شده است:

- مشتق آن برای مقادیر مثبت دقیقاً برابر با ۱ است؛ بنابراین هرگز دچار محوشدگی گرادیان در جهت مثبت نمی‌شود.
- محاسبه آن فوق‌العاده سریع و نیازمند یک مقایسه ساده است.
- منجر به تنک شدن خودکار فعال‌سازی نرون‌ها (Sparsity) می‌گردد.

۴. تابع Softmax برای طبقه‌بندی چندکلاسه: در لایه خروجی برای پیش‌بینی احتمال کلاس k از میان C کلاس مختلف:

$$P(y = k | x) = \frac{e^{z_k}}{\sum_{j=1}^C e^{z_j}} \quad (۶.۱۳)$$

۴.۱۳ الگوریتم پس‌انتشار خطا (Backpropagation)

هدف از آموزش شبکه، کمینه‌سازی تابع هزینه خطا (نظیر انتروپی متقاطع)^۵ نسبت به تمامی وزن‌ها و بایاس‌هاست:

$$\mathcal{L} = - \sum_k y_k \log \hat{y}_k \quad (۷.۱۳)$$

الگوریتم پس‌انتشار خطا^۶ با استفاده از قاعده زنجیره‌ای دیفرانسیل^۷، سیگنال خطا را از لایه خروجی به لایه‌های قبلی منتشر می‌کند:

الگوریتم ۱.۱۳: الگوریتم پس‌انتشار خطا

۱. گذر رو به جلو: محاسبه فعال‌سازی تمامی لایه‌ها تا خروجی نهایی.

۲. محاسبه خطای لایه خروجی (L) :

$$\delta^{(L)} = \nabla_a \mathcal{L} \odot g'(z^{(L)}) = \hat{y} - y \quad (۸.۱۳)$$

(برای انتروپی متقاطع و سافت‌مکس، تفاضل پیش‌بینی و برچسب واقعی است).

۳. پس‌انتشار خطا به لایه‌های پنهان $(l = L - 1, \dots, 1)$:

$$\delta^{(l)} = \left((W^{(l+1)})^T \delta^{(l+1)} \right) \odot g'(z^{(l)}) \quad (۹.۱۳)$$

که در آن $\odot$ بیانگر ضرب درایه‌به‌درایه (Hadamard product) است.

۴. محاسبه گرادیان پارامترها:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^T, \quad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)} \quad (۱۰.۱۳)$$

۵. به‌روزرسانی وزن‌ها: با الگوریتم‌های SGD، ممانعت یا Adam:

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial W^{(l)}} \quad (۱۱.۱۳)$$

۵.۱۳ شبکه‌های عصبی پیچشی (CNN) برای داده‌های مکانی

هنگامی که ورودی تصویر است (مثلاً تصویر ۱۰۰۰×۱۰۰۰ پیکسلی)، استفاده از لایه‌های تمام‌متصل نیازمند میلیون‌ها وزن به ازای هر نرون است که باعث انفجار حافظه و بیش‌برازش شدید می‌شود. شبکه‌های پیچشی^۸ با دو نوآوری بنیادین این مشکل را حل می‌کنند:

۱. میدان‌های پذیرش محلی (Local Receptive Fields): هر نرون تنها به پنجره کوچکی از پیکسل‌های مجاور (مثلاً فیلتر ۳×۳ یا ۵×۵) متصل است.

^۵ Cross-Entropy Loss

^۶ Backpropagation

^۷ Chain Rule

^۸ Convolutional Neural Networks (CNN)

۲. **اشتراک‌گذاری وزن‌ها (Weight Sharing):** فیلتر بر روی کل تصویر لغزانده می‌شود و وزن‌های آن برای تمامی موقعیت‌ها یکسان است. این کار تعداد پارامترهای آموزش‌پذیر را از میلیاردها به چند صد پارامتر کاهش داده و **نوردایی نسبت به انتقال**^۹ ایجاد می‌کند.

۳. **لایه‌های ادغام (Pooling):** لایه‌هایی نظیر Pooling Max با انتخاب بیشینه مقدار در پنجره‌های 2×2 ، ابعاد فضایی نقشه ویژگی‌ها را کاهش داده و محاسبات را تسریع می‌بخشند.

۶.۱۳ شبکه‌های بازگشتی (RNN) و ترنسفورمرها برای داده‌های متوالی و متنی

برای داده‌های دنباله‌ای با طول متغیر (مانند متون، جریان لاگ‌ها یا قیمت‌های سهام):

• **شبکه‌های بازگشتی (RNN):** دارای حالت پنهان h_t هستند که به عنوان حافظه عمل کرده و اطلاعات مراحل زمانی گذشته را به گام جاری منتقل می‌کند:

$$h_t = \tanh(Wx_t + Uh_{t-1} + b) \quad (۱۲.۱۳)$$

برای حل مشکل فراموشی در دنباله‌های طولانی، معماری‌های حافظه کوتاه‌مدت ماندگار^{۱۰} و GRU با سازوکارهای دروازه‌ای (Gates) توسعه یافتند.

• **معماری ترنسفورمر و سازوکار توجه (Self-Attention):** در سال‌های اخیر، ترنسفورمرها با پردازش کاملاً موازی (بدون وابستگی زمانی گام‌به‌گام) و محاسبه وزن‌های توجه میان تمامی جفت‌کلمات، استانداردهای مدل‌های زبانی عظیم (LLMs) و تحلیل کلان‌داده‌های متنی را دگرگون ساخته‌اند.

در سازوکار **توجه ضرب داخلی مقیاس‌شده**^{۱۱}، از سه ماتریس پرس‌وجو (Q)، کلید (K) و مقدار (V) استفاده می‌شود:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (۱۳.۱۳)$$

ضریب مقیاس‌بندی $\frac{1}{\sqrt{d_k}}$ نقشی حیاتی دارد: در ابعاد بالا، حاصل ضرب داخلی بردارها رشد نمایی پیدا کرده و خروجی تابع Softmax را به نواحی با گرادیان بی‌نهایت کوچک (محوشدگی گرادیان) می‌راند؛ این ضریب واریانس را تثبیت می‌کند.

همچنین در **توجه چندسری**^{۱۲}، نمایش‌های مستقل چندگانه به صورت هم‌زمان آموخته می‌شوند:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (۱۴.۱۳)$$

که در آن $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$ است.

۷.۱۳ زیرساخت‌های توزیع‌شده برای آموزش یادگیری عمیق در مقیاس بزرگ

آموزش مدل‌های عمیق بر روی ترابایت‌ها داده بدون توزیع محاسبات ناممکن است:

• **موازی‌سازی داده (Data Parallelism):** یک نسخه از کل مدل روی چندین شتاب‌دهنده گرافیکی (GPU) بارگذاری می‌شود. هر GPU یک بخش از دسته داده (Mini-batch) را پردازش کرده و گرادیان‌ها با پروتکل پرسرعت Ring-AllReduce با یکدیگر همگام می‌شوند.

• **موازی‌سازی مدل (Model Parallelism):** لایه‌های مختلف یک مدل غول‌پیکر میان چندین کارت گرافیکی افراز می‌گردند.

^۹ Translation Invariance

^{۱۰} Long Short-Term Memory (LSTM)

^{۱۱} Scaled Dot-Product Attention

^{۱۲} Multi-Head Attention

جمع‌بندی فصل سیزدهم

- شبکه‌های عصبی عمیق با یادگیری سلسله‌مراتبی ویژگی‌ها، تفکیک غیرخطی داده‌های کلان را میسر می‌سازند.
- تابع فعال‌ساز ReLU با ممانعت از محوشدگی گرادیان، ستون فقرات یادگیری عمیق است.
- الگوریتم پس‌انتشار خطا گرادیان‌های تحلیلی را با قاعده زنجیره‌ای برای به‌روزرسانی وزن‌ها محاسبه می‌کند.
- شبکه‌های CNN با اشتراک وزن‌ها، داده‌های تصویری و مکانی را به صورت مقیاس‌پذیر پردازش می‌نمایند.
- شبکه‌های RNN و ترنسفورمرها پردازش دنباله‌ها و متون عظیم را محقق می‌سازند.
- موازی‌سازی داده با خوشه‌های GPU آموزش شبکه‌های عصبی عمیق بر روی کلان‌داده را عملیاتی می‌نماید.

کتابنامه

- [۱] In rules. association mining for algorithms Fast al. et Srikant, Ramakrishnan Agrawal, Rakesh [۱]
۱۹۹۴، ۴۹۹-۴۸۷ pages، ۱۲۱۵ volume VLDB، bases، data large very conf. int. th۲۰ Proc.
- [۲] Communications errors. allowable with coding hash in trade-offs Space/time Bloom. H Burton [۲]
۱۹۷۰، ۴۲۶-۴۲۲:(۷)۱۳ ACM، the of
- [۳] databases. large to algorithms clustering Scaling Reina. Cory and Fayyad، Usama Bradley، S Paul [۳]
۱۹۹۸، ۱۵-۹ pages، ۹۸ volume KDD، In
- [۴] Com- and Compression In documents. of containment and resemblance the On Broder. Z Andrei [۴]
۱۹۹۷ IEEE، ۲۹-۲۱ pages Proceedings، ۱۹۹۷ Sequences of plexity
- [۵] over statistics stream Maintaining Motwani. Rajeev and Indyk، Piotr Gionis، Aristides Datar، Mayur [۵]
۲۰۰۲، ۱۸۱۳-۱۷۹۴:(۶)۳۱ Computing، on Journal SIAM windows. sliding
- [۶] clusters. large on processing data simplified Mapreduce: Ghemawat. Sanjay and Dean Jeffrey [۶]
۲۰۰۸، ۱۱۳-۱۰۷:(۱)۵۱ ACM، the of Communications
- [۷] applications. base data for algorithms counting Probabilistic Martin. Nigel G and Flajolet Philippe [۷]
۱۹۸۵، ۲۰۹-۱۸۲:(۲)۳۱ Sciences، System and Computer of Journal
- [۸] large for algorithm clustering efficient an Cure: Shim. Kyuseok and Rastogi، Rajeev Guha، Sudipto [۸]
of Management on conference international SIGMOD ACM ۱۹۹۸ the of Proceedings In databases.
۱۹۹۸، ۸۴-۷۳ pages data،
- [۹] (JACM)، ACM the of Journal environment. hyperlinked a in sources Authoritative Kleinberg. M Jon [۹]
۱۹۹۹، ۶۳۲-۶۰۴:(۵)۴۶
- [۱۰] Cam- Datasets. Massive of Mining Ullman. David Jeffrey and Rajaraman، Anand Leskovec. Jure [۱۰]
۲۰۲۰ edition، rd۳ Press. University bridge
- [۱۱] ranking: citation pagerank The Winograd. Terry and Motwani، Rajeev Brin، Sergey Page، Lawrence [۱۱]
۱۹۹۹ web. the to order Bringing